

Data Mining and Big data anlysis

数据挖掘及大数据分析

Chpter 4：数据仓库与联机分析处理

王东、张谦、丁玥

上海交通大学软件学院RFID与物联网实验室

2025年春季

数据服务与数据分析

- **数据服务与OLTP**
  - **数据服务**
    - ✓ 主要是指面向各种操作型业务，提供数据的增加、删除、修改、简单查询功能的服务
    - ✓ 提供数据服务功能的系统就是数据服务系统，包括支撑操作型业务的关系数据库、NoSQL数据库以及NewSQL数据库
  - **OLTP（on-line transaction processing，联机事务处理）**
    - ✓ OLTP主要用来记录某类业务事件的发生，如购买行为，当行为产生后，系统会记录是谁在何时何地做了何事，这样的一行（或多行）数据会以增删改的方式在数据库中进行数据的更新处理操作，要求实时性高、稳定性强、确保数据及时更新成功，常见的企业业务系统如ERP、CRM、OA等系统都属于OLTP
    - ✓ OLTP要求的响应时间一般是几秒钟时间，比如在银行的存款、取款、查账、转账等业务中

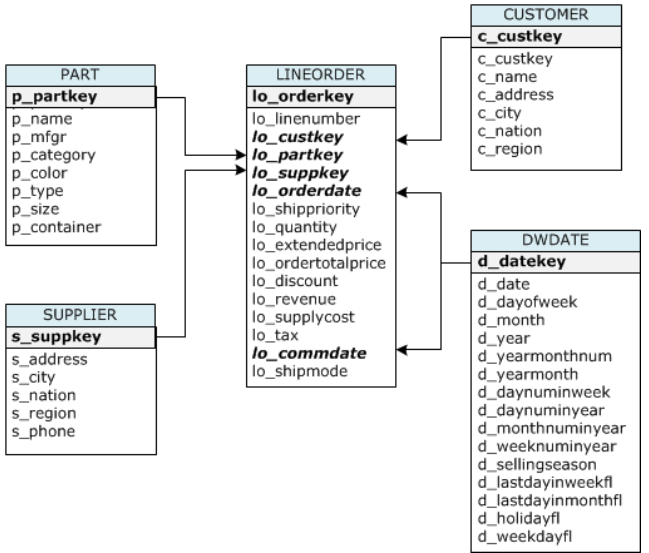
数据服务与数据分析

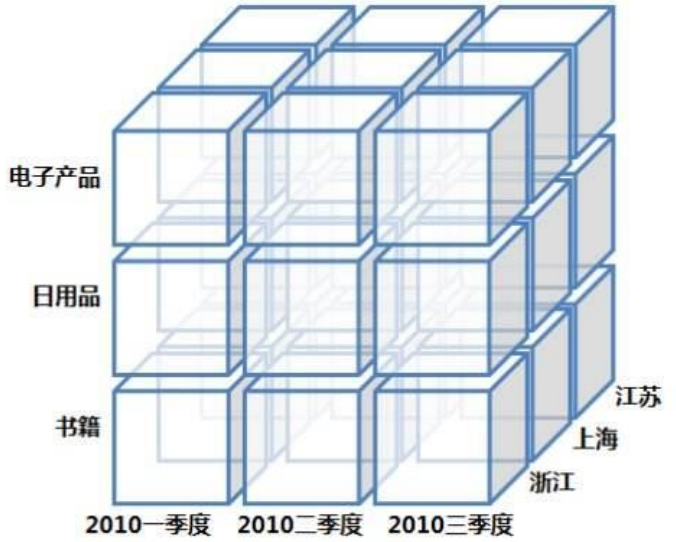
- **数据分析、简单分析、复杂分析**
  - 数据分析
    - ✓ 大量的业务系统采用数据库来进行数据管理以后，随着业务的不断发展，积攒了大量的业务数据。**数据分析任务是面向分析人员（或者管理层），通过对大量数据（甚至在整个数据库数据上）使用复杂的计算模型进行分析，从而发现数据中隐藏的模式或规律**，侧重对决策人员和高层管理人员的决策支持，可以根据分析人员的要求快速、灵活地进行大数据量的复杂查询处理，并且以一种直观而易懂的形式将查询结果提供给决策人员，以便准确掌握企业（公司）的经营状况并进行宏观决策。**这类任务的响应时间一般比较长，比如以分钟和小时计**
    - ✓ **为了避免这类数据处理任务对业务顺利运行产生干扰，一般在业务数据库之外建立数据仓库系统。它从业务数据库抽取、转换和装载(Extract Transform and Load, 简称ETL)数据，帮助人们在上面执行各种分析任务**



数据服务与数据分析

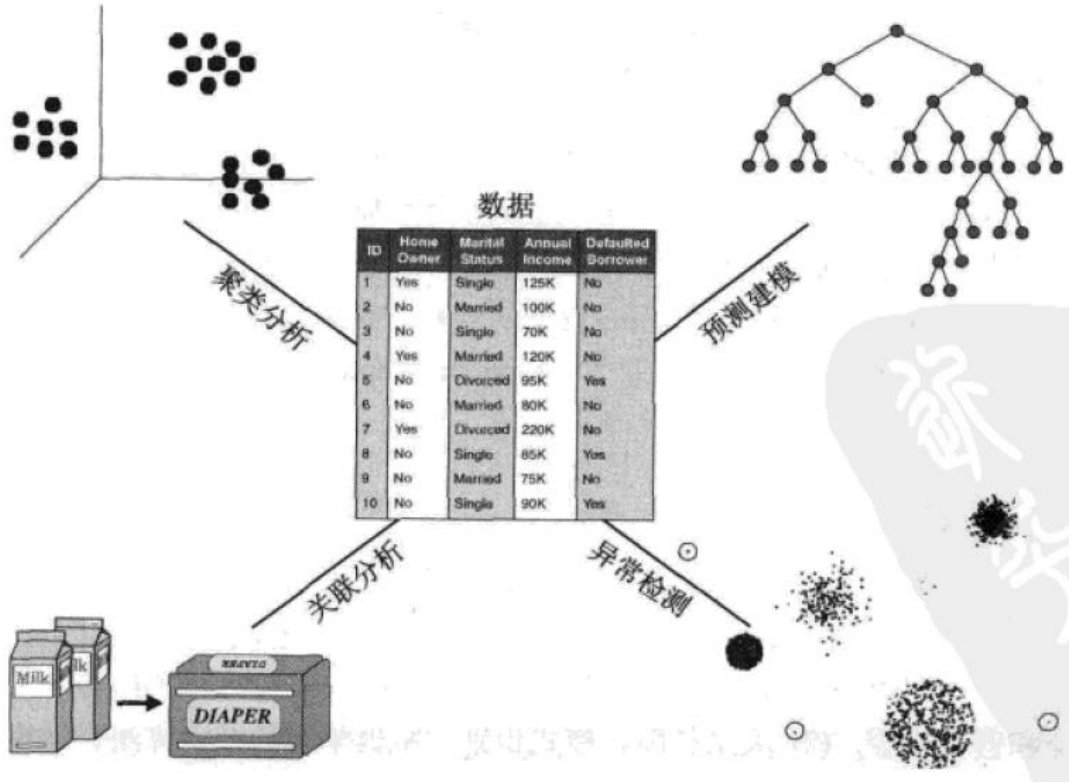
- 数据分析、简单分析、复杂分析
  - 简单分析与OLAP（Online Analytic Processing，联机分析处理）
    - ✓ 简单分析是简单汇总和报表。OLAP采用星型模型或者数据立方体组织数据，保存在数据仓库中，然后进行多维度的聚集查询处理
    - ✓ 在OLAP应用中，在事实表中记录一系列的事件，在维表中记录事件的维度，然后从维表中选取一些维度对事实表进行汇总操作，这些汇总操作包括计数、求和、求平均值、求最大值、求最小值
    - ✓ 将OLAP分析所用到的多维数据，物理上存储为多维数组形式，形成“立方体”(Cube)的结构。星型模型的各个维的属性值，被映射成多维数组的下标值或下标的范围，而聚集数据作为多维数组的值存储在数组的单元中





数据服务与数据分析

- 数据分析、简单分析、**复杂分析**
  - **复杂分析与数据挖掘、机器学习**
    - ✓ 复杂分析一般需要利用统计分析方法、数据挖掘与机器学习方法，**对数据进行深入分析**，提取有用信息并形成结论，辅助决策



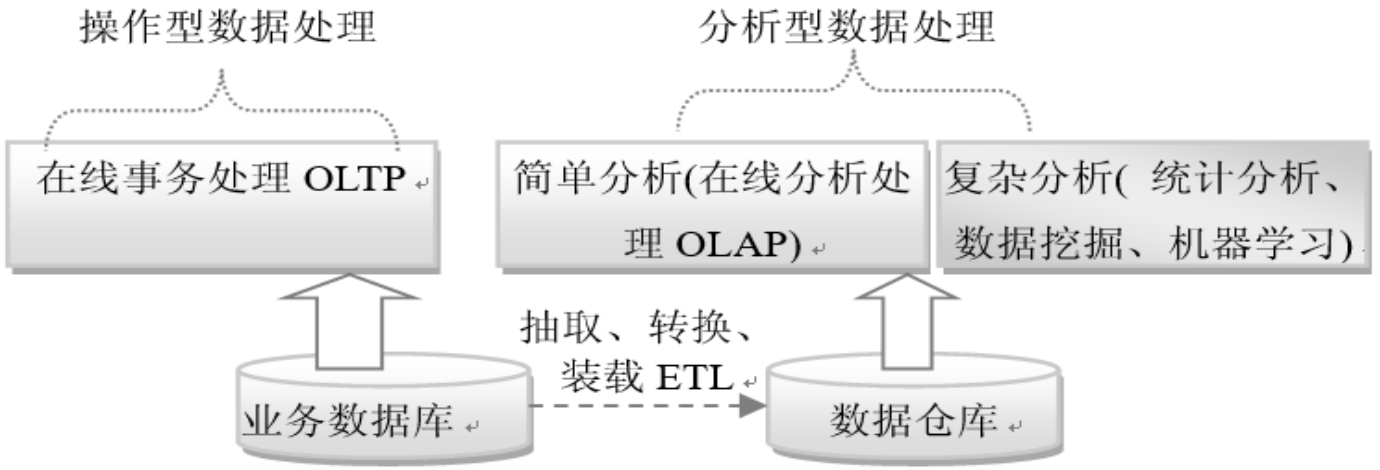
ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	80K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

数据服务与数据分析

- 数据服务与数据分析的区别
  - 数据服务一般在一个事务中，只需存取少量的数据记录，目的是反映操作型业务对数据的修改，比如新建订单、为订单付款、账户余额的扣减等
  - 数据分析则往往要在大量数据上（可以是整个数据集），进行统计汇总以及更深层次的分析操作，目的是从大量数据中获得少量的指标和分析结果，以便从总体上对数据中隐藏的规律有所认识
- 在用户和数据处理系统进行一次交互的过程中，用户使用到的功能可能囊括数据服务、简单数据分析和复杂数据分析

数据仓库与数据挖掘的关系

- 构造数据仓库涉及数据清理、数据集成和数据变换
  - — 可以看做数据挖掘的一个重要预处理步骤
- 数据仓库泛化、合并多维空间的数据，提供联机分析处理OLAP（Online Analytical Processing） 工具，用于不同粒度的数据分析，有利于在此之上进行关联、分类、聚类等
- 构造数据仓库和OLAP已经成为知识发现过程的基本步骤
  - — 许多其他数据挖掘功能，如关联、分类、预测和聚类，都可以与OLAP操作集成，以加强多个抽象层上的交互知识挖掘。数据仓库已经成为数据分析和联机数据分析处理的日趋重要的平台，并将为数据挖掘提供有效的平台



Chapter 5: Data Warehousing and On-line Analytical Processing

- **Data Warehouse: Basic Concepts**
  - (a) What Is a Data Warehouse?
  - (b) Data Warehouse: A Multi-Tiered Architecture
  - (c) Three Data Warehouse Models: Enterprise Warehouse, Data Mart, ad Virtual Warehouse
  - (d) Extraction, Transformation and Loading
  - (e) Metadata Repository
- **Data Warehouse Modeling: Data Cube and OLAP**
  - (a) Cube: A Lattice of Cuboids
  - (b) Conceptual Modeling of Data Warehouses
  - (c) Stars, Snowflakes, and Fact Constellations: Schemas for Multidimensional Databases
  - (d) Dimensions: The Role of Concept Hierarchy
  - (e) Measures: Their Categorization and Computation
  - (f) Cube Definitions in Database systems
  - (g) Typical OLAP Operations
  - (h) A Starnet Query Model for Querying Multidimensional Databases
- **Data Warehouse Design and Usage**
  - (a) Design of Data Warehouses: A Business Analysis Framework
  - (b) Data Warehouses Design Processes
  - (c) Data Warehouse Usage
  - (d) From On-Line Analytical Processing to On-Line Analytical Mining
- **Data Warehouse Implementation**
  - (a) Efficient Data Cube Computation: Cube Operation, Materialization of Data Cubes, and Iceberg Cubes
  - (b) Indexing OLAP Data: Bitmap Index and Join Index
  - (c) Efficient Processing of OLAP Queries
  - (d) OLAP Server Architectures: ROLAP vs. MOLAP vs. HOLAP
- **Summary**

Chapter 5: Data Warehousing and On-line Analytical Processing

- **Data Warehouse: Basic Concepts**
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

What is a Data Warehouse?

什么是 数据仓库？

数据库与数据仓库简史

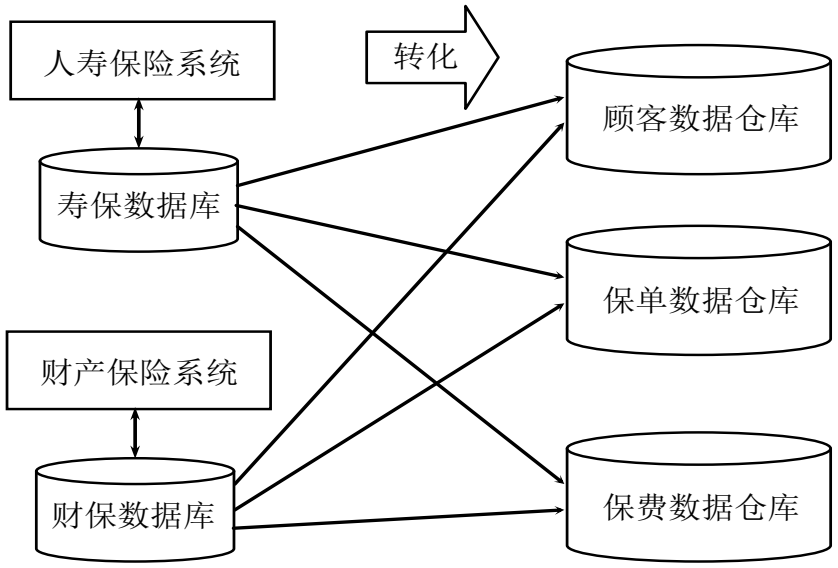
- 20世纪60年代出现了数据库的概念，确立了数据库系统的许多概念、方法和技术
- 70年代由E.F.Codd提出了数据库的关系理模型，开创了数据库关系方法和关系数据理论的研究，为关系数据库技术奠定了理论基础
- 80年代出现成熟的关系数据库管理系统（DBMS）
- 1988年，IBM公司第一次提出了信息仓库（Information Warehouse）的概念，数据仓库初具雏形
- 90年代以后进入数据处理大发展时期，各种数据模型、数据库新技术层出不穷地涌现，如数据仓库和数据挖掘、商务智能、多媒体数据库和Web数据库等
  - 1991年，W.H.Inmon出版了《Building Data Warehouse》一书。W.H.Inmon主张建立数据仓库时采用自上而下方式，以第3范式进行数据仓库模型设计
  - 1993年，R.Kimball（金博尔）出版了《The DataWarehouse Toolkit》一书，认同了比尔恩门对于数据仓库的定义，但对具体的构建方法做了更进一步的研究。R.Kimball主张自下而上的方式，力推数据集市建设
  - .....

What is a Data Warehouse?

- Defined in many different ways, but not rigorously.
  - A decision support database that is maintained separately from the organization's operational database
  - 它是一个提供决策支持功能的数据库，它与公司的操作数据库分开维护
  - Support information processing by providing a solid platform of consolidated, historical data for analysis.
  - 为统一的历史数据分析提供坚实的平台，对信息处理提供支持
- "A data warehouse is a subject-oriented, integrated, time-variant, and nonvolatile collection of data in support of management's decision-making process."—W. H. Inmon
- "数据仓库是一个 面向主题的、集成的、随时间而变化的、不容易丢失的数据集合，支持管理部门的决策过程."— W. H. Inmon
- Data warehousing
  - The process of constructing and using data warehouses

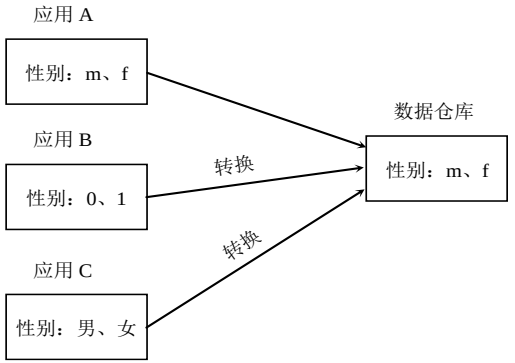
Data Warehouse—Subject-Oriented

- **面向主题，是数据仓库显著区别于关系数据库系统的一个特征**
  - — Organized around major subjects, such as customer, product, sales
  - — 总是围绕一些主题，如顾客、供应商、产品、销售组织等
  - — Focusing on the modeling and analysis of data for decision makers, not on daily operations or transaction processing
  - — 关注决策者的数据建模与分析，而不是集中于组织机构的日常操作和事务处理
  - — Provide a simple and concise view around particular subject issues by excluding data that are not useful in the decision support process
  - — 排除对于决策无用的数据，提供特定主题的简明视图



Data Warehouse—Integrated

- Constructed by integrating multiple, heterogeneous data sources
  - relational databases, flat files, on-line transaction records
- 一个数据仓库是通过 集成多个异种数据源 来构造的
  - E.g., 关系数据库，一般文件，*联机事务处理记录*
- Data cleaning and data integration techniques are applied.
  - Ensure consistency in naming conventions, encoding structures, attribute measures, etc. among different data sources
    - E.g., Hotel price: currency, tax, breakfast covered, etc.
  - When data is moved to the warehouse, it is converted.
- 使用 数据清理 和 数据集成 技术
  - 当数据被移到数据仓库时，它们要经过转化
  - 确保命名约定、编码结构、属性度量等的一致性
    - E.g., Hotel price: currency, tax, breakfast covered, etc.

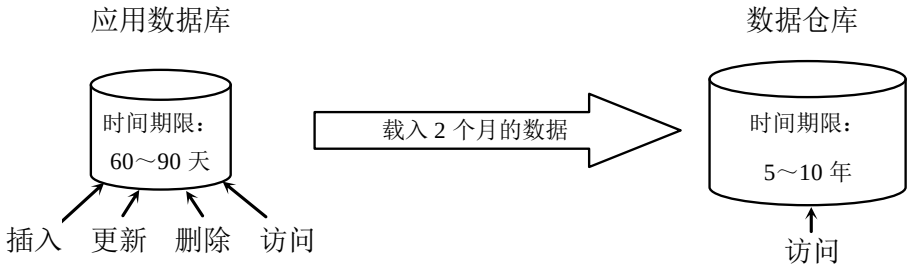


集成

数据仓库中存储的数据一般从企业原来已建立的数据库系统中提取出来，但并不是原有数据的简单拷贝，而是经过了抽取、筛选、清理、转换、综合等工作

Data Warehouse—Time Variant

- The time horizon for the data warehouse is significantly longer than that of operational systems
  - Operational database: current value data
  - Data warehouse data: provide information from a historical perspective (e.g., past 5-10 years)
- 数据仓库是从历史的角度提供信息，时间范围比操作数据库系统要长的多
  - 数据仓库:从历史的角度提供信息（比如过去 5-10 年）
  - 操作数据库系统: 主要保存当前数据
- Every key structure in the data warehouse
  - Contains an element of time, explicitly or implicitly
  - But the key of operational data may or may not contain “time element”
- 数据仓库的关键结构
  - 数据仓库中每一个关键结构都隐式或显式地包含时间元素
  - 操作数据库中的关键结构可能不包括时间元素



随时间而变化即时变的

数据仓库大多关注的是历史数据，其中数据是批量载入的，即定期从操作型应用系统中接收新的数据内容，这使得数据仓库中的数据总是拥有时间维度

Data Warehouse—Nonvolatile

- A physically separate store of data transformed from the operational environment
- 尽管数据仓库中的数据来自于操作数据库，但数据仓库总是在物理上分离保存的
- Operational update of data does not occur in the data warehouse environment
  - Does not require transaction processing, recovery, and concurrency control mechanisms
  - Requires only two operations in data accessing:
    - *initial loading of data* and *access of data*
- 操作数据库的 更新操作 不会出现在数据仓库环境下
  - 不需要事务处理、恢复和并发控制等机制
  - 只需要两种数据访问:
    - Initial loading of data 数据的初始装载
    - Access of data 数据访问



稳定性即非易失的

操作型数据库系统中一般只存储短期数据，因此其数据是不稳定的，它记录的是系统中数据变化的瞬态。操作型数据库要经常进行修改、添加

数据仓库中数据大多表示过去某一时刻的数据，主要用于查询、分析，不需要事务处理、恢复和并发控制等机制，只需进行数据装载和访问

“为什么不直接在传统数据库上进行联机分析处理，
而是另外花费时间和资源去构造分离的数据仓库？”

OLTP vs. OLAP

“数据仓库与操作数据库系统有什么区别？”

Why a Separate Data Warehouse?

- High performance for both systems
  - DBMS— tuned for OLTP: access methods, indexing, concurrency control, recovery
  - Warehouse—tuned for OLAP: complex OLAP queries, multidimensional view, consolidation
- Different functions and different data:
  - missing data: Decision support requires historical data which operational DBs do not typically maintain
  - data consolidation: DS requires consolidation (aggregation, summarization) of data from heterogeneous sources
  - data quality: different sources typically use inconsistent data representations, codes and formats which have to be reconciled
- Note: There are more and more systems which perform OLAP analysis directly on relational databases

Why a Separate Data Warehouse?
为什么需要一个分离的数据仓库?

- *“为什么不直接在传统数据库上进行联机分析处理，而是另外花费时间和资源去构造分离的数据仓库？”*
- 提高两个系统的性能
  - DBMS是为OLTP而设计的：存储方式,索引, 并发控制, 恢复
  - 数据仓库是为OLAP而设计：复杂的 OLAP查询, 多维视图，汇总
- 不同的功能和不同的数据:
  - 历史数据: 决策支持需要历史数据，而这些数据在操作数据库中一般不会去维护
  - 数据汇总：决策支持需要将来自异种源的数据统一（如聚集和汇总）
  - 数据质量: 不同的源使用不一致的数据表示、编码和格式，对这些数据进行有效的分析需要将他们转化后进行集成

OLTP vs. OLAP

	OLTP	OLAP
users	clerk, IT professional	knowledge worker
function	day to day operations	decision support
DB design	application-oriented	subject-oriented
data	current, up-to-date detailed, flat relational isolated	historical, summarized, multidimensional integrated, consolidated
usage	repetitive	ad-hoc
access	read/write index/hash on prim. key	lots of scans
unit of work	short, simple transaction	complex query
# records accessed	tens	millions
#users	thousands	hundreds
DB size	100MB-GB	100GB-TB
metric	transaction throughput	query throughput, response

OLTP vs. OLAP
Online Transaction Processing vs. Online Analytic Processing

	OLTP	OLAP
用户	办事员、数据库专业人员	知识工人
功能	日常操作	决策支持
DB 设计	面向用户	面向主题
数据	当前的、最新的、详细的	历史的、汇总的、多维的
操作	读/写 主关键字上的索引/散列	大量扫描
工作单位	短的，简单事务	复杂查询
# 访问记录数量	数十个	数百万
#用户数	数千	数百
DB 规模	100MB-GB	100GB-TB
度量	事务吞吐量	查询吞吐量，响应时间

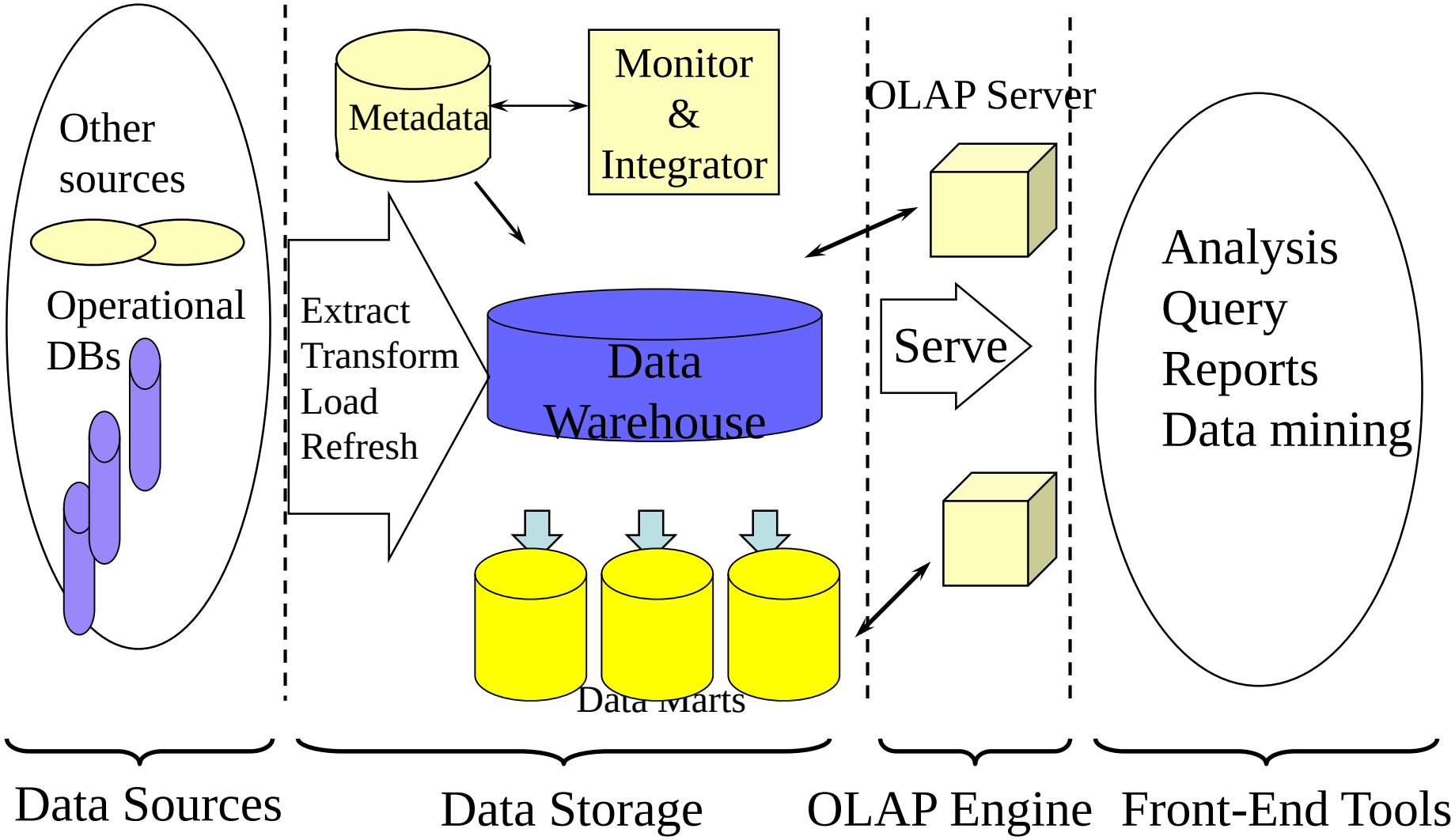
操作数据库系统vs数据仓库

数据仓库	操作型数据库
面向主题	面向应用
容量巨大	容量相对较小
数据是综合的或提炼的	数据是详细的
保存历史的数据	保存当前的数据
通常数据是不可更新的	数据是可更新的
操作需求是临时决定的	操作需求是事先可知的
一个操作存取一个数据集合	一个操作存取一个记录
数据常冗余	数据非冗余
操作相对不频繁	操作较频繁
所查询的是经过加工的数据	所查询的是原始数据
支持决策分析	支持事务处理
决策分析需要历史数据	事务处理需要当前数据
需做复杂的计算	鲜有复杂的计算
服务对象为企业高层决策人员	服务对象为企业业务处理方面的工作人员

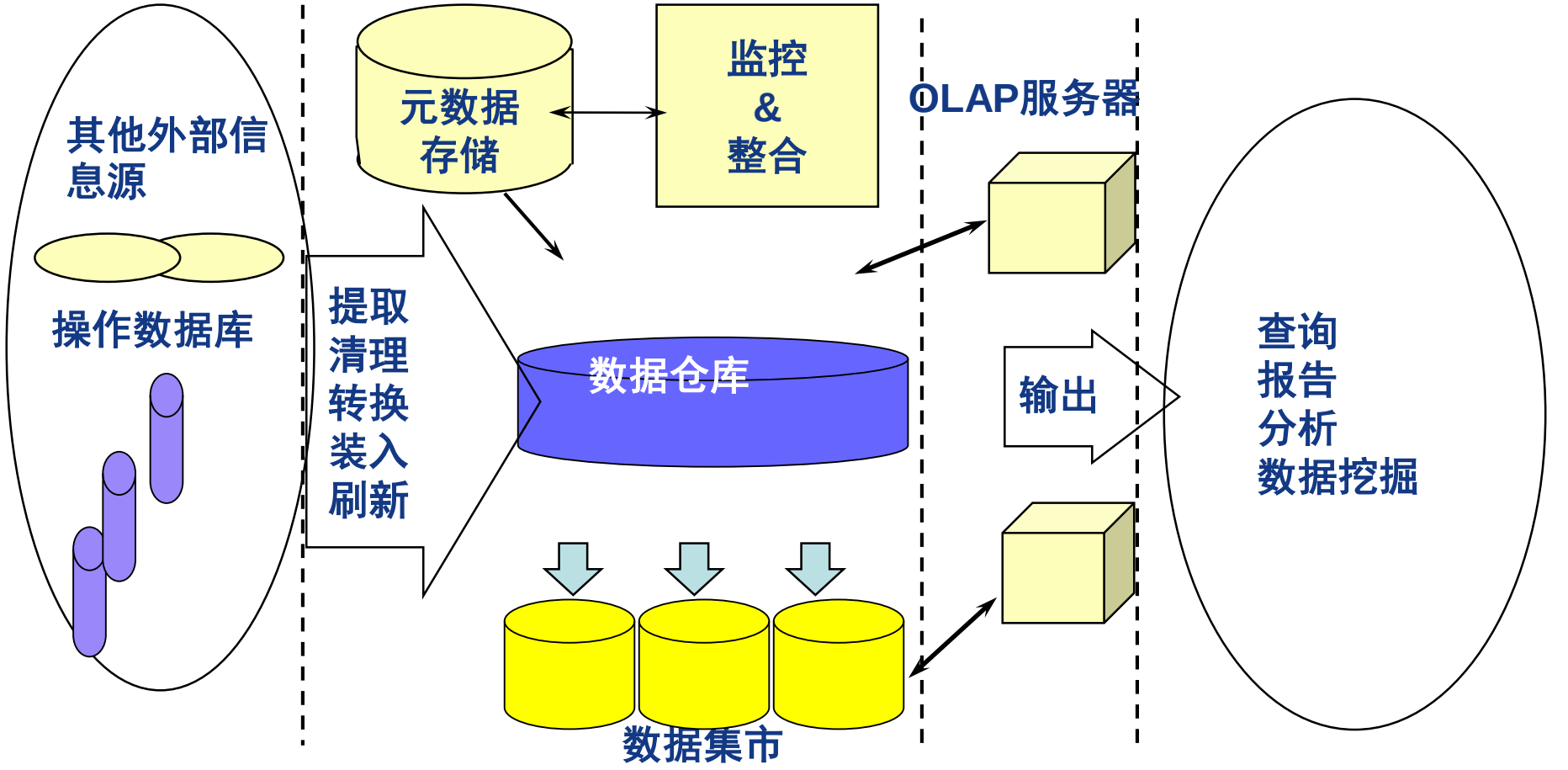
数据仓库是一种多层体系结构

Data Warehouse: A Multi-Tiered Architecture

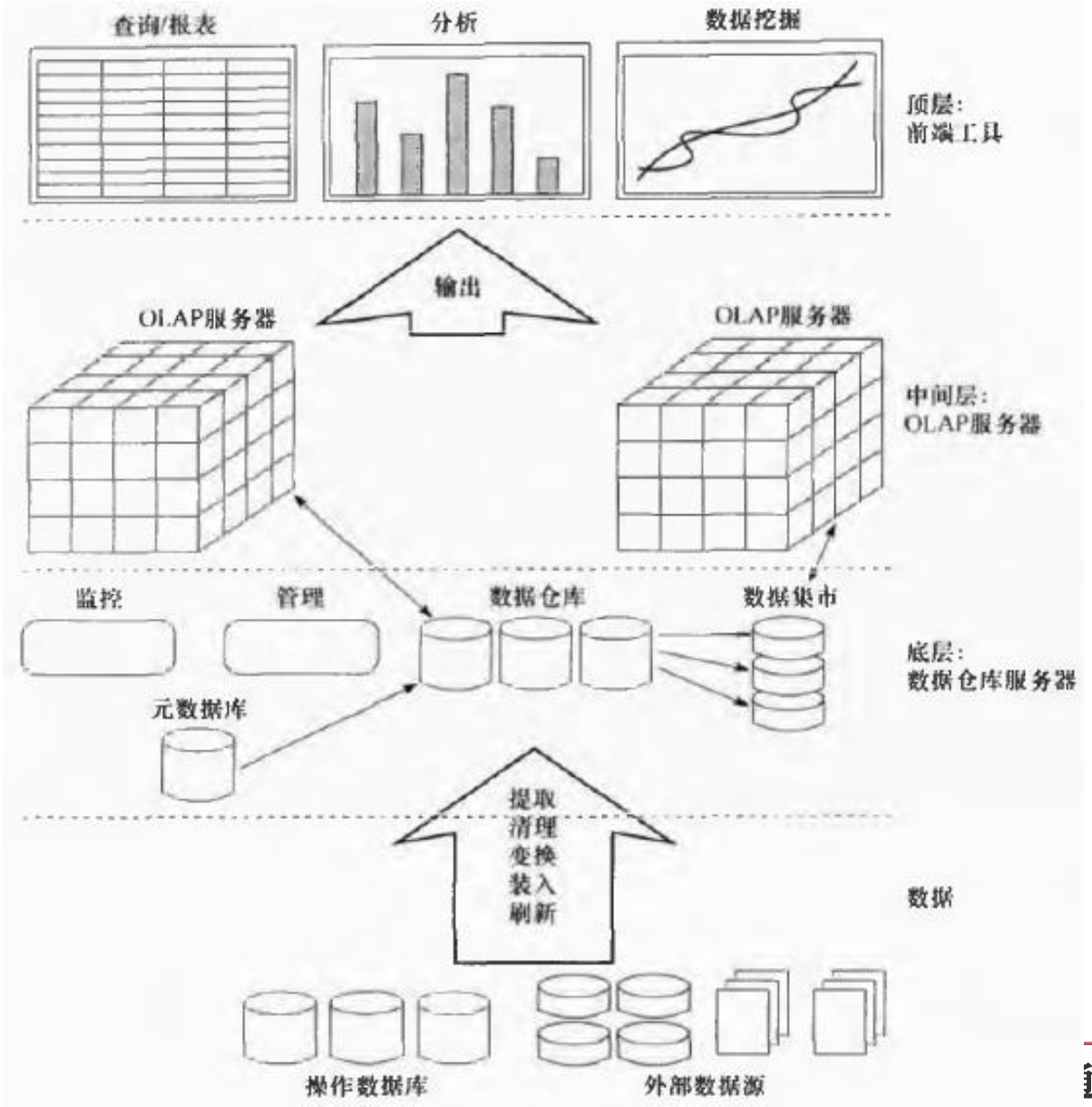
Data Warehouse: A Multi-Tiered Architecture



Data Warehouse: A Multi-Tiered Architecture
三层数据仓库架构



Data Warehouse: A Multi-Tiered Architecture
三层数据仓库架构



Data Warehouse: A Multi-Tiered Architecture
三层数据仓库架构

- 底层：数据仓库的数据库服务器
  - —关注的问题：如何通过这一层从操作数据库或其它外部数据源提取数据来构建数据仓库
    - 通过称为信关Gateway的应用程序提取数据
      - 开放数据库连接ODBC
      - Java数据库连接 JDBC
      - 数据库开放连接和嵌入 OLE-DB
- 中间层：OLAP服务器
  - —关注的问题：OLAP服务器如何实施（关系型OLAP，多维OLAP等）
- 前端客户工具层
  - —关注的问题：查询工具、报表工具、分析工具、挖掘工具等

Three Data Warehouse Models

- **Enterprise warehouse**
  - —collects all of the information about subjects spanning the entire organization
- **Data Mart**
  - —a subset of corporate-wide data that is of value to a specific groups of users. Its scope is confined to specific, selected groups, such as marketing data mart
    - Independent vs. dependent (directly from warehouse) data mart
- **Virtual warehouse**
  - —A set of views over operational databases
  - —Only some of the possible summary views may be materialized

Three Data Warehouse Models
三种数据仓库模型

- 从体系结构的角度去看，数据仓库模型可以有以下三种：

—企业仓库（ Enterprise warehouse）

- 搜集关于跨越整个组织的主题的所有信息，提供企业范围内的数据集成
- 通常来自一个或多个操作数据库系统或外部信息提供者，并且是多功能的。通常，它包含细节数据和汇总数据，其规模由数兆兆字节，到数百兆兆字节，数千兆兆字节，甚至更多。
- 企业数据仓库可以在传统的大型机、超级计算机服务器或并行结构平台上实现。它需要广泛的商务建模，可能需要多年设计和建设。

—数据集市（Data mart）

- 企业范围数据的一个子集，对于特定的客户是有用的。其范围限于选定的主题，如：销售数据集市可能限定其主题为顾客、商品和销售。
- 包括在数据集市中的数据通常是汇总的。
- 独立的数据集市 VS. 非独立的数据集市（数据来自于企业数据仓库）

—虚拟仓库（Virtual warehouse）

- 操作数据库上的一系列视图
- 只有一些可能的汇总视图被物化

数据仓库与异种数据库集成

- 异种数据库的集成方法

—传统的异种数据库集成:（查询驱动）

- 在多个异种数据库上建立包装程序（wrappers）和中介程序（mediators ）
- 查询驱动方法——当从客户端传过来一个查询时，首先使用元数据字典将查询转换成相应异种数据库上的查询；然后，将这些查询映射和发送到局部查询处理器

—数据仓库:（更新驱动）

- 将来自多个异种源的信息预先集成，并存储在数据仓库中，供直接查询和分析
- 数据仓库被看做一种体系结构，通过将异构数据源中的数据集成在一起而构建，支持结构化和/或专门的查询、分析报告和决策制定

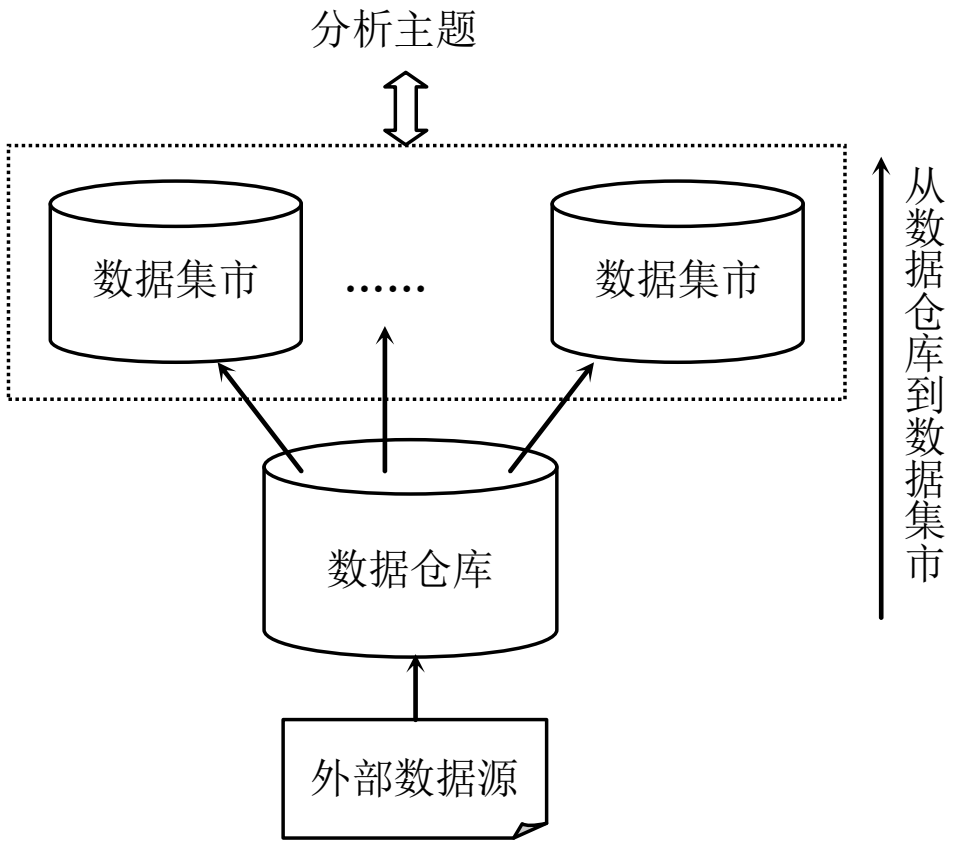
Extraction, Transformation, and Loading (ETL)

- Data extraction
  - —get data from multiple, heterogeneous, and external sources
- Data cleaning
  - —detect errors in the data and rectify them when possible
- Data transformation
  - —convert data from legacy or host format to warehouse format
- Load
  - —sort, summarize, consolidate, compute views, check integrity, and build indicies and partitions
- Refresh
  - —propagate the updates from the data sources to the warehouse

数据仓库后端工具和程序

- 数据仓库用来装入和刷新数据的工具，包括：
  - —数据提取
    - 从多个外部的异构数据源收集数据
  - —数据清理
    - 检测数据中的错误并作可能的订正
  - —数据变换
    - 将数据由历史或主机的格式转化为数据仓库的格式
  - —装载
    - 排序、汇总、合并、计算视图，检查完整性，并建立索引和分区
  - —刷新
    - 将数据源的更新传播到数据仓库中

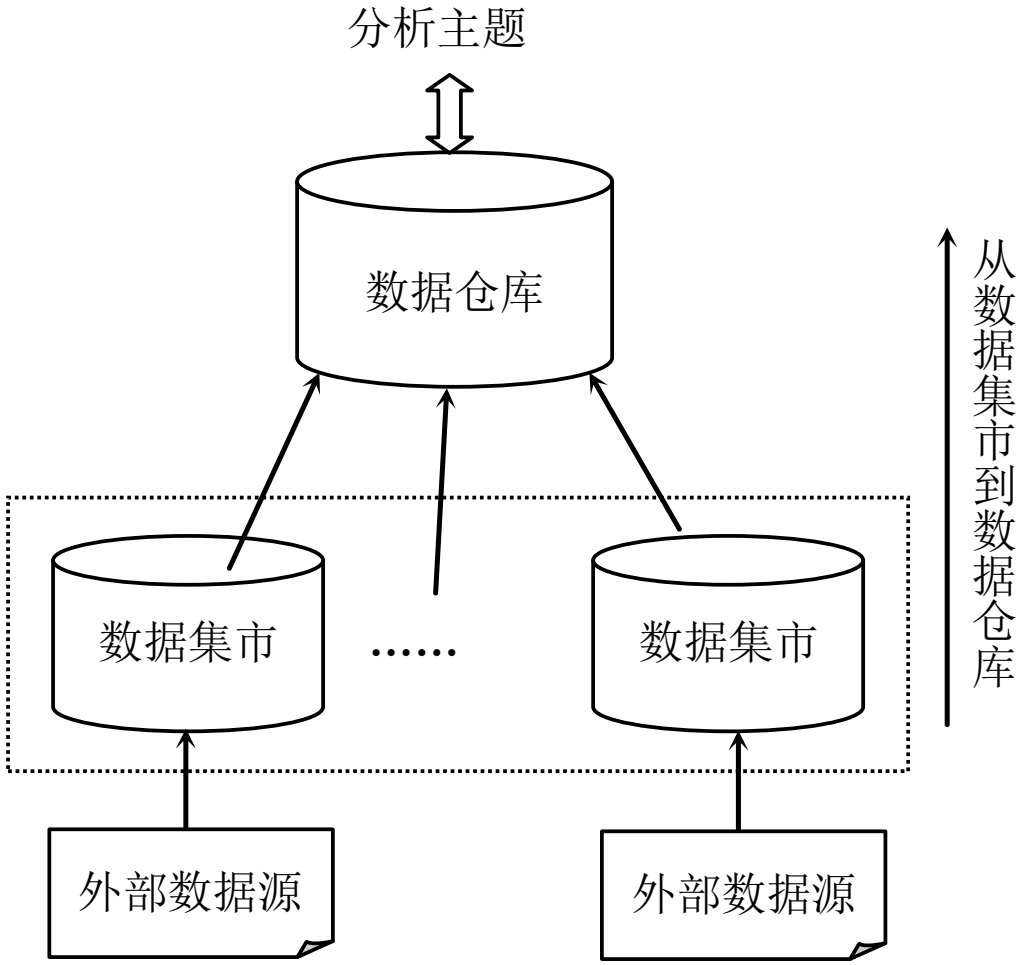
数据仓库先整体再局部的构建模式：W.H.Inmon模式



优点：
数据规范化程度高，最小化数据冗余与不一致性；便于全局数据的分析和挖掘

缺点：
建设周期长、见效慢；风险程度相对大

数据仓库先局部再整体的构建模式：Ralph Kimball 模式



优点：
投资少、见效快；在设计上相对灵活；易于实现
缺点：
会有一定级别的冗余和不一致性

数据仓库开发：自顶向下 V.S. 自底向上

数据仓库开发上的困难

—自顶向下方法

- 从全系统的角度提供解决方案，使得（模块）集成的问题最小；但是该方法十分昂贵，需要对组织进行长期研究和建模分析。

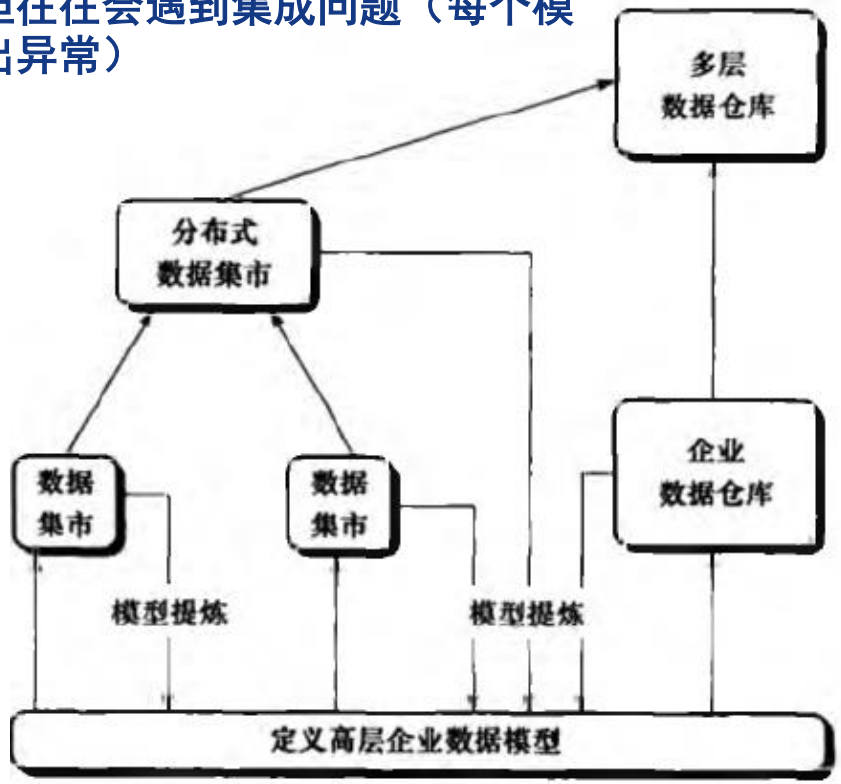
—自底向上方法

- 提供了更多的开发灵活性，价格便宜；但往往会遇到集成问题（每个模块单独运行都没有问题，但是一集成就出异常）

解决方法：

—使用递增性、演化性的开发方法

- 高层数据模型
- 企业仓库和数据集市并行开发
- 通过分布式模型集成各数据集市
- 多层数据仓库



元数据库

Metadata Repository

Metadata Repository

- Meta data is the data defining warehouse objects. It stores:
- Description of the structure of the data warehouse
  - schema, view, dimensions, hierarchies, derived data defn, data mart locations and contents
- Operational meta-data
  - data lineage (history of migrated data and transformation path), currency of data (active, archived, or purged), monitoring information (warehouse usage statistics, error reports, audit trails)
- The algorithms used for summarization
- The mapping from operational environment to the data warehouse
- Data related to system performance
  - warehouse schema, view and derived data definitions
- Business data
  - business terms and definitions, ownership of data, charging policies

Metadata Repository
元数据存储

- 元数据是关于数据的数据
- 在数据仓库中，元数据是定义数据仓库对象的数据，包括：
  - 数据仓库结构的描述
    - 仓库模式、视图、维、层次结构、导出数据的定义，以及数据集市的位置和内容
  - 操作元数据
    - 数据血统（迁移数据的历史和它所使用的变换序列）、数据流通（主动的、档案的或净化的）、管理信息（仓库使用的统计量、错误报告和审计跟踪）
  - 汇总用的算法
    - 包括度量和维定义算法，数据所处的粒度、划分、主题领域、聚集、汇总、预定义的查询和报告
  - 由操作环境到数据仓库的映射
    - 源数据库和它们的内容，信关描述，数据划分，数据提取、清理、转换规则和默认值，数据刷新和净化规则，以及安全性（用户授权和存取控制）
  - 关于系统性能的数据
    - 除刷新、更新和复制周期的定时和调度的规则外，还包括改善数据存取和检索性能的索引和概要
  - 商务元数据
    - 商务术语和定义、数据拥有者信息、收费政策等

Metadata Repository
元数据的使用

- 元数据与数据一起，构成了数据仓库中的数据模型，元数据所描述的更多的是这个模型的结构方面的信息。

- 在数据仓库中，元数据的主要用途包括：
  - —用作目录，帮助决策支持系统分析者对数据仓库的内容定义
  - —作为数据仓库和操作性数据库之间进行数据转换时的映射标准
  - —用于指导当前细节数据和稍加综合的数据之间的汇总算法，指导稍加综合的数据和高度综合的数据之间的汇总算法

Chapter 5: Data Warehousing and On-line Analytical Processing

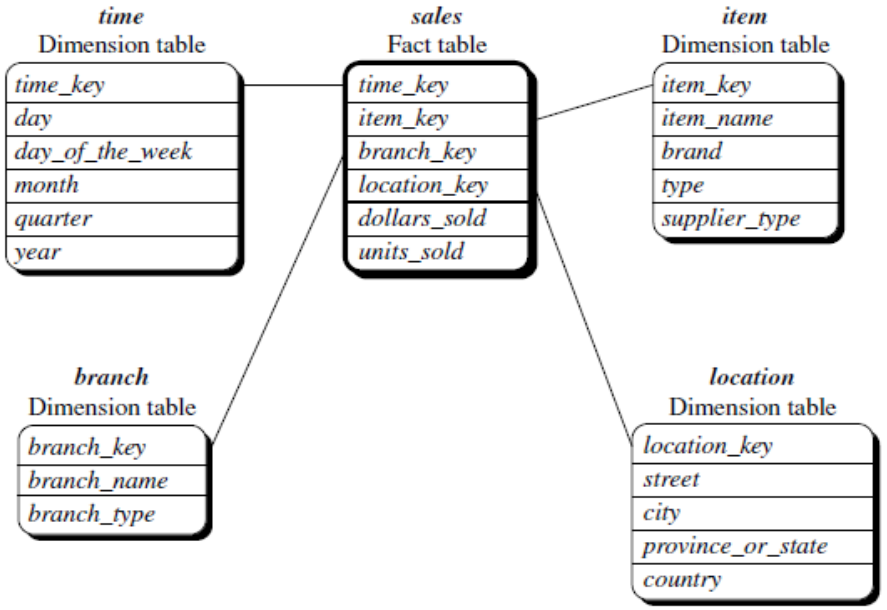
- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- **Data Cube and OLAP**
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

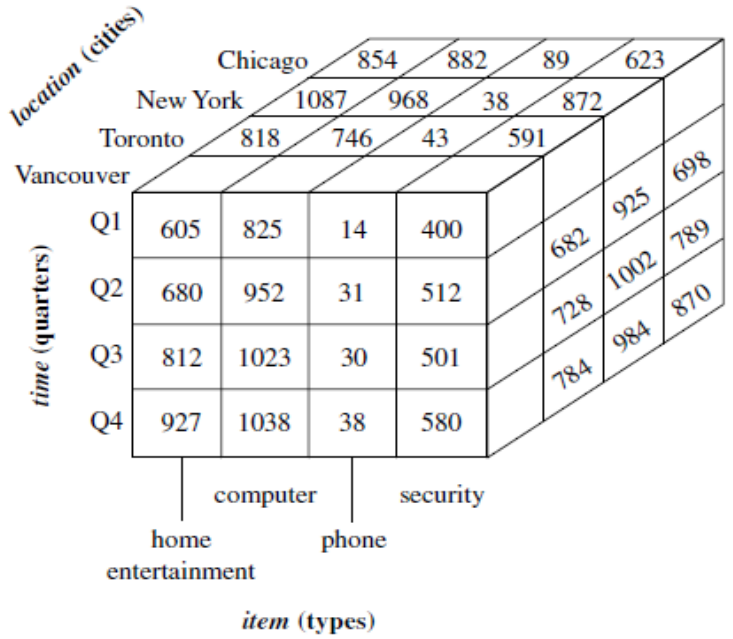
From Tables and Spreadsheets to Data Cubes

- A data warehouse is based on a multidimensional data model which views data in the form of a data cube
- A data cube, such as sales, allows data to be modeled and viewed in multiple dimensions
  - Dimension tables, such as item (item_name, brand, type), or time(day, week, month, quarter, year)
  - Fact table contains measures (such as dollars_sold) and keys to each of the related dimension tables
- In data warehousing literature, an n-D base cube is called a base cuboid. The top most 0-D cuboid, which holds the highest-level of summarization, is called the apex cuboid. The lattice of cuboids forms a data cube.

由表和电子数据表到数据立方体

- 一个数据仓库建立在多维数据模型上，多维数据模型把数据看成数据立方体的形式
- 一个数据立方体，像sales，允许以多维对数据建模和观察
  - —维表，例如item的维表包含属性(item_name, brand, type)，time的维表包含属性(day, week, month, quarter, year)
  - —事实表包含度量(例如dollars_sold) 和每个相关维表的关键字

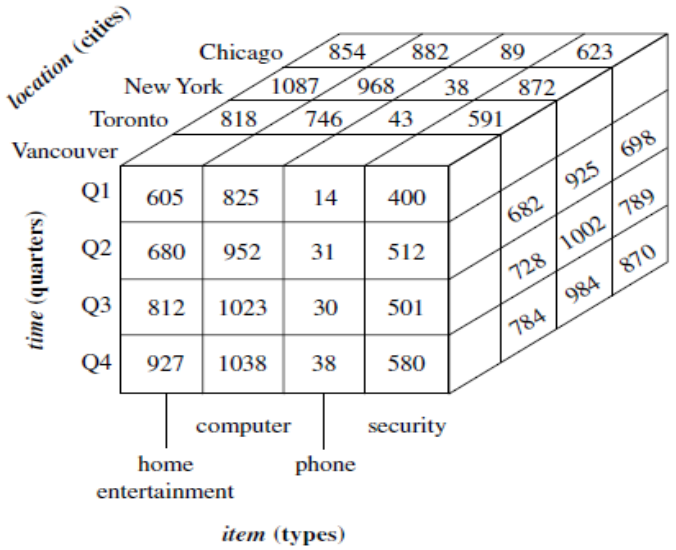




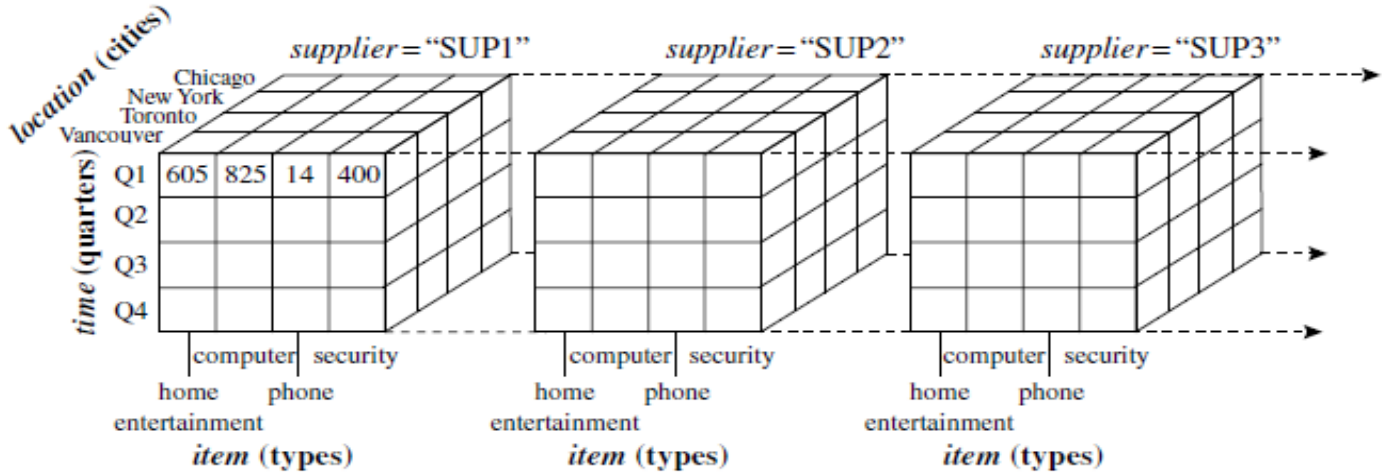
多维数据模型

数据仓库中，数据立方体是n维的

- **数据的3-D数据立方体表示**



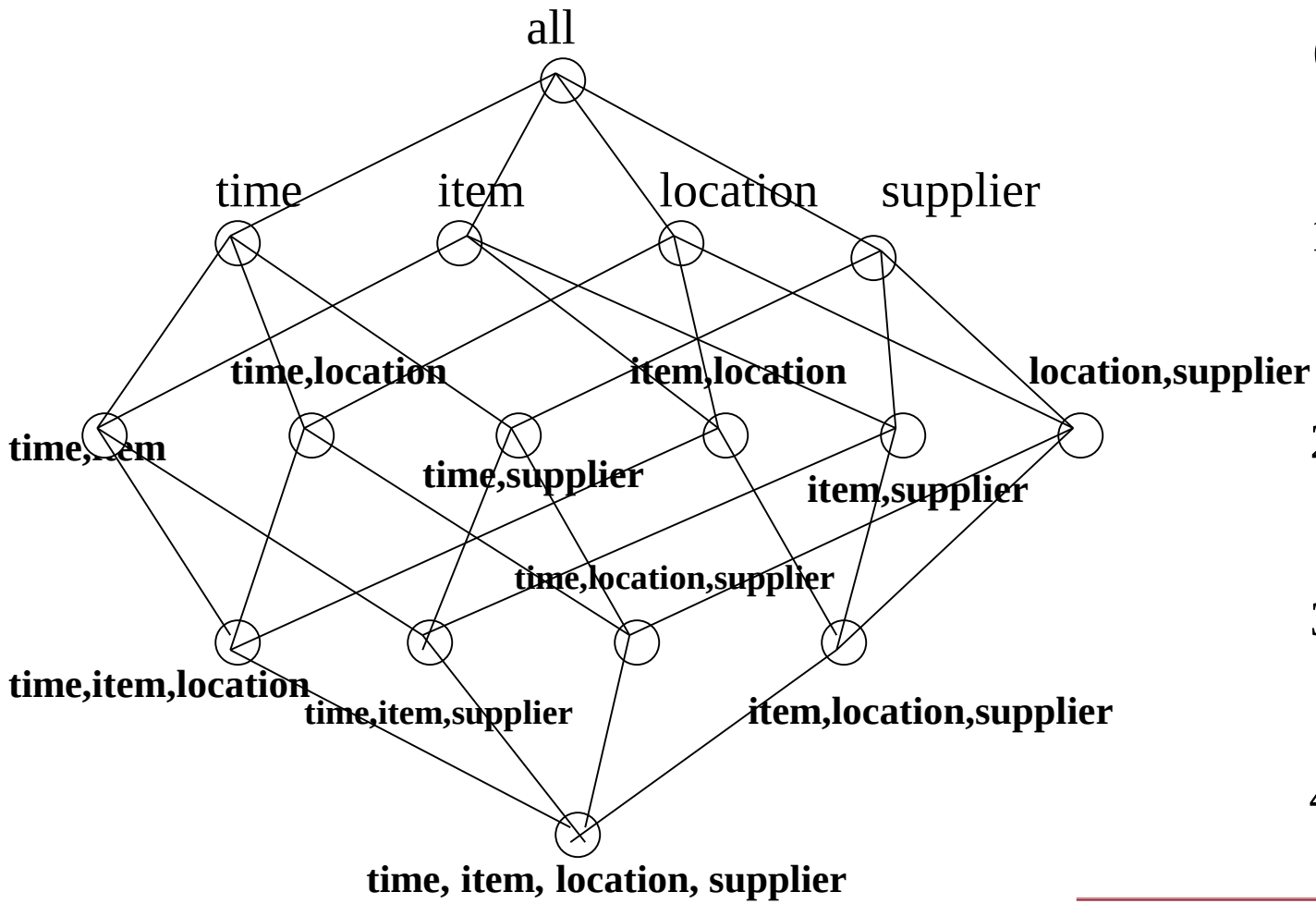
- **4-D情况下，可以把4-D立方体看成3-D立方体的序列**
  - 把任意n维数据立方体显示成(n-1)维“立方体”的序列



- **数据立方体只是对多维数据存储的一种比喻**
  - 实际物理存储可以不同于逻辑表示

Cube: A Lattice of Cuboids
立方体：一个方体格

- 数据仓库语义中，一个n-D 底层方体称为**基本方体**；最高层的0-D方体，存放最高层的汇总，称为**顶点方体**；所有的方体格组成了**数据立方体**



0-D (*apex*) cuboid

1-D cuboids

2-D cuboids

3-D cuboids

4-D (*base*) cuboid

Conceptual Modeling of Data Warehouses

- Modeling data warehouses: dimensions & measures
  - Star schema: A fact table in the middle connected to a set of dimension tables
  - Snowflake schema: A refinement of star schema where some dimensional hierarchy is normalized into a set of smaller dimension tables, forming a shape similar to snowflake
  - Fact constellations: Multiple fact tables share dimension tables, viewed as a collection of stars, therefore called galaxy schema or fact constellation

数据仓库的概念模型

- **最流行的数据仓库概念模型是多维数据模型。这种模型可以以星型模式、雪花模式、或事实星座模式的形式存在。**
  - —星型模式(Star schema): 事实表在中心，周围围绕地连接着维表（每维一个），事实表含有大量数据，没有冗余
  - —雪花模式(Snowflake schema): 是星型模式的变种，其中某些维表是规范化的，因而把数据进一步分解到附加表中。结果，模式图形成类似于雪花的形状
  - —事实星座模式(Fact constellations): 多个事实表共享维表，这种模式可以看作星型模式集，因此称为星系模式(galaxy schema)，或者事实星座(fact constellation)

Star Schema

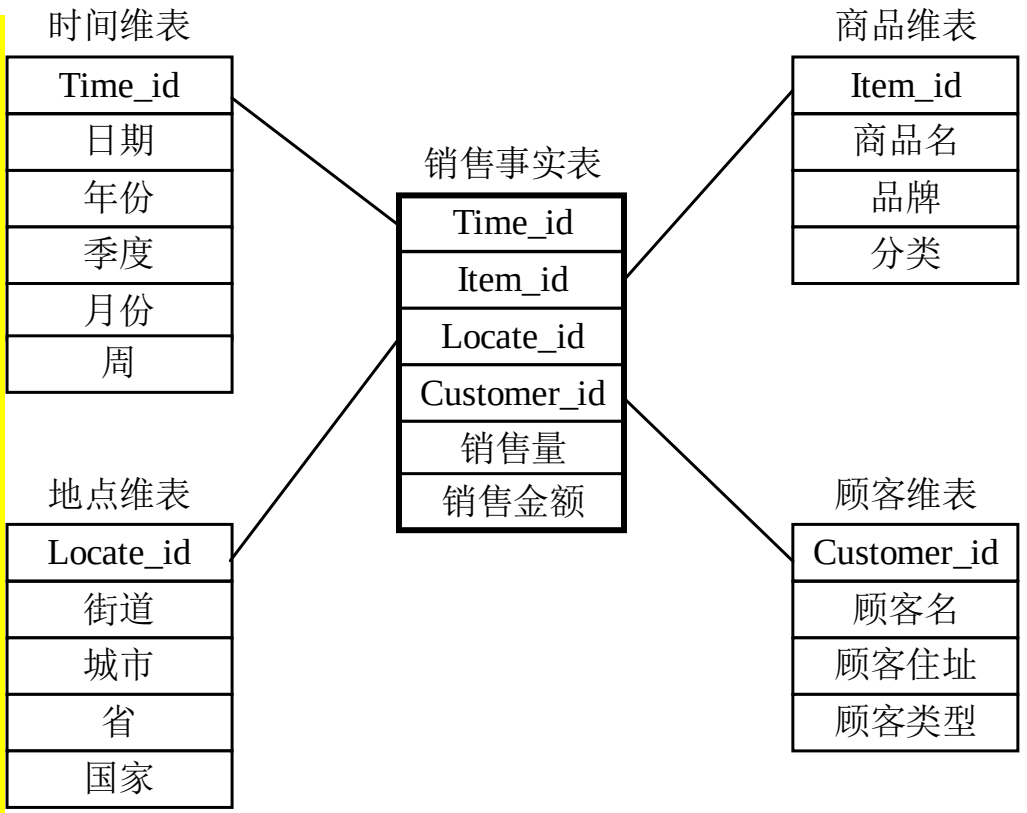
- 星形模式是由一个事实表和一组维表组成，每个维表都有一个维主键，所有维组合成事实表的主键，换言之，事实表主键的每个元素都是维表的外键。该模式的核心是事实表，通过事实表将各种不同的维表连接起来，各个维表都连接到中央事实表

一个“销售”数据仓库的星形模式

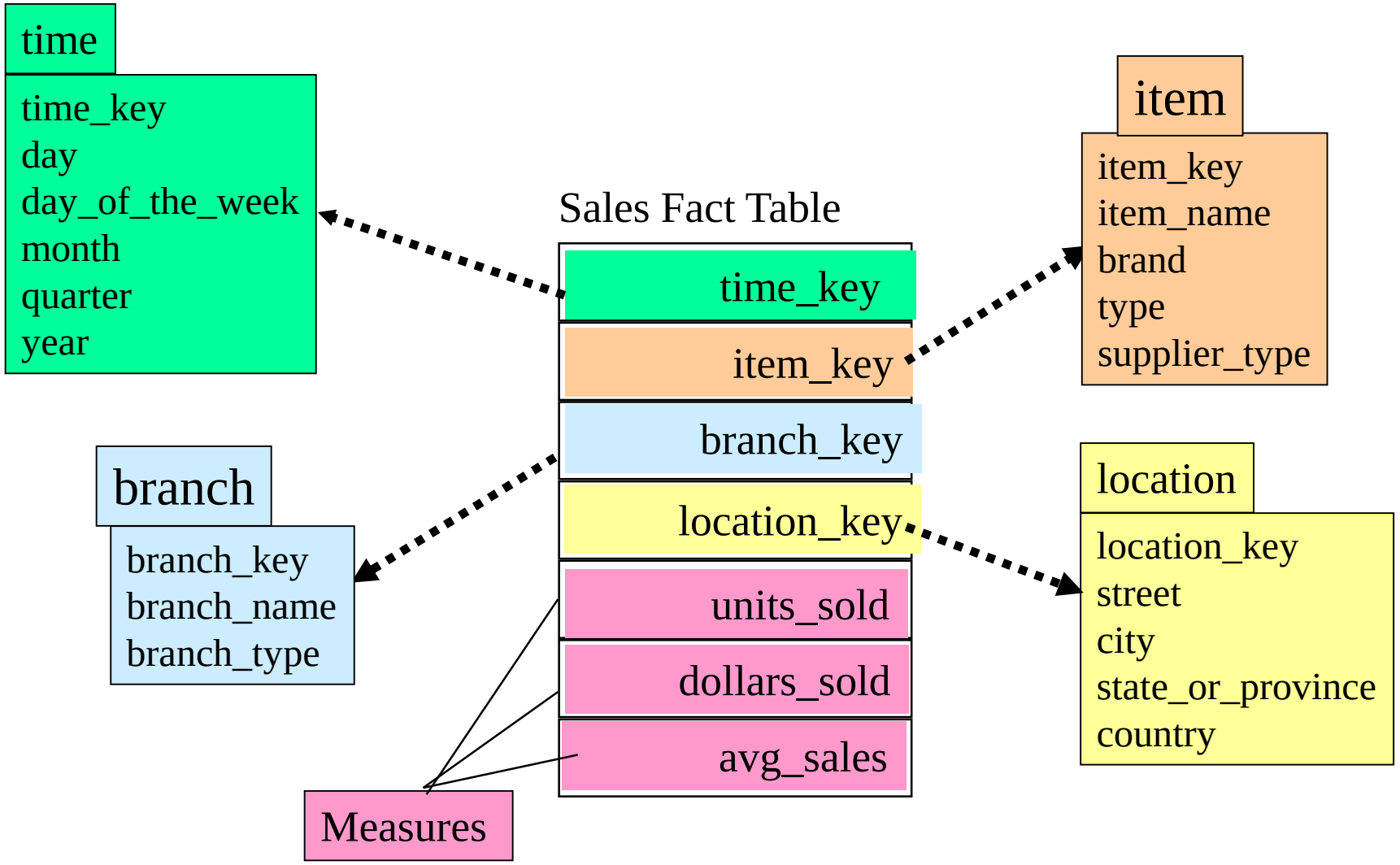
包含一个中心事实表“销售事实表”和4个维表：时间维表、销售商品维表、销售地点维表和顾客维表。在销售事实表中存储着四个维表的主键和两个度量“销售量”和“销售金额”

维表通常不使用业务数据库的关键字作为主键，而是对每个维表另外增加一个额外的字段作为主键来识别维表中的对象。在维表中新设定的键也称为代理键

事实表中除度量外，其他字段都是维表或中间表（对于雪花模式）的关键字（外键）。如果事实相关的维很多，则事实表的字段个数也会比较多。事实表记录数量很多，应当尽量减小一条记录的长度，避免事实表过大而难于管理



Star Schema


time
time_key
day
day_of_the_week
month
quarter
year
Sales Fact Table
time_key
item_key
branch_key
location_key
units_sold
dollars_sold
avg_sales
item
item_key
item_name
brand
type
supplier_type
branch
branch_key
branch_name
branch_type
location
location_key
street
city
state_or_province
country
Measures

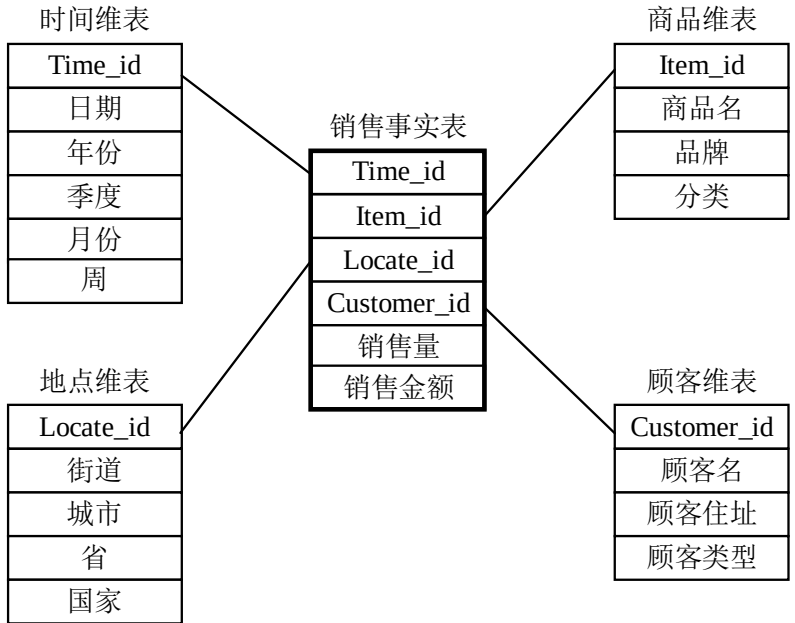
Star Schema

- 注意，在星型模式中，每维只用一个表表示，而每个表包含一组属性，这种限制可能会造成某些冗余
  - 例如，销售地点维表包含属性集{Location_id，街道，城市，省，国家}。这种模式可能造成某些冗余，例如，可能存在城市、省、国家字段的数据冗余

{101，“解放大道100号”，“武汉”，“湖北省”，“中国”}

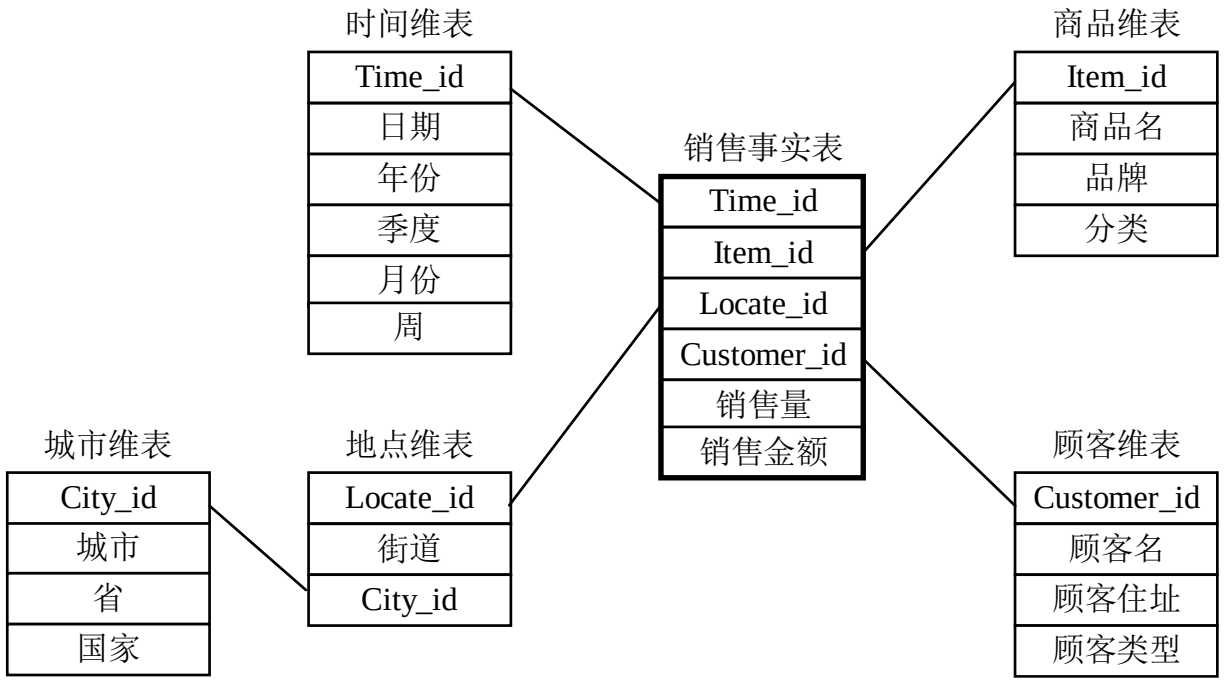
{201，“解放大道85号”，“武汉”，“湖北省”，“中国”}

{255，“解放大道205号”，“武汉”，“湖北省”，“中国”}

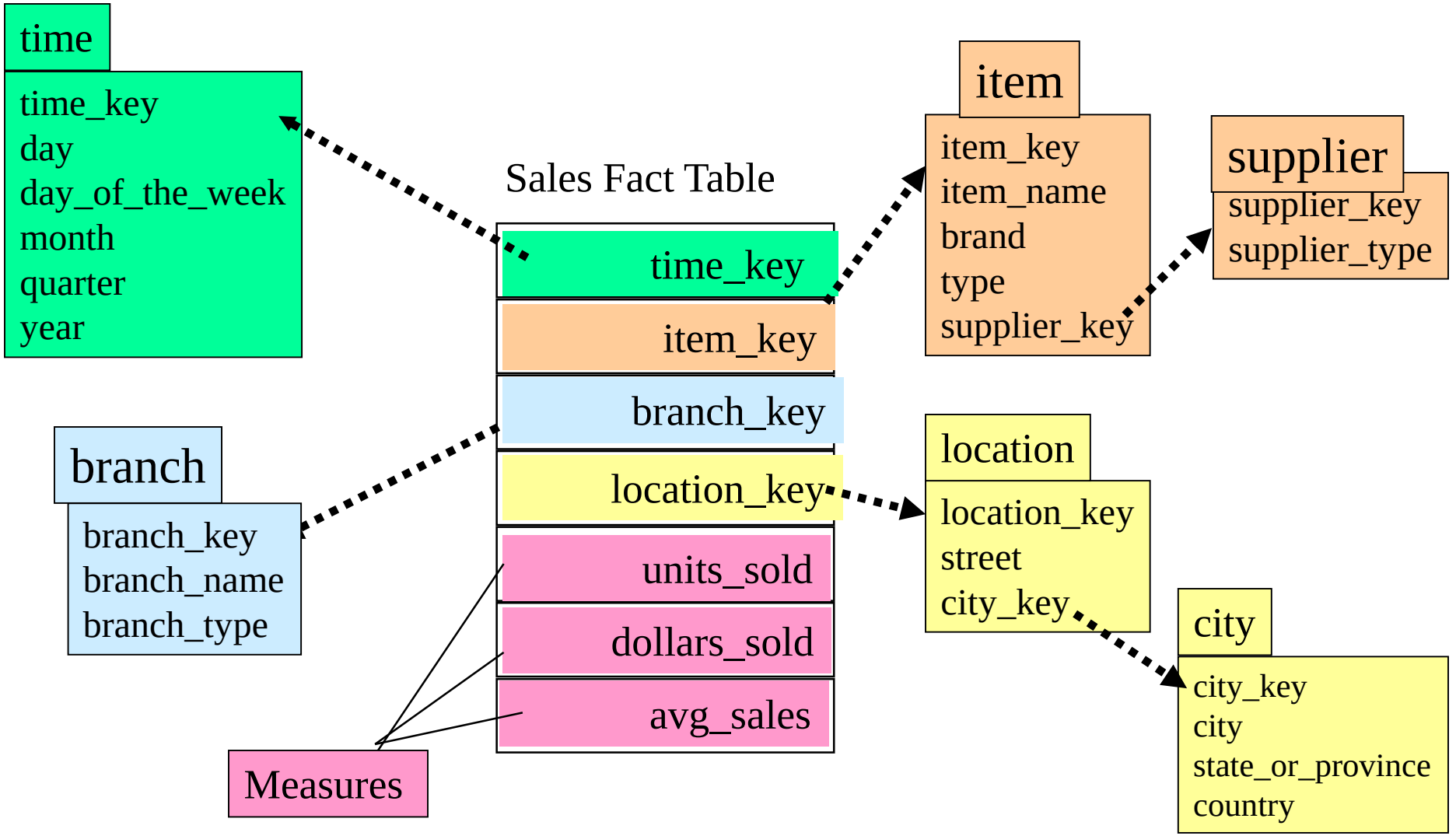


Snowflake Schema

- 雪花模式是对星形模式的扩展，每一个维表都可以向外连接多个详细类别表
- 在这种模式中，维表除了具有星形模式中维表的功能外，还连接对事实表进行详细描述的详细类别表，详细类别表通过对事实表在有关维上的详细描述达到了缩小事实表和提高查询效率的目的



Snowflake Schema



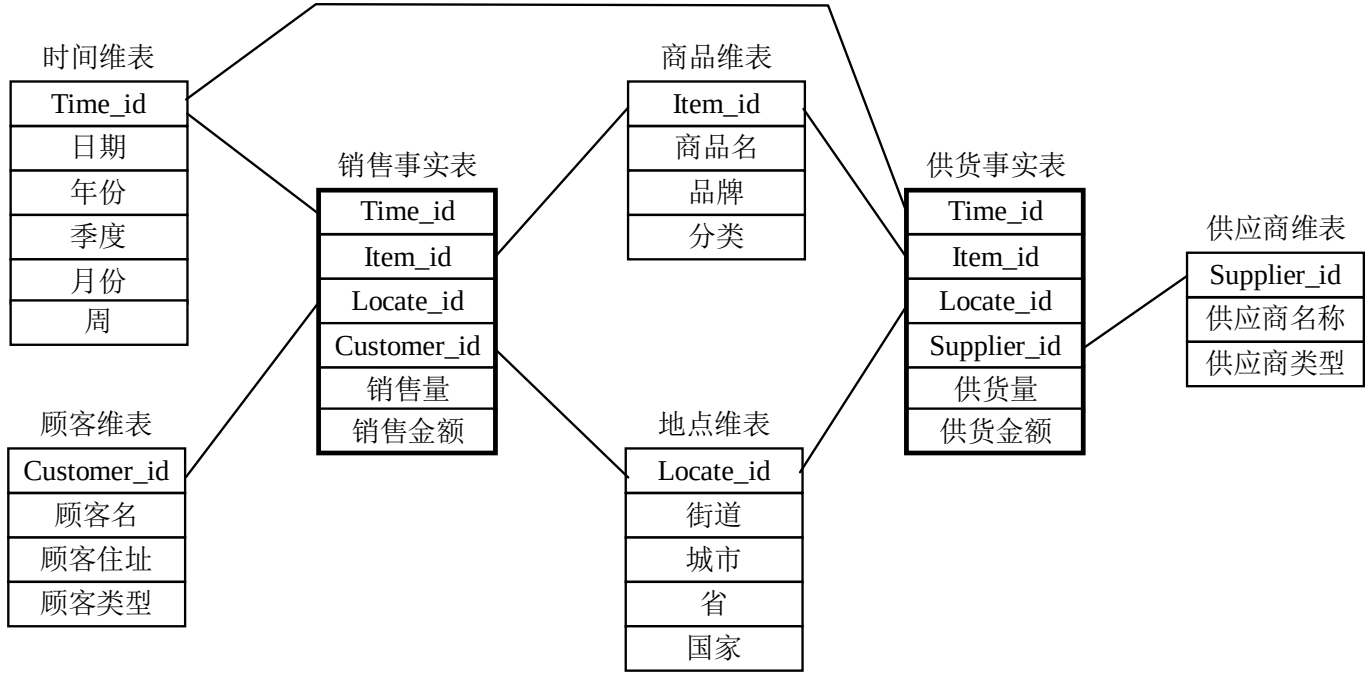
雪花模式

- 规范化后的维表易于维护，并节省存储空间
  - —与典型的巨大事实表相比，这种空间的节省可以忽略
- 执行查询需要更多的连接操作，雪花结构可能降低浏览的效率
  - —系统性能可能相对受到影响

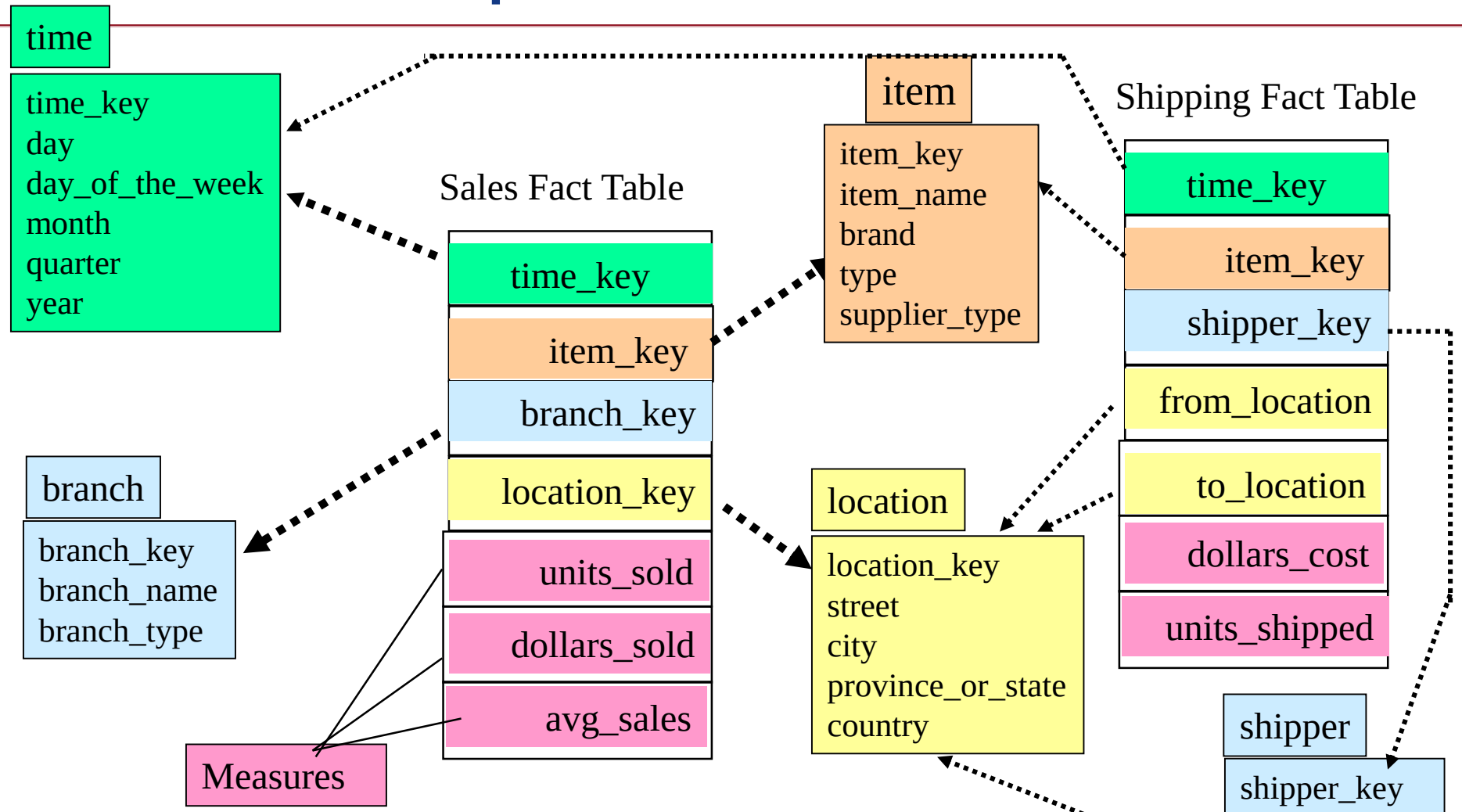
比较项目	星形模式	雪花模式
行数	多	少
可读性	容易	难
表数量	少	多
搜索维的时间	快	慢

Fact Constellation

- 通常一个星形模式或雪花模式对应一个问题的解决（一个主题），它们都有多个维表，但是只能存在一个事实表
- 在一个多主题的复杂数据仓库中可能存放多个事实表，此时就会出现多个事实表共享某一个或多个维表的情况，这就是事实星座模式（Fact Constellations schema）



Example of Fact Constellation

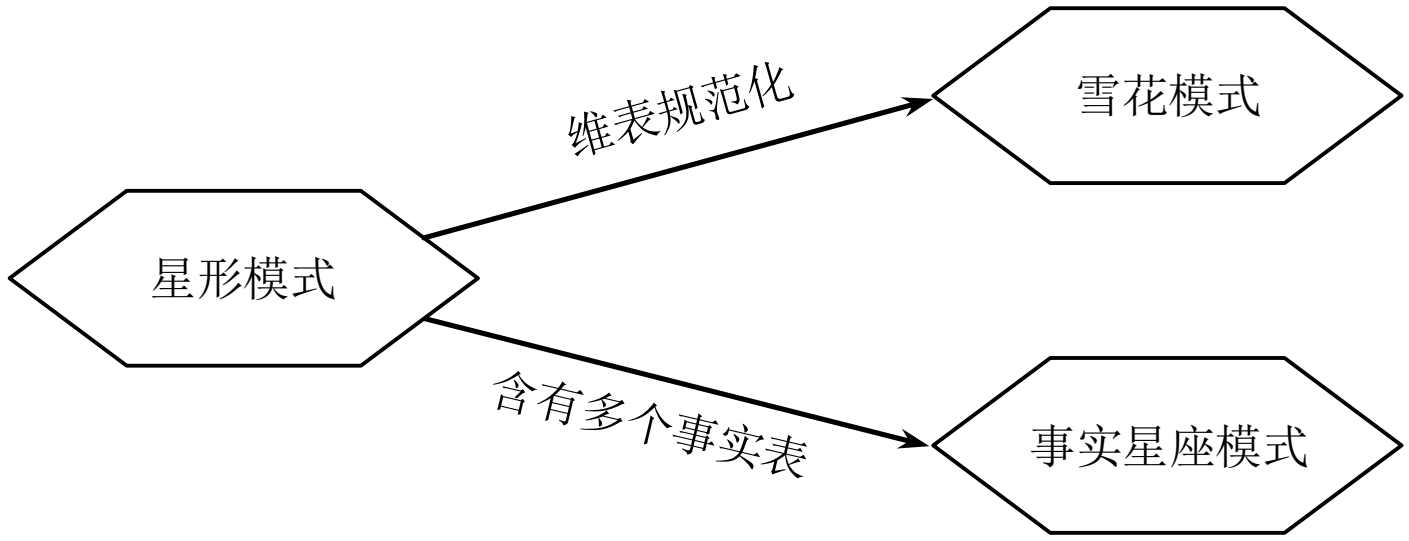


事实星座模式

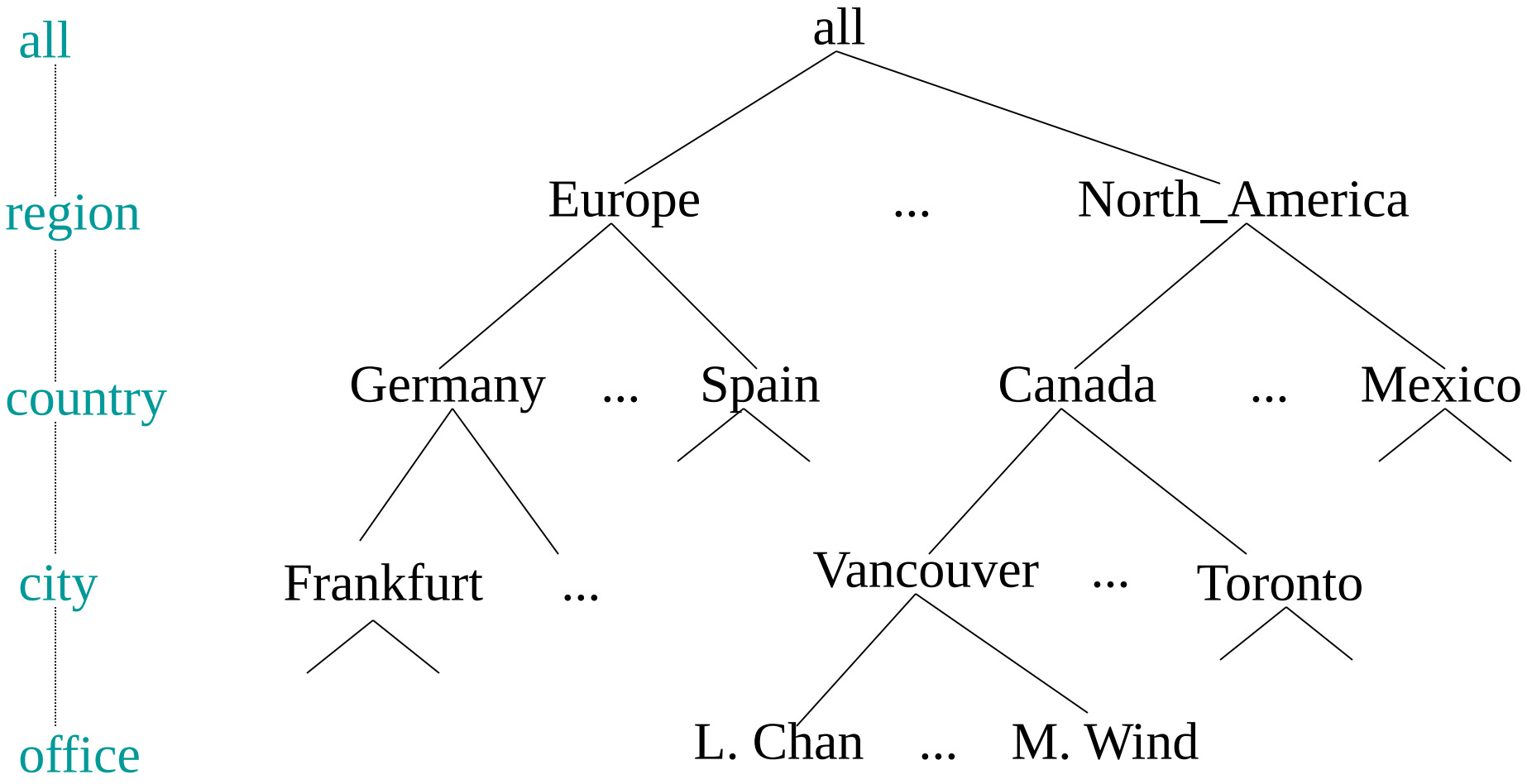
在建立数据仓库时，数据仓库和数据集市之间是有区别的

- 数据仓库收集关于整个组织的主题（如顾客，商品，销售，资产和员工）信息，因此是企业范围的
  - —通常使用事实星座模式，能对多个相关的主题建模
- 数据集市是数据仓库的一个部门子集，针对选定的主题，因此是部门范围的
  - —流行采用星形或雪花模式，适合对单个主题建模

数据仓库概念模型之间的关系



A Concept Hierarchy: Dimension (location)

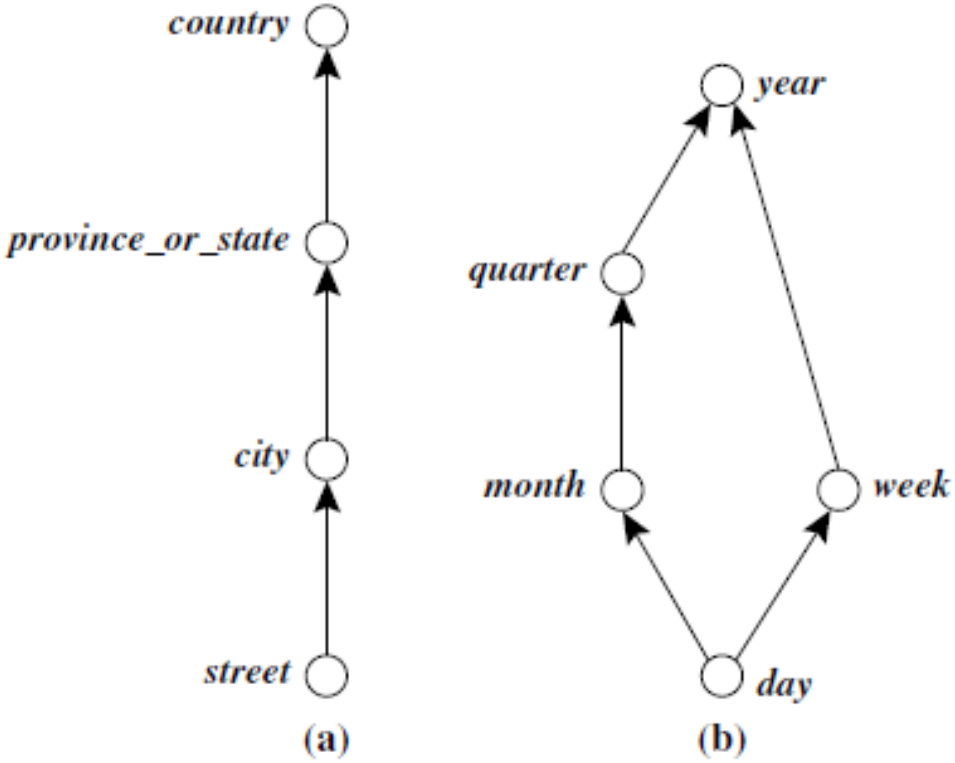


维：概念分层

- 一个概念分层（concept hierarchy）定义一个映射序列，将低层概念映射到更一般的高层概念
  - —E.g. 表示location的概念：杭州→浙江→中国→亚洲
  - —概念分层允许在各种抽象级审查和处理数据
- 许多概念分层的定义隐含在数据库的模式中
  - —比如根据location维的属性，可以得到
    street<city<province_or_state<country
    - 这些属性按一个全序相关，形成一个层次结构
- 概念分层可以由系统用户、领域专家、知识工程师人工的提供，也可以根据数据分布的统计分析自动的产生

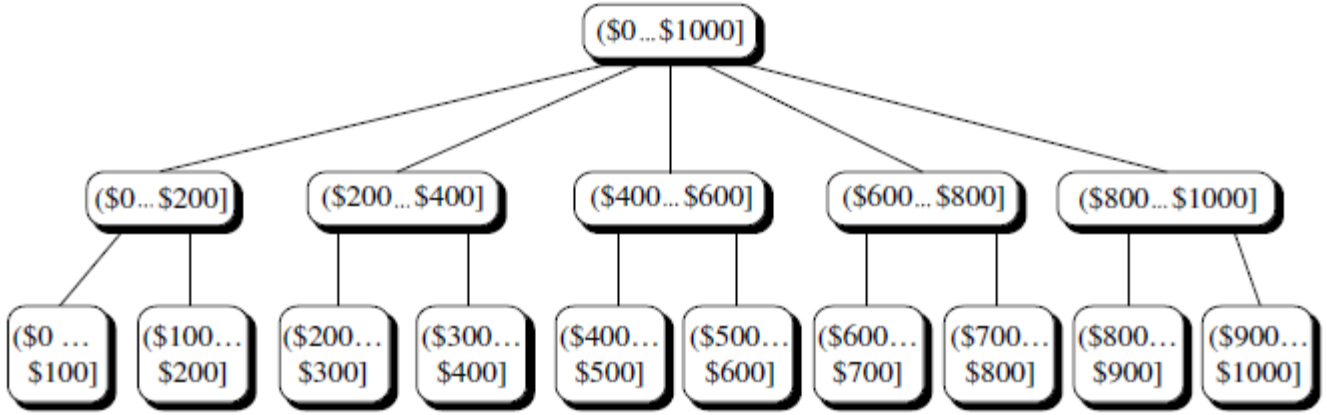
概念分层

- 维的属性也可以组成一个偏序，形成一个格
- 形成数据库模式中属性的全序或偏序的概念分层称作**模式分层**



概念分层

- 可以将给定维或属性的值离散化或分组来定义概念分层，产生**集合分组分层**
- 可以在值的组之间定义全序或偏序
- 以下是关于维price的集合分组概念分层



- 对于一个给定的属性或维，按照不同的用户观点，可能有多个概念分层
  - 用inexpensive，moderately_priced和expensive来组织price

概念分层

- 概念分层为不同级别上的数据汇总提供了一个良好的基础
- 综合概念分层和多维数据模型的潜力，可以对数据获得更深入的洞察力
  - —通过在多维数据模型中，在不同的维上定义概念分层，使得用户在不同的维上从不同的层次对数据进行观察成为可能

- 多维数据模型（数据立方体）使得从不同的角度对数据进行观察成为可能
- 概念分层则提供了从不同层次对数据进行观察的能力
- 结合这两者的特征，可以在多维数据模型上定义各种OLAP操作，为用户从不同角度不同层次观察数据提供了灵活性

度量的分类和计算

- 数据立方体空间的多维点可以用维-值对的集合来定义
  - —<time="Q1",location="温哥华",item= "计算机" >
- 数据立方体度量是一个数值函数
  - —该函数可以对数据立方体空间的每个点求值
  - —通过对给定点的各维-值对聚集数据，计算该点的度量值
- 度量根据其所用的聚集函数可以分成三类：分布的，代数的和整体的

Data Cube Measures: Three Categories

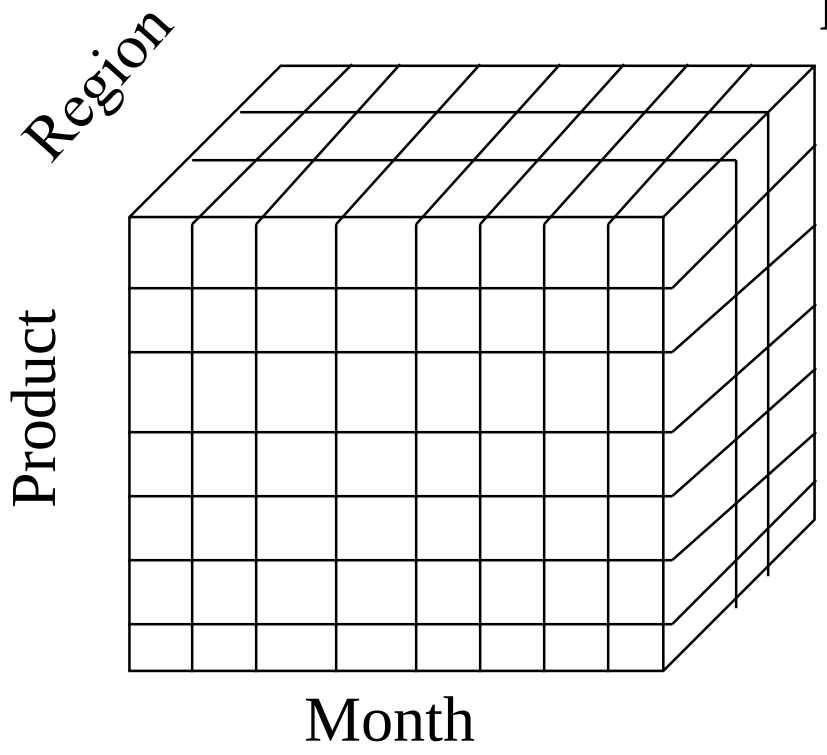
- **Distributive**: if the result derived by applying the function to *n* aggregate values is the same as that derived by applying the function on all the data without partitioning
  - E.g., count(), sum(), min(), max()
- **Algebraic**: if it can be computed by an algebraic function with *M* arguments (where *M* is a bounded integer), each of which is obtained by applying a distributive aggregate function
  - E.g., avg(), min_N(), standard_deviation()
- **Holistic**: if there is no constant bound on the storage size needed to describe a subaggregate.
  - E.g., median(), mode(), rank()

度量的分类和计算

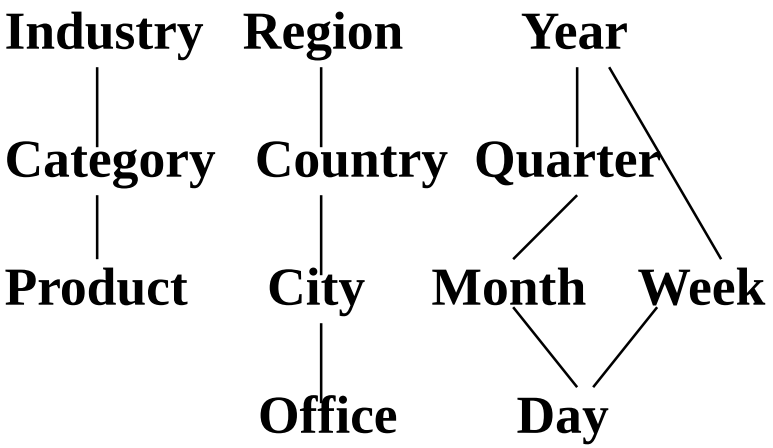
- **分布的**：假设数据被划分为n个集合，将函数用于每一部分，得到n个聚集值。如果将函数用于n个聚集值得到的结果与将函数用于整个数据集（不划分）得到的结果一样
  - —E.g., count(),sum(),min(),max()
- **代数的**：一个聚集函数如果能够用一个具有M个参数的函数计算（其中M是有界正整数），而每个参数可以用一个分布聚集函数求得
  - —E.g., avg(), standard_deviation()
- **整体的**：一个聚集函数如果描述它的子聚集所需的存储没有一个常数界（不存在一个具有M个参数的代数函数进行这一运算，其中M是常数）
  - —E.g.,median()

Multidimensional Data

- **Sales volume as a function of product, month, and region**



Dimensions: ***Product, Location, Time***
Hierarchical summarization paths



Typical OLAP Operations

- **Roll up (drill-up): summarize data**
  - ***by climbing up hierarchy or by dimension reduction***
- **Drill down (roll down): reverse of roll-up**
  - ***from higher level summary to lower level summary or detailed data, or introducing new dimensions***
- **Slice and dice: *project and select***
- **Pivot (rotate):**
  - ***reorient the cube, visualization, 3D to series of 2D planes***
- **Other operations**
  - ***drill across: involving (across) more than one fact table***
  - ***drill through: through the bottom level of the cube to its back-end relational tables (using SQL)***

多维数据模型上的OLAP操作

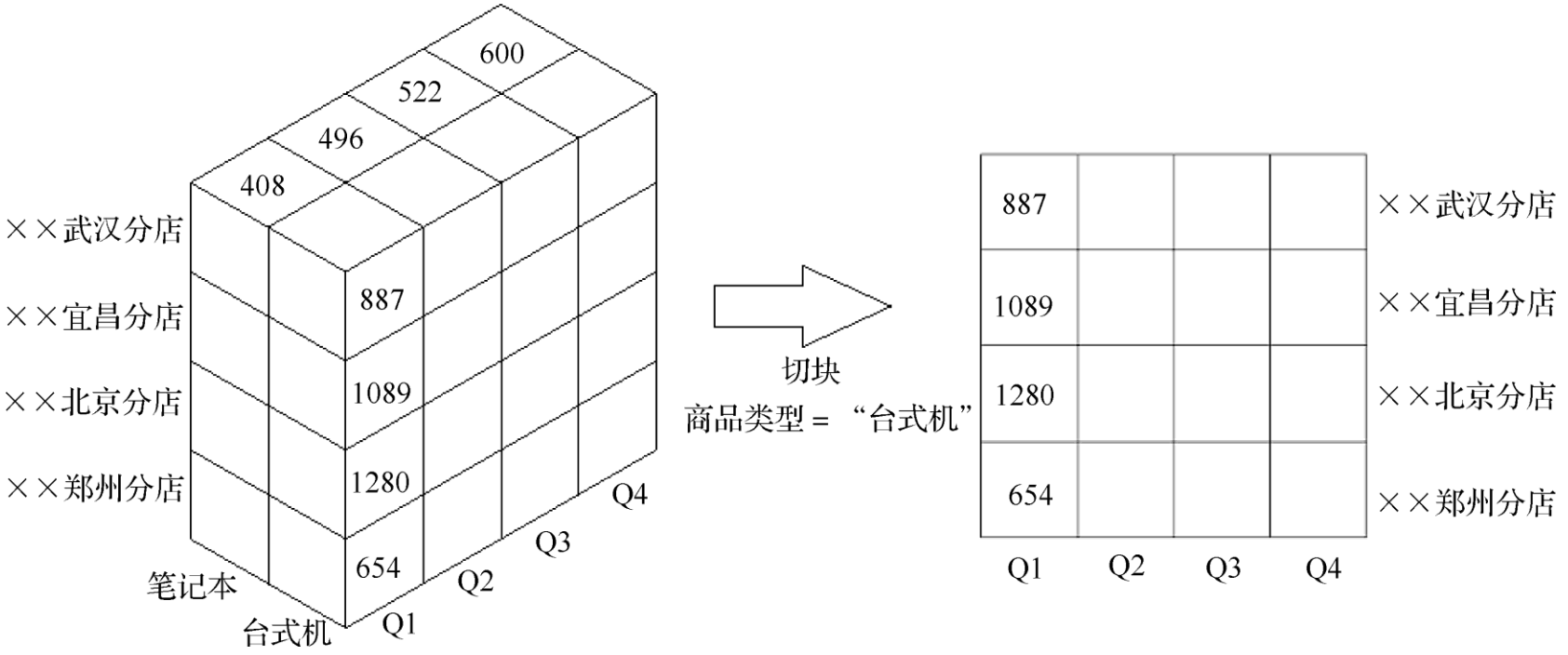
- **上卷(roll-up，也称上钻drill-up):汇总数据**
  - 通过一个维的概念分层向上攀升或者通过维规约在数据立方体上进行聚集
  - 当用维归约进行上卷时，一个或多个维由给定的数据立方体删除
- **下钻(drill-down)：上卷的逆操作**
  - 由不太详细的数据到更详细的数据，可以通过沿维的概念分层向下或引入新的维来实现 (为给定数据添加更多细节)
- **切片和切块(slice and dice)**
  - 切片操作在给定的数据立方体的一个维上进行选择，导致一个子立方体
  - 切块操作通过对两个或多个维进行选择，定义个子立方体

多维数据模型上的OLAP操作Cont.

- **转轴(pivot)**
  - —立方体的重定位，可视化，或将一个3维立方体转化为一个2维平面序列
  - —转轴是一种可视化操作，通过转动当前数据的视图来提供一个数据的替代表示
- **其他OLAP操作**
  - —钻过(drill_across)：执行涉及多个事实表的查询
  - —钻透(drill_through)：使用关系SQL机制，钻到数据立方体的底层，到后端关系表
  - —其他OLAP操作可能包括列出表中最高或最低的N项，以及计算移动平均值、增长率、利润、统计函数等等

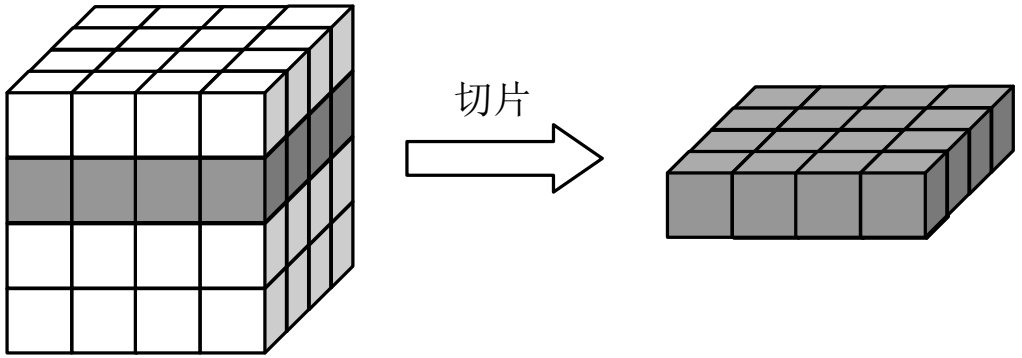
切片（Slice）

- 切片是指选择维中特定区间的数据或者某批特定值进行分析



切片

- 切片定义1
  - 在多维数据集的某一维上选定一个维成员的操作称为切片
  - 例如，在多维数组：（维1，维2，…，维i，…，维n，度量列表）中选定一维，即维i，并取其一维成员（维成员vi），所得的多维数组的子集（维1，…，维i-1，维成员vi，维i+1，…，维n，度量列表）称为维i上的一个切片
- 切片定义2
  - 选定多维数据集的一个两维子集的方法称为切片
  - 例如在多维数组（维1，维2，…，维i，…，维n，度量列表）选定两个维i和维j，在这两个维上取某一区间或任意维成员，而将其余的维都选取一个维成员，则得到的就是多维数据集在维i和维j上的一个二维子集，称这个二维子集为多维数据集在维i和维j上的一个切片，表示为（维i，维j，度量列表）
- 对于n维的多维数据集，切片定义1的结果是n-1维，而切片定义2的结果是二维的。按切片定义1进行n-2次切片会得到切片定义2的结果
  - 无论哪个定义，切片的作用或结果就是合弃一些观察角度，使人们能在较少维（如两个维）上集中观察数据



切片（Slice）

```
SELECT Locates.地区,Products.分类,SUM(数量)
FROM Sales,Dates,Products,Locates
WHERE Dates.季度=2   指定切片条件
   AND Sales.Date_key=Dates.Date_key
   AND Sales.Locate_key=Locates.Locate_key      事实表和维表连接
   AND Sales.Prod_key=Products.Prod_key
GROUP BY Locates.地区,Products.分类 WITH ROLLUP
```

		地区										
		华北			华东		华南		华中			总计
		家用电器	通信设备	汇总	家用电器	汇总	家用电器	汇总	家用电器	通信设备	汇总	
季度												
1	数量	2	2	4	2	2	4	4	4	5	9	19
2	数量	1	2	3	1	1	1	1	3	3	6	11
3	数量	1	2	3	1	1	3	3	5	1	6	13
4	数量	1	1	2	3	3	3	3	2	1	3	11
总计	数量	5	7	12	7	7	11	11	14	10	24	54

对 2 季度进行切片

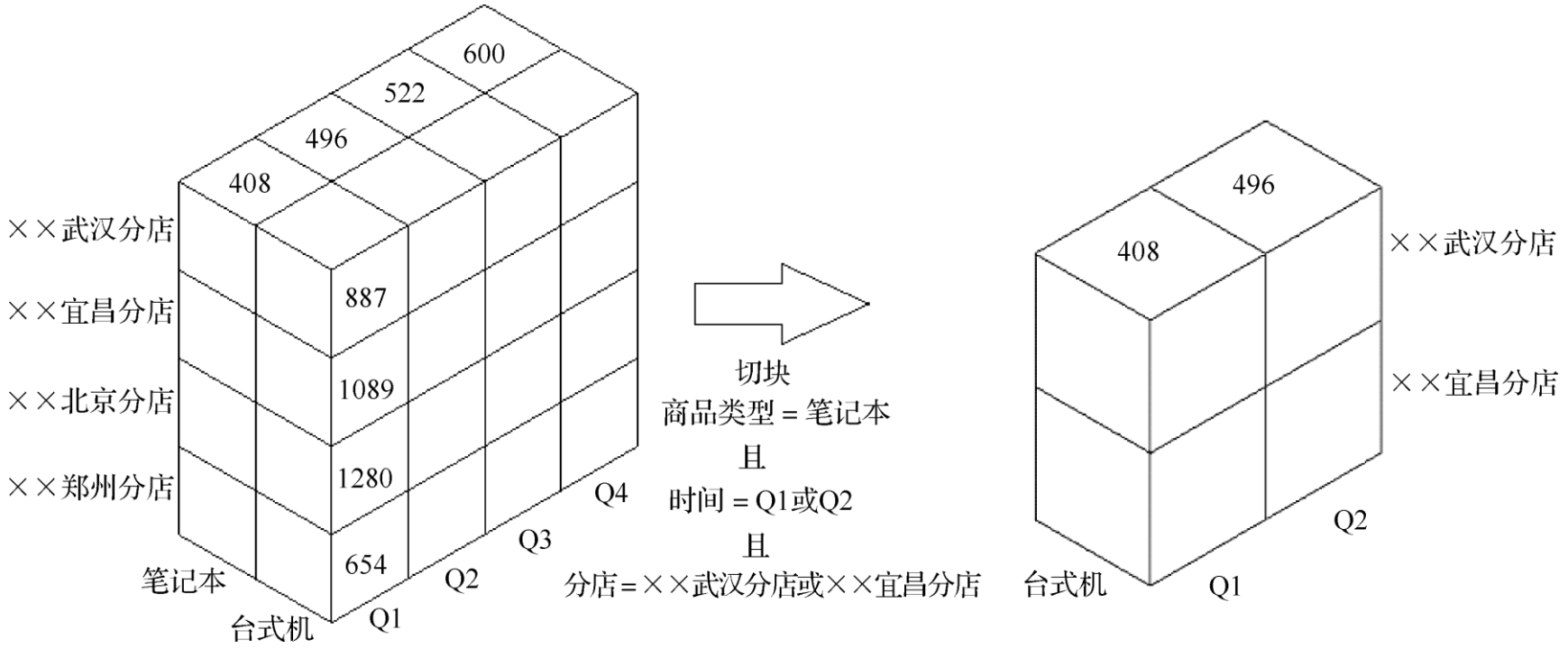
维度	层次结构	运算符	筛选表达式
Dates	季度	等于	{2}
〈选择维度〉			

将筛选字段拖至此处

		地区										
		华北			华东		华南		华中			总计
		家用电器	通信设备	汇总	家用电器	汇总	家用电器	汇总	家用电器	通信设备	汇总	
季度												
2	数量	1	2	3	1	1	1	1	3	3	6	11
总计	数量	1	2	3	1	1	1	1	3	3	6	11

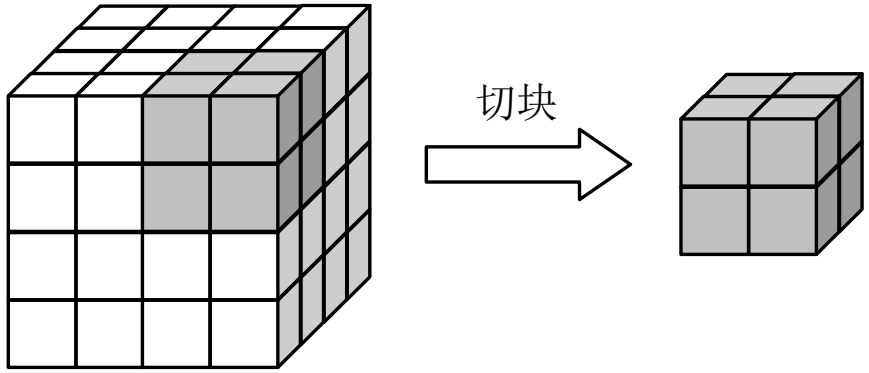
切块（Dice）

- **切块**操作通过在两个或多个维上进行选择，定义子数据立方体



切块（Dice）

- 切块定义1
  - —在多维数据集（维1，维2，…，维n，度量列表）中通过对两个或多个维执行选择得到子集的操作称为切块
- 切块定义2
  - —选定多维数据集的一个三维子集的方法称为切块。例如，选定：（维1，维2，…，维n，度量列表）中的三个维：维i、维j和维k，在这三个维上取某一区间或任意的维成员，而将其余的维都取定一个维成员，则得到的是多维数据集在维i、维j和维k上的一个三维子集，称之为切块，表示为：（维i，维j，维k，度量列表）
- 切块和切片操作的作用是相似的
  - —实际上，切块操作也可以看成进行多次切片，即切块操作结果可以看成进行多次切片叠合而成



切块（Dice）

		地区										
		华北			华东		华南		华中			总计
季度		家用电器	通信设备	汇总	家用电器	汇总	家用电器	汇总	家用电器	通信设备	汇总	
1	数量	2	2	4	2	2	4	4	4	5	9	19
2	数量	1	2	3	1	1	1	1	3	3	6	11
3	数量	1	2	3	1	1	3	3	5	1	6	13
4	数量	1	1	2	3	3	3	3	2	1	3	11
总计	数量	5	7	12	7	7	11	11	14	10	24	54

对 2、3 季度和华东、华南进行切块

维度	层次结构	运算符	筛选表达式
Dates	季度	等于	{ 2, 3 }
Locates	地区	等于	{ 华东, 华南 }
<选择维度>			

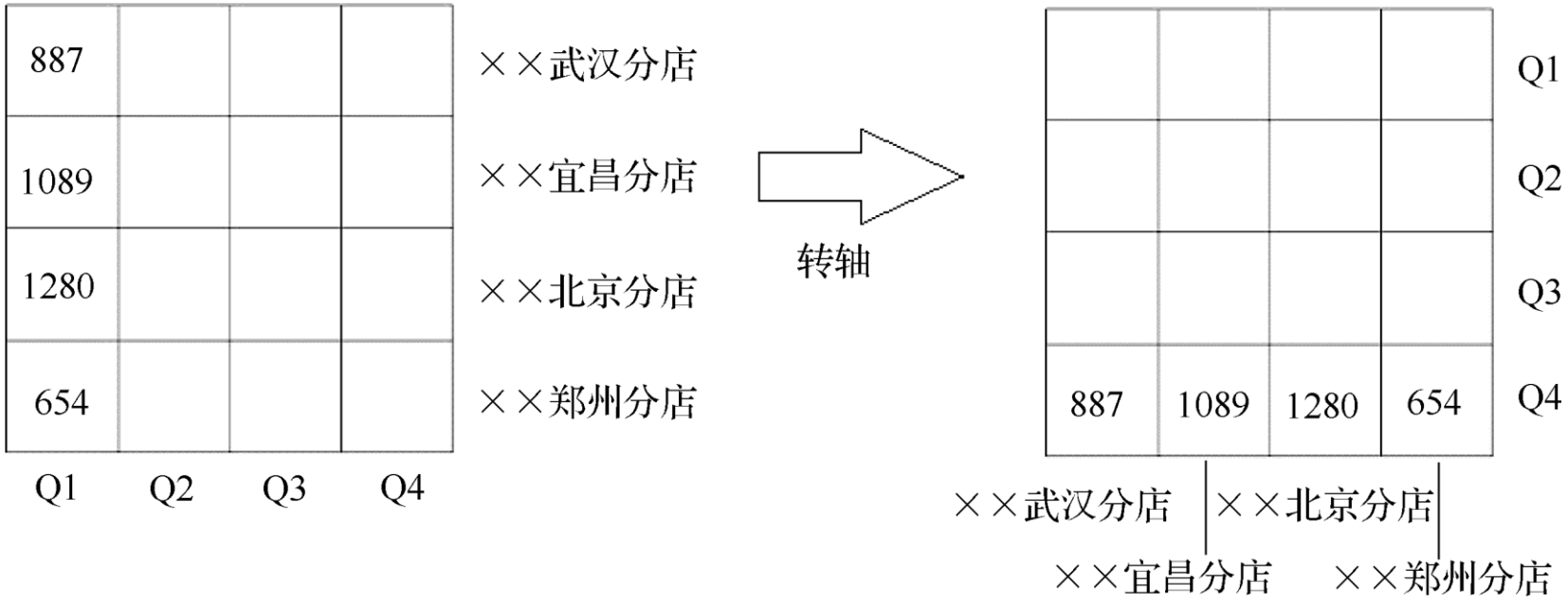
将筛选字段拖至此处

		地区				
		华东		华南		总计
季度		家用电器	汇总	家用电器	汇总	
2	数量	1	1	1	1	2
3	数量	1	1	3	3	4
总计	数量	2	2	4	4	6

```
SELECT Dates.季度,Locates.地区,Products.分类,SUM(数量)
FROM Sales,Dates,Products,Locates
WHERE (Dates.季度=2 OR Dates.季度=3)
      AND (Locates.地区='华东' OR Locates.地区='华南')   指定切块条件
      AND Sales.Date_key=Dates.Date_key
      AND Sales.Locate_key=Locates.Locate_key      事实表和维表连接
      AND Sales.Prod_key=Products.Prod_key
GROUP BY Dates.季度,Locates.地区,Products.分类 WITH ROLLUP
```

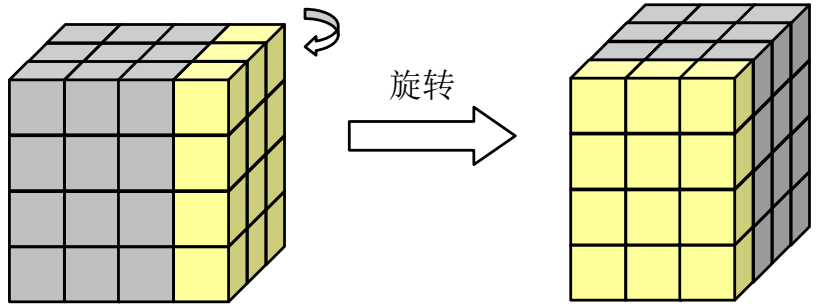
转轴（Pivot）

- 转轴（Pivot）即维的位置的互换，就像是二维表的行列转换。转轴操作只是转动数据的视角，提供数据的替代表示



转轴（Pivot）

- **转轴（Pivot，旋转）是一种视图操作，即改变一个报告或页面显示的维方向，可以得到不同视角的数据，即转动数据的视角以提供数据的替代表示**



	年龄层次 / 分类							
	老年		青年			中年		总计 *
	通信设备	汇总 *	家用电器	通信设备	汇总 *	家用电器	汇总 *	
省份	数量	数量	数量	数量	数量	数量	数量	数量
北京	7	7				5	5	12
广东			11		11			11
湖北				10	10	14	14	24
江苏			7		7			7
总计 *	7	7	18	10	28	19	19	54

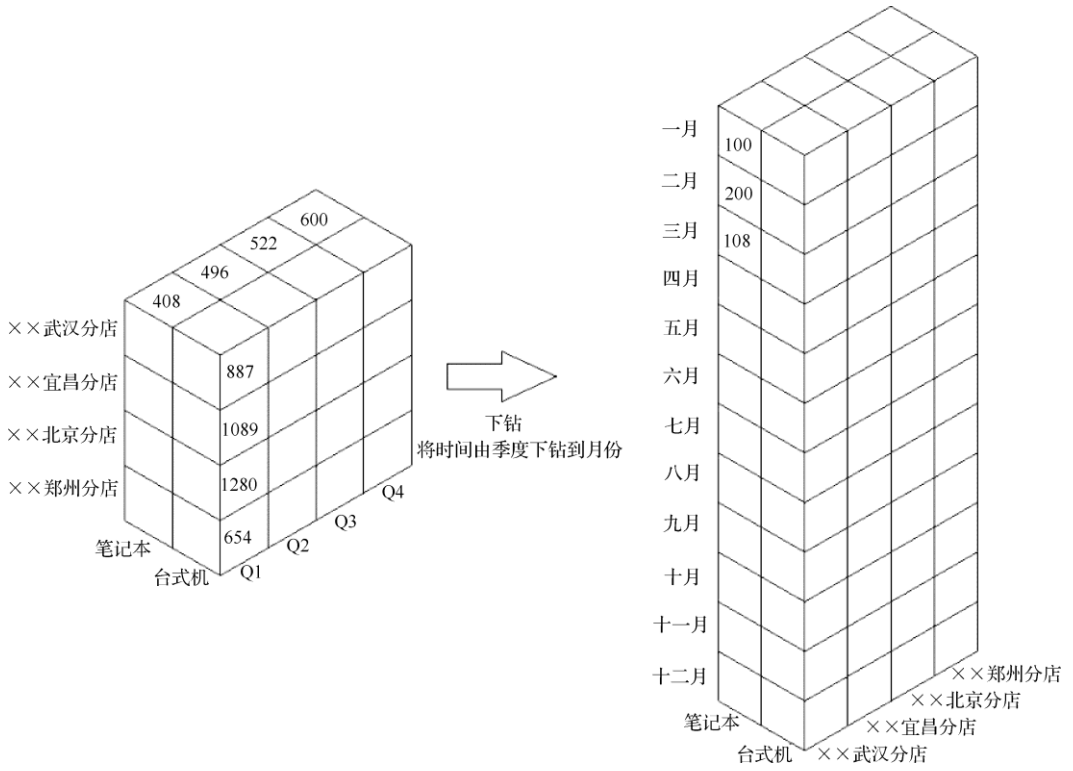
一个多维数据集通过“年龄层次”和“分类”交换的旋转操作结果，将原来各省的各年龄层次顾客购买各类商品的数量，改为各省的各类商品被各年龄层次顾客购买的数量，这样观察的视角发生了改变

对年龄层次和分类旋转

	分类 / 年龄层次						
	家用电器			通信设备			总计 *
	青年	中年	汇总 *	老年	青年	汇总 *	
省份	数量	数量	数量	数量	数量	数量	数量
北京		5	5	7		7	12
广东	11		11				11
湖北		14	14		10	10	24
江苏	7		7				7
总计 *	18	19	37	7	10	17	54

下钻（Drill-down）

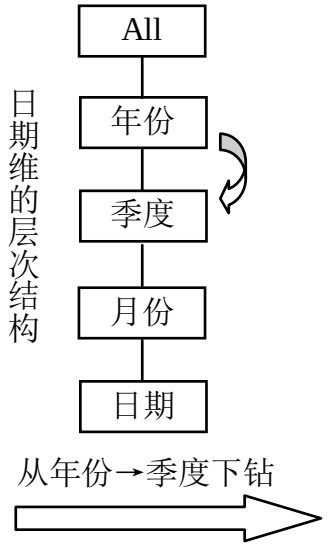
- 下钻是指在维的不同层次间的变化，从上层降到下一层，或者说是将汇总数据拆分成更加细节的数据
  - —如图所示，下钻操作从时间这一维度对数据立方体进行更深一步的细分，从季度下钻到月份，从而能够针对每个月份的数据进行进一步更加细化的分析



下钻（Drill-down）

- **下钻是上卷的逆操作，它由不太详细的数据到更详细的数据**
  - **—使用户在多层数据中能通过导航信息而获得更多的细节数据。下钻可以沿维的概念分层向下或引入新的维或维的层次来实现**

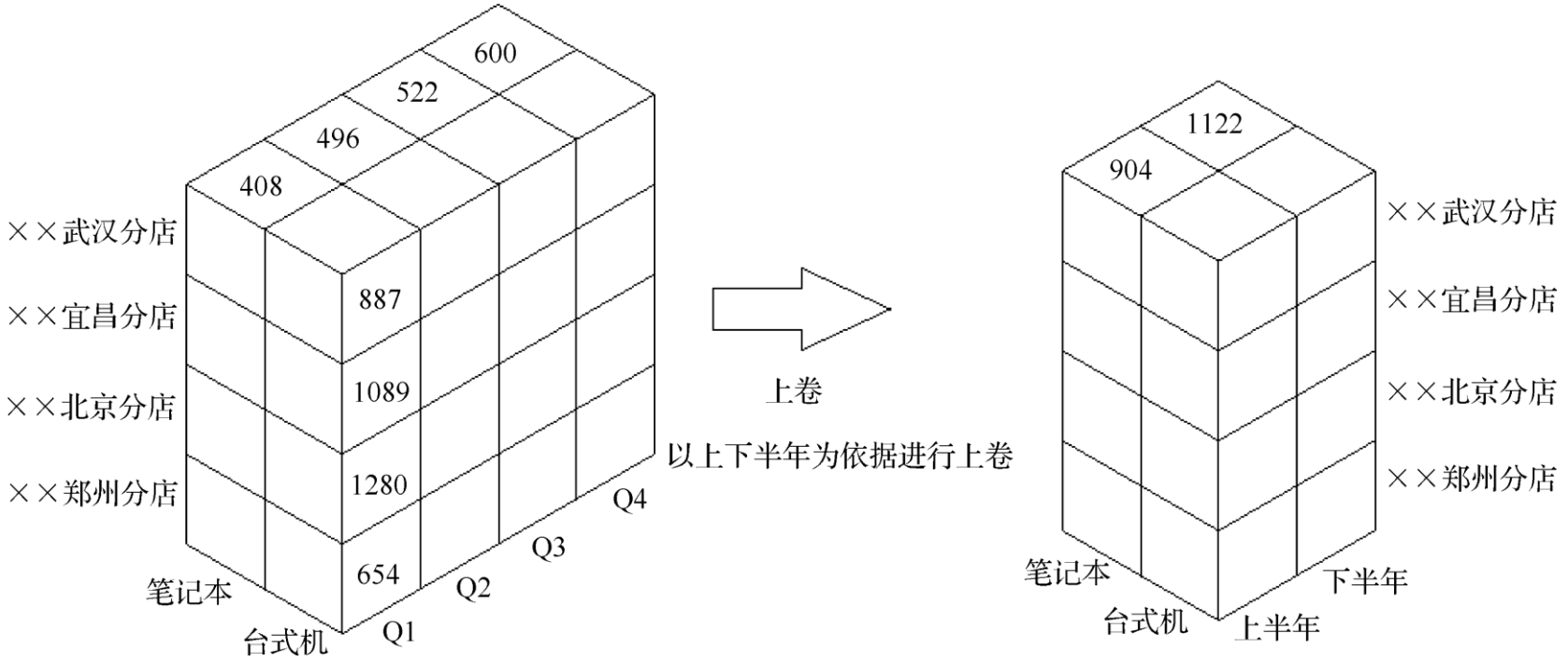
		省份				
		北京	广东	湖北	江苏	总计
年份	季度	数量	数量	数量	数量	数量
⊞2013		10	10	20	6	46
⊞2014		2	1	4	1	8
总计		12	11	24	7	54



		省份				
		北京	广东	湖北	江苏	总计
年份	季度	数量	数量	数量	数量	数量
⊟2013	1	2	3	5	1	11
	2	3	1	6	1	11
	3	3	3	6	1	13
	4	2	3	3	3	11
	汇总	10	10	20	6	46
⊟2014	1	2	1	4	1	8
	汇总	2	1	4	1	8
总计		12	11	24	7	54

上卷（Roll-up）

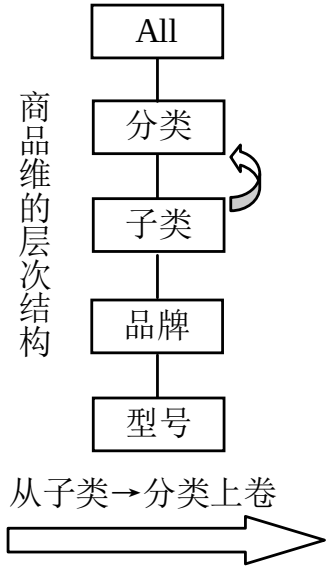
- 上卷实际上就是下钻的逆操作，即从细粒度数据向高层的聚合



上卷（Roll-up）

- 上卷通过维的概念分层向上攀升或者通过维归约（即将4个季度的值加到一起为一年的结果）在数据立方体上进行聚集
  - —如在产品维度上，由产品向小类上卷，可得到小类的聚集数据，再由小类向大类上卷，可得到大类层次的聚集数据

		地区				
		华北	华东	华南	华中	总计
分类	子类	数量	数量	数量	数量	数量
⊟家用电器	电冰箱		7	11		18
	电视机	5			14	19
	汇总	5	7	11	14	37
⊟通信设备	电话				10	10
	手机	7				7
	汇总	7			10	17
总计		12	7	11	24	54



		地区				
		华北	华东	华南	华中	总计
分类	子类	数量	数量	数量	数量	数量
⊞家用电器		5	7	11	14	37
⊞通信设备		7			10	17
总计		12	7	11	24	54

其他OLAP操作

- 除了上述的OLAP基本操作外，有些OLAP系统还提供其他钻取操作
  - —钻过（drill_across）执行涉及多个多维数组（事实表）的查询
  - —钻透（drill_thriugh）操作使用关系SQL机制，钻透数据立方体的底层，到后端关系表
  - —其他OLAP操作可能包括列出表中最高或最低的N项，以及计算移动平均值、增长率、利润、内部返回率、贬值、流通转换和统计功能

一个复杂的查询统计是一系列OLAP基本操作叠加

- 例如

（1）统计2014年的总销售量

—其过程是：在Dates维上卷到“年份”属性，再按“年份=2014”进行切片操作

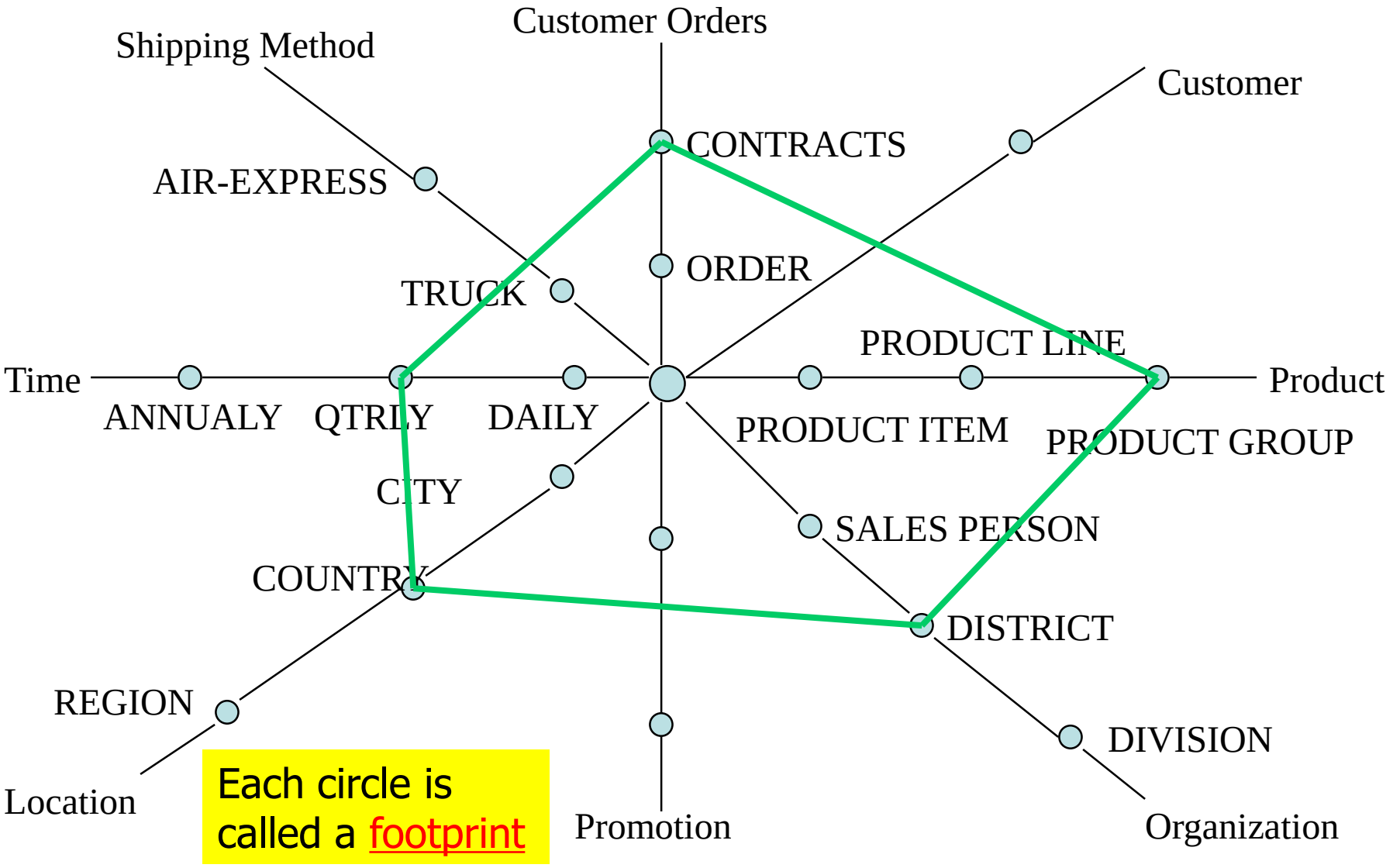
（2）统计“华中”地区“通信设备”的总销售量

—其过程是：在Locates维上卷到“地区”属性，在Products维上卷到“分类”属性，再对“地区=华中 AND 分类=通信设备”进行切块操作

（3）统计2013年“北京市”各年龄层次的顾客购买商品的金额

—其过程是：在Dates维上卷到“年份”属性，在Locates维上卷到“省份”属性，“按“年份=2013 AND 省份=北京”进行切块操作，再在Customers维上卷到“年龄层次”属性

A Star-Net Query Model

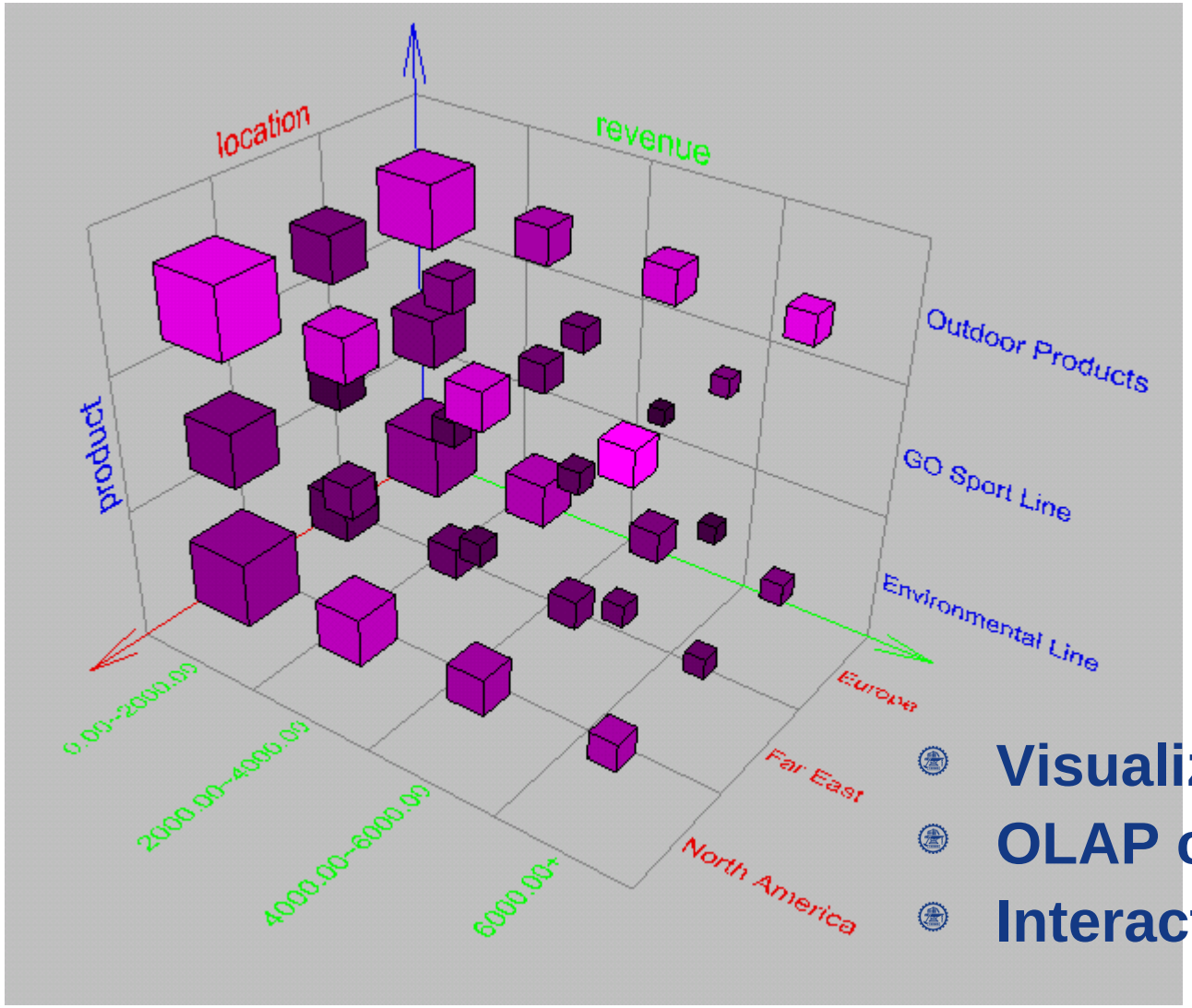


Each circle is called a footprint

星网模型

- 多维数据查询可以基于**星网模型**
  - —由从中心点发出的射线组成，其中每一条射线代表一个维的概念分层
  - —概念分层上的每个“抽象级”称为一个**足迹**，代表诸如上卷，下钻等OLAP可用的粒度
- 通过用较高层抽象（如time维的year）值替换低层抽象（如time维的day）值，概念分层可以用于**泛化**数据。反之，可以**特殊化**数据

Browsing a Data Cube



- Visualization
- OLAP capabilities
- Interactive manipulation

Chapter 5: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- **Data Warehouse Design and Usage**
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- Summary

Data Warehouse: A Business Analysis Framework
数据仓库设计：一个商务分析框架

- **数据仓库给商业分析专家提供了什么？**
  - —通过提供相关数据与信息，获得竞争优势
  - —通过有效的收集精确的描述组织的数据，获得生产力的提高
  - —通过提供不同级别（部门、市场、商业）的客户视图，协助客户关系管理
  - —通过追踪长期趋势、异常等，降低成本

- **有效构建数据仓库的关键：理解和分析商业需求**
  - —通过提供一个商业分析框架，综合各种 **不同** 的数据使用者的 **视图**

Data Warehouse: A Business Analysis Framework
数据仓库设计：一个商务分析框架

- **为了设计有效地数据仓库，**
  需要理解和分析商务需求，并构造一个 商务分析框架

- 构建一个大型复杂的信息系统就像构建一个大型复杂的建筑
  - 业主、设计师、建筑商…… 各自有不同的视图
  - 这些视图结合在一起，形成一个复杂的框架
  - E.g., 业主的视图：自顶向下，商务驱动的视图
    信息系统实现者的视图：自底向上，建筑商驱动的视图

Design of Data Warehouse: A Business Analysis Framework

- **Four views regarding the design of a data warehouse**
  - **—Top-down view**
    - allows selection of the relevant information necessary for the data warehouse
  - **—Data source view**
    - exposes the information being captured, stored, and managed by operational systems
  - **—Data warehouse view**
    - consists of fact tables and dimension tables
  - **—Business query view**
    - sees the perspectives of data in the warehouse from the view of end-user

Data Warehouse: A Business Analysis Framework
数据仓库设计：一个商务分析框架

- 数据仓库设计的 **四种视图**

  —自顶向下视图
    - 允许我们选择数据仓库所需的相关信息

  —数据源视图
    - 揭示被操作数据库系统所捕获、存储和管理的信息

  —数据仓库视图
    - 由事实表和维表所组成

  —商务查询视图
    - 从最终用户的角度透视数据仓库中的数据

Data Warehouse Design Process

- Top-down, bottom-up approaches or a combination of both
  - Top-down: Starts with overall design and planning (mature)
  - Bottom-up: Starts with experiments and prototypes (rapid)
- From software engineering point of view
  - Waterfall: structured and systematic analysis at each step before proceeding to the next
  - Spiral: rapid generation of increasingly functional systems, short turn around time, quick turn around
- Typical data warehouse design process
  - Choose a business process to model, e.g., orders, invoices, etc.
  - Choose the *grain* (*atomic level of data*) of the business process
  - Choose the dimensions that will apply to each fact table record
  - Choose the measure that will populate each fact table record

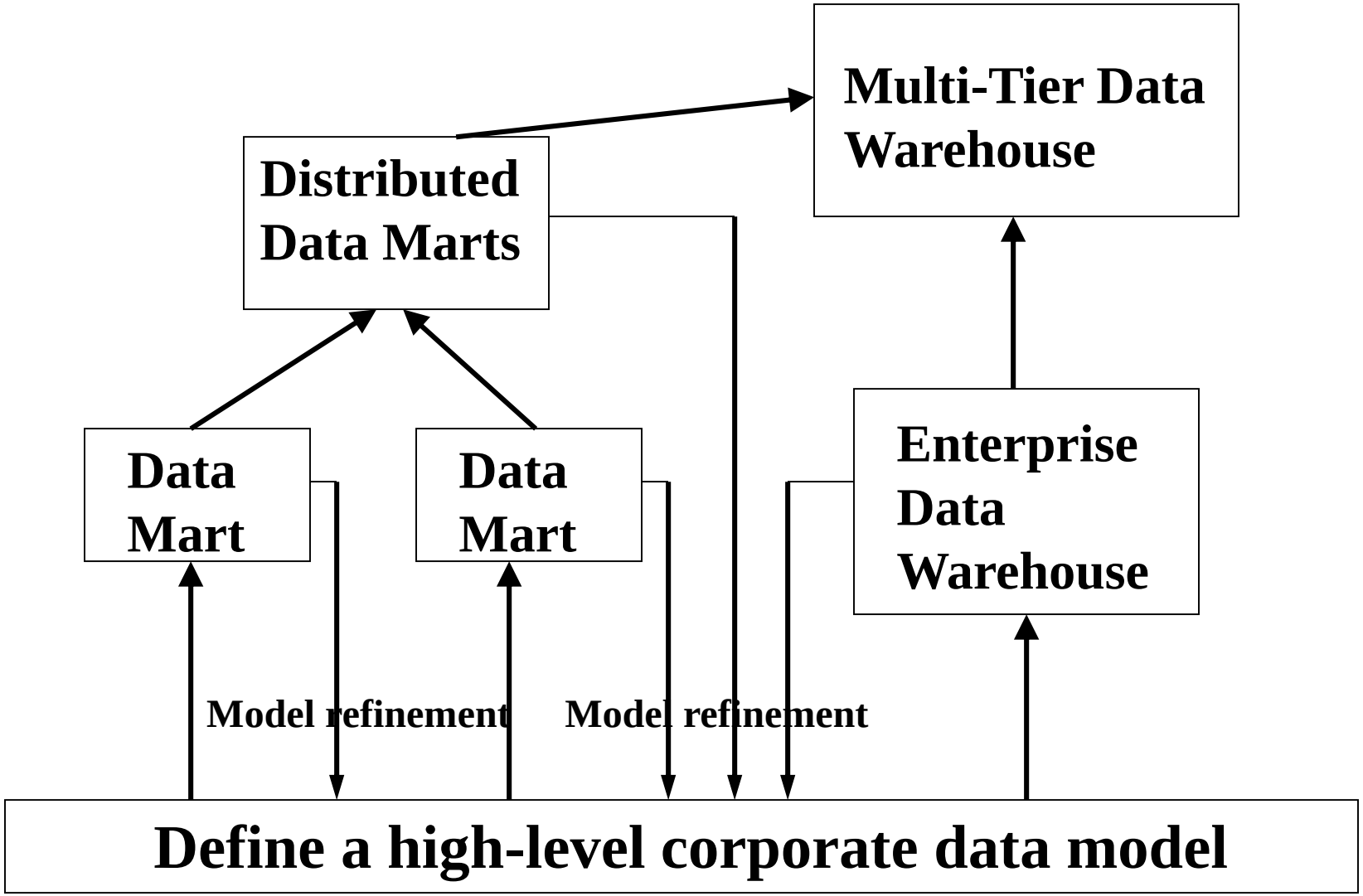
Data Warehouse Design Process
数据仓库的设计过程

- 自顶向下法、自底向上法或者两者的混合方法
  - —自顶向下法：由总体设计和规划开始
    - 在技术成熟、商业理解透彻的情况下使用
  - —自底向上法：以实验和原型开始
    - 常用在模型和技术开发的初期，可以有效的对使用的技术和模型进行评估，降低风险
  - —混合方法：上述两者的结合
- 从软件过程的观点
  - —瀑布式方法：在进行下一步前，每一步都进行结构化和系统的分析
  - —螺旋式方法：功能渐增的系统的快速产生，相继版本之间间隔很短

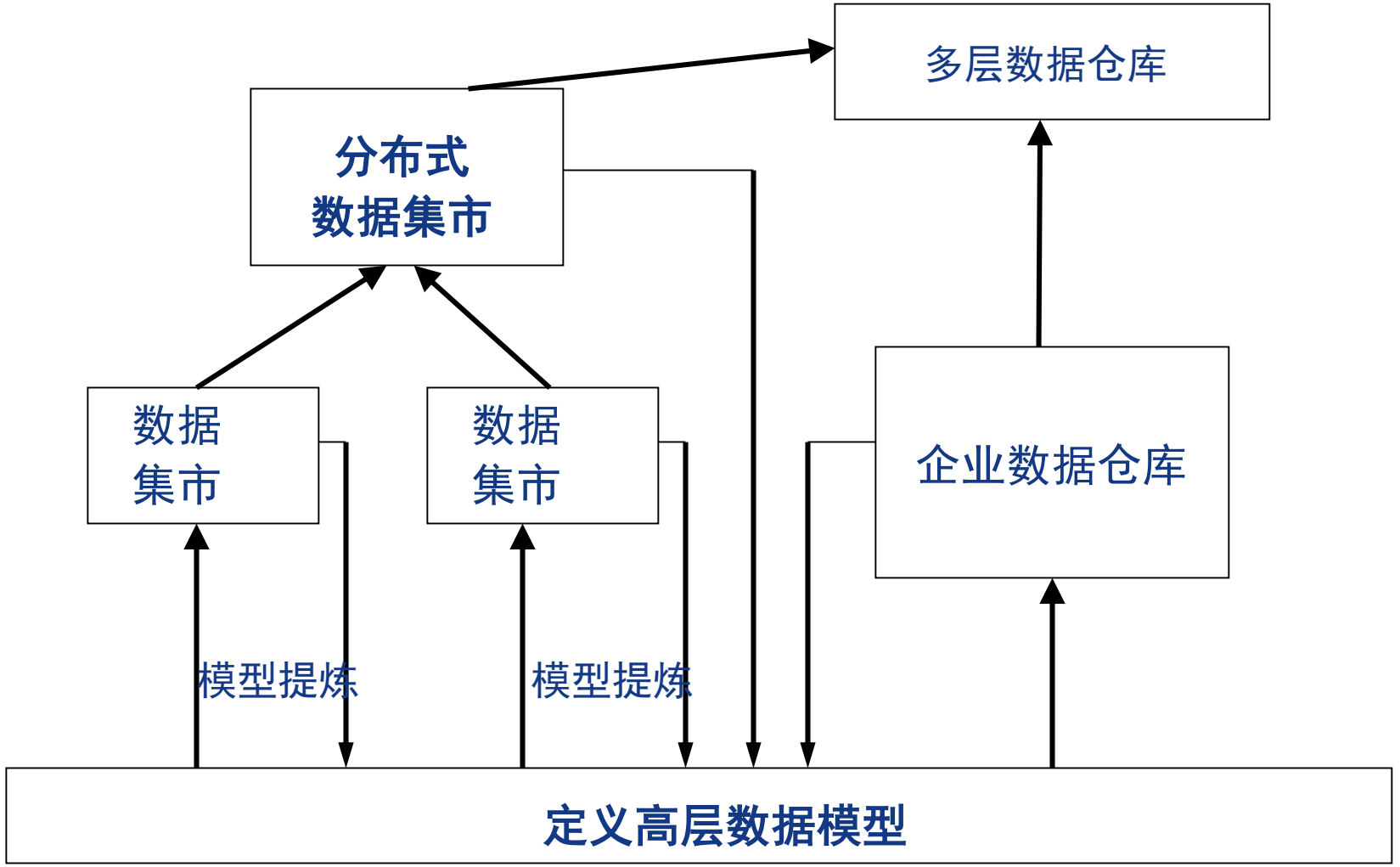
Data Warehouse Design Process
数据仓库的设计过程

- 数据仓库的设计过程包含如下步骤：
  - 选取待建模的商务处理（例如，订单、发票、发货、库存、记账管理、销售或一般分类账）。如果一个商务过程是整个组织的，并涉及多个复杂的对象，应当选用数据仓库模型。如果处理是部门的，并关注某一类商务处理的分析，则应选择数据集市。
  - 选取商务处理的粒度。对于处理，该粒度是基本的，在事实表中是数据的原子级（例如，单个事务、一天的快照等）
  - 选取用于每个事实表记录的维。典型的维是时间、商品、顾客、供应商、仓库、事务类型和状态。
  - 选取将安放在每个事实表记录中的度量。典型的度量是可加的数值量，dollars_sold和units_sold。

Data Warehouse Development: A Recommended Approach



Data Warehouse Design Process
数据仓库的设计过程



数据仓库开发：自顶向下 V.S. 自底向上

数据仓库开发上的困难

—自顶向下方法

- 从全系统的角度提供解决方案，使得（模块）集成的问题最小；但是该方法十分昂贵，需要对组织进行长期研究和建模分析。

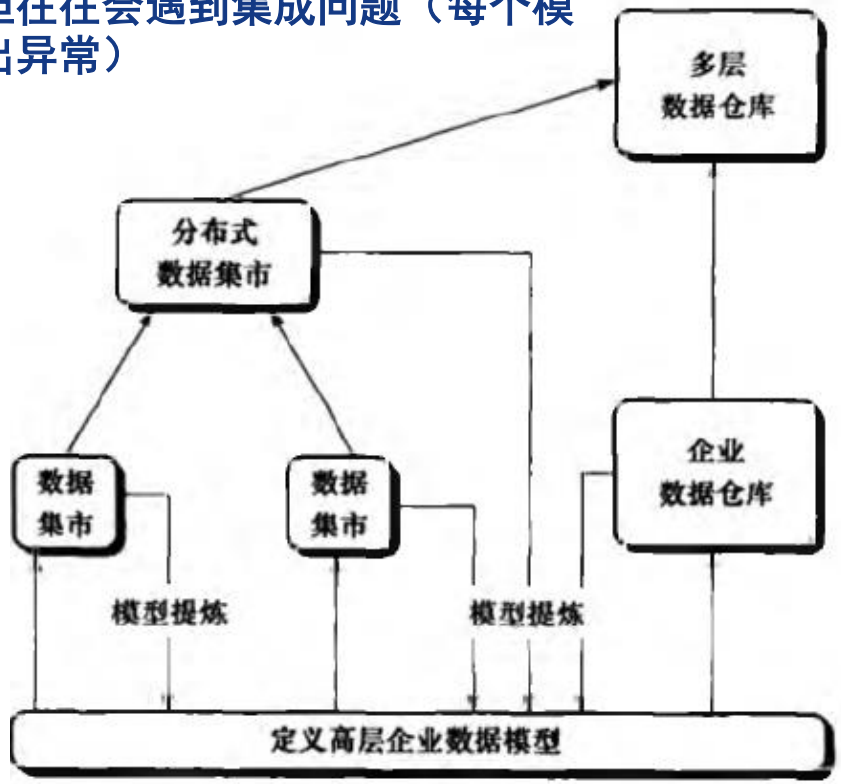
—自底向上方法

- 提供了更多的开发灵活性，价格便宜；但往往会遇到集成问题（每个模块单独运行都没有问题，但是一集成就出异常）

解决方法：

—使用递增性、演化性的开发方法

- 高层数据模型
- 企业仓库和数据集市并行开发
- 通过分布式模型集成各数据集市
- 多层数据仓库



Data Warehouse Usage

- Three kinds of data warehouse applications
  - —Information processing
    - supports querying, basic statistical analysis, and reporting using crosstabs, tables, charts and graphs
  - —Analytical processing
    - multidimensional analysis of data warehouse data
    - supports basic OLAP operations, slice-dice, drilling, pivoting
  - —Data mining
    - knowledge discovery from hidden patterns
    - supports associations, constructing analytical models, performing classification and prediction, and presenting the mining results using visualization tools

Three Application of Data Warehouse
数据仓库的应用

- 数据仓库的三种应用
  - —信息处理
    - 支持查询和基本的统计分析，并使用交叉表、表、图标和图进行报表处理
  - —分析处理
    - 对数据仓库中的数据进行多维数据分析
    - 支持基本的OLAP操作，切块、切片、上卷、下钻、转轴等
  - —数据挖掘
    - 从隐藏模式中发现知识
    - 支持关联分析，构建分析性模型，分类和预测，并用可视化工具呈现挖掘的结果

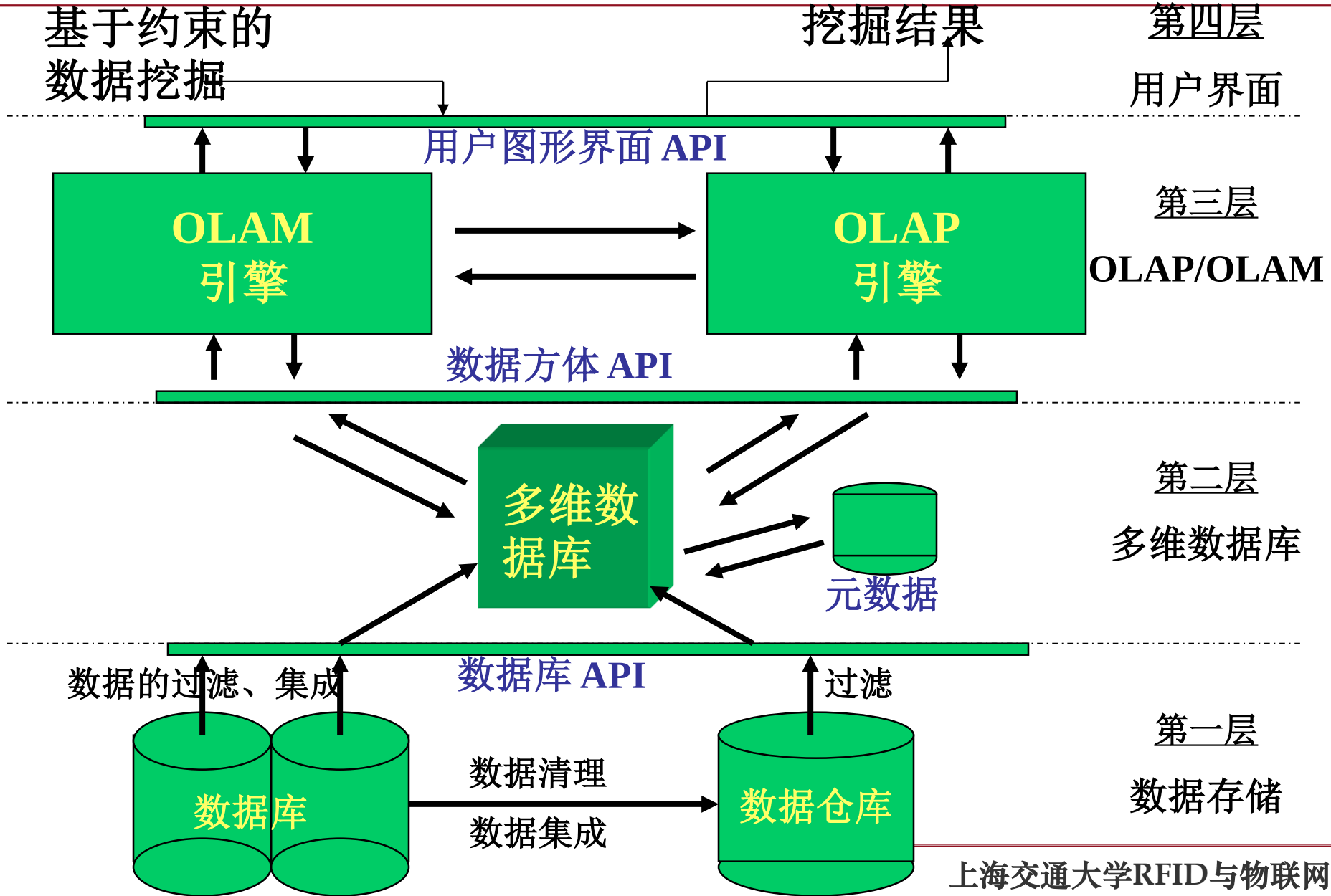
From On-Line Analytical Processing (OLAP) to On Line Analytical Mining (OLAM)

- Why online analytical mining?
  - —High quality of data in data warehouses
    - DW contains integrated, consistent, cleaned data
  - —Available information processing structure surrounding data warehouses
    - ODBC, OLEDB, Web accessing, service facilities, reporting and OLAP tools
  - —OLAP-based exploratory data analysis
    - Mining with drilling, dicing, pivoting, etc.
  - —On-line selection of data mining functions
    - Integration and swapping of multiple mining functions, algorithms, and tasks

从联机分析处理到联机分析挖掘

- 为什么要联机分析挖掘
  - —数据仓库中有高质量的数据
    - 数据仓库中存放着整合的、一致的、清理过的数据
  - —围绕数据仓库的信息处理结构
    - 存取、集成、合并多个异种数据库的转换，ODBC/OLEDB连接,Web访问和访问工具等
  - —基于OLAP的探测式数据分析
    - 使用上卷、下钻、切片、转轴等技术进行数据挖掘
  - —数据挖掘功能的联机选择
    - 多种数据挖掘功能、算法和任务的整合

联机分析挖掘的体系结构



Chapter 5: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- **Data Warehouse Implementation**
  数据仓库的实现
- Summary

数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

Efficient Data Cube Computation

- Data cube can be viewed as a lattice of cuboids
  - —The bottom-most cuboid is the base cuboid
  - —The top-most cuboid (apex) contains only one cell
  - —How many cuboids in an n-dimensional cube with L levels?

$$T = \prod_{i=1}^{n} (L_i + 1)$$

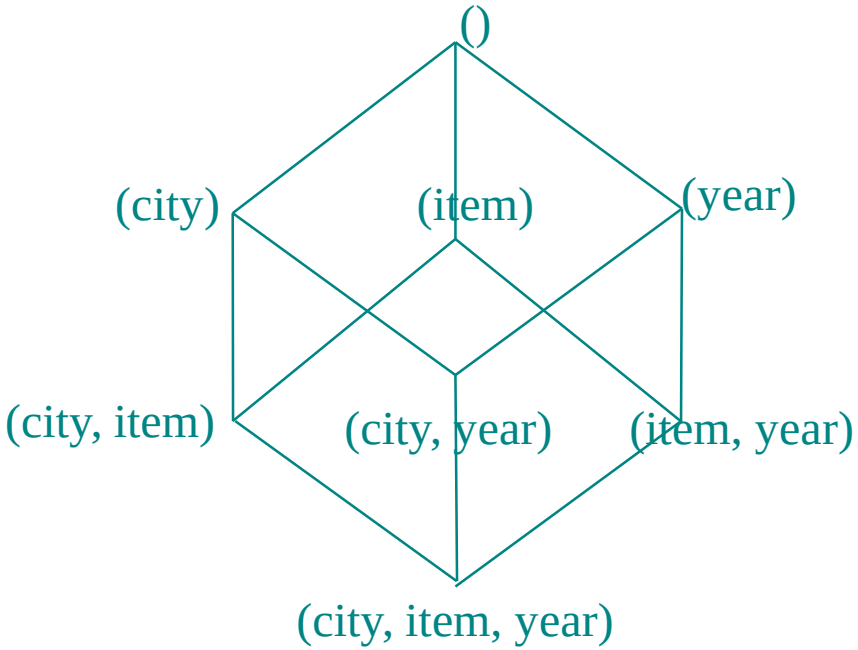
- Materialization of data cube
  - —Materialize every (cuboid) (full materialization), none (no materialization), or some (partial materialization)
  - —Selection of which cuboids to materialize
    - Based on size, sharing, access frequency, etc.

数据仓库的实现——数据立方体的有效计算

- 数据仓库中的OLAP查询是一种海量数据计算 （想象一下对过去10年各地区的软件产品销售的汇总查询）
  - —用户却希望这个计算能在数秒钟内完成
  - —解决方法在于给出一种有效的计算数据立方体的方法`
- 数据立方体可以被看成是一个***方体的格***
  - —最底层的方体是基本方体
  - —最顶端的方体（顶点）只包含一个单元的值
  - —一个n维的数据立方体，每维L层，可能产生的方体总数是多少？

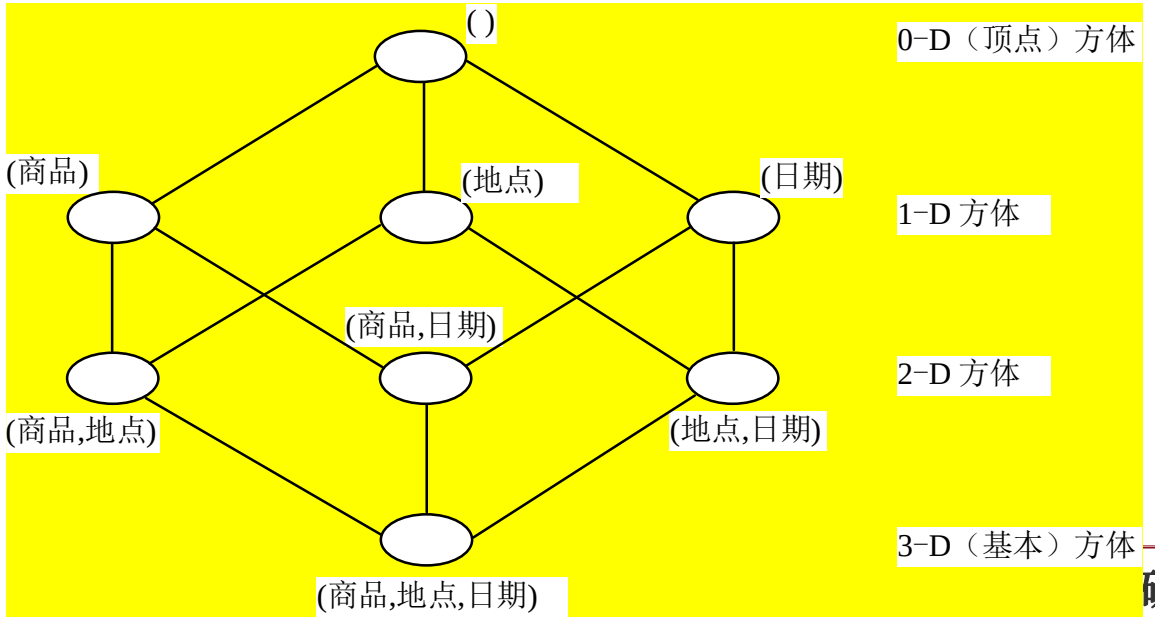
$$T = \prod_{i=1}^{n} (L_i + 1)$$

- **维灾难**
  - —**E.g. 假设数据立方体有10个维，每维5层，则可能产生的方体总数为 $5^{10} \approx 9.8 \times 10^6$个**



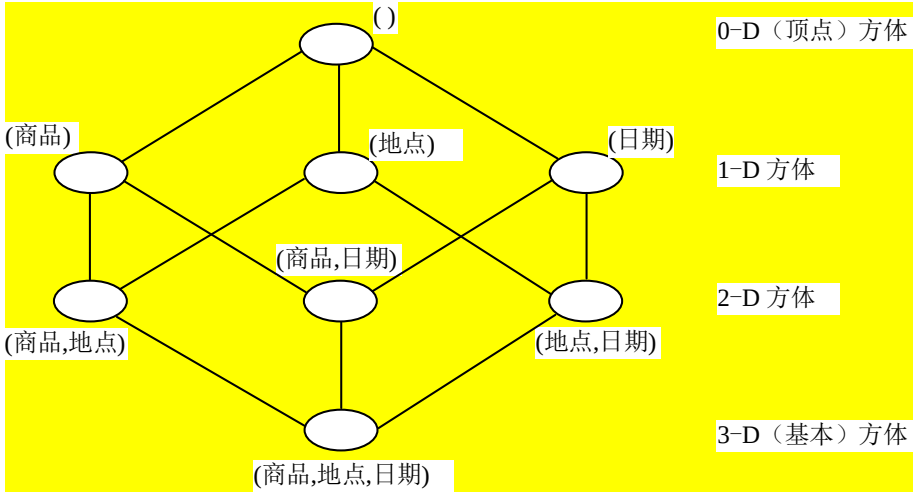
数据立方体的预计算与维灾难

- 给定一个维的集合，例如，对于销售数据立方体(商品，地点，日期，销售量)，其维集合为{商品，地点，日期}，由它可以构造一个方体的格，其中0维方体存放最高层的汇总，称作**顶点方体**，是总销售量在所有的三个维汇总，也就是该立方体中所有数据的总销售量，顶点方体通常用All标记，它仅包含一个值。存放最底层汇总的方体称为**基本方体**，图中3维方体是给定维时间、地点和商品的基本方体。除基本方体外的其他方体称为**非基本方体**，基本方体中数据单元称为**基本单元**，非基本方体中数据单元称为**聚集单元**
- 每个方体在不同的汇总级显示数据，汇总级越高，其维数就越小，对应的数据量（元组个数）也越少，也就是说，一个k+1维方体的数据量大于其中在任一维上聚集得到的k维方体的数据量



数据立方体的预计算与维灾难

- 一个OLAP查询问题可以转化成方体的计算
- 求某个*k*维方体有以下两种方法
  - （1）通过基本方体进行聚集产生*k*维方体
    - 由于基本方体维数最多，其数据量最大，所以该方法是十分耗时的
  - （2）从*k*+1维方体计算出*k*维方体
    - 如给定(商品，地点)这个2维方体，在各维上聚集可以产生(商品)和(地点)两个1维方体，显然比从(商品，地点，日期)基本方体产生(商品)和(地点)方体要省时，但需要预先计算出(商品，地点)方体



数据立方体的预计算与维灾难

- 从方法（2）看出，对于不同的查询分析，OLAP可能需要访问不同的方体
- 提前计算数据立方体中所有的或者一部分方体，这称为预计算。预计算带来快速的响应时间，并避免一些冗余计算
- 然而，预计算的主要挑战是，如果数据立方体中所有的方体都预先计算，所需的存储空间可能爆炸，特别是当立方体包含许多维，这许多维都具有多层的概念分层时，存储需求甚至更多，那么这个数据量究竟有多少呢？

  —给定n维{d1，d2，…，dn}的数据集，对于任何一个维di，假设在聚集时仅从自身或All两个值中选取，所以可以生成聚集方体的个数 $\prod_{i=1}^{n} 2 = 2^n$

数据立方体的预计算与维灾难

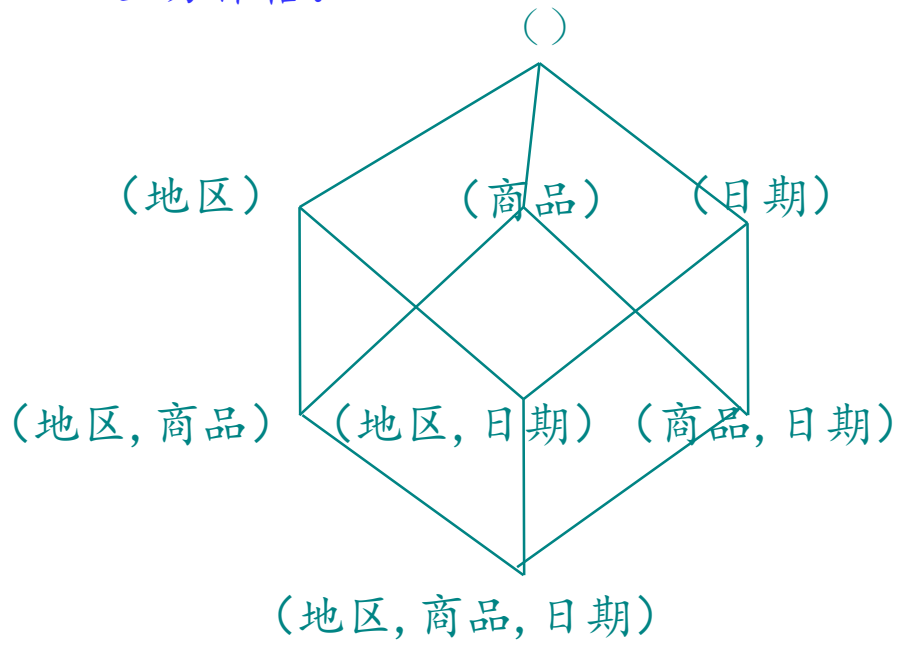
- 如果在聚集时，假设每个维d_i只有一层，聚集数据单元从维d_i（$1 \leqslant i \leqslant n$）的可能取值为该维的不同属性取值个数加上All，即$|d_i|+1$（$|d_i|$表示维d_i的不同取值个数），所以由n个维聚集的方体中聚集的数据单元个数为这n个维中各个维属性的可能取值个数的乘积，即 $\prod_{i=1}^{n}(|d_i|+1)$

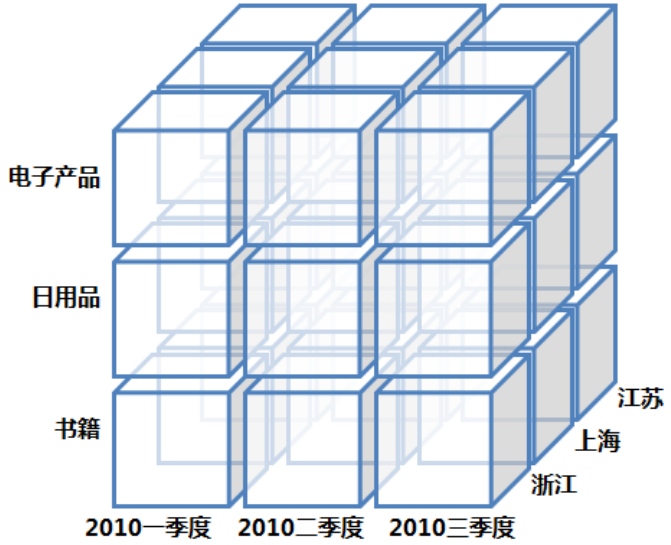
- 如果在聚集时，假设每个维di有hi层，聚集数据单元从维di（1≤i≤n）的可能取值为该维的各层的属性取值，则可产生 $\prod_{i=1}^{n}(h_i+1)$ 个聚集方体

- 总的聚集数据单元个数为 $\prod_{i=1}^{n}\prod_{j=1}^{h_i}(|l_{i,j}|+1|)$ ，其中|li，j|表示维di的第j层上不同取值的个数

数据立方体的预计算与维灾难

立方体格：



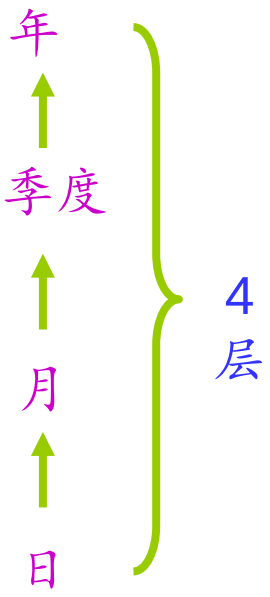


当L_i=1时（没有概念分层），$T=2^n$

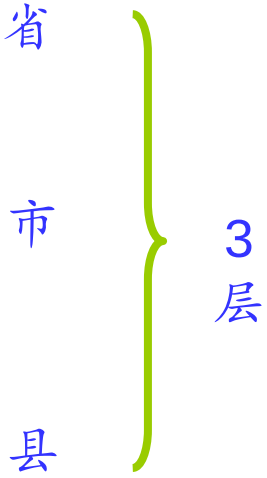
这里有8个立方体

数据立方体的预计算与维灾难

假设日期:



假设地区:



假设商品:



则方体总数$T=(4+1)\times(3+1)\times(2+1)=60$个



All

数据立方体的预计算与维灾难

- 如果一个多维数据集有10维，每维5层（含All层），则可能产生的方体总数是$5^{10}\approx9800000$，而总的聚集数据单元个数更是大的惊人
- 这种当数据立方体的维数较多和各维的层次较多时，可能的聚集计算量剧增，导致存储空间出现爆炸的现象称作维灾难

The "Compute Cube" Operator

- **Cube definition and computation in DMQL**

  define cube sales [item, city, year]: sum (sales_in_dollars)

  compute cube sales

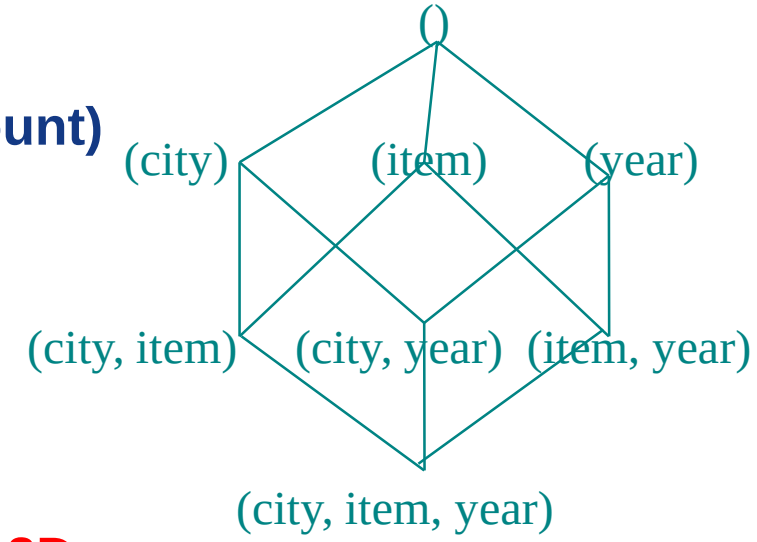
- **Transform it into a SQL-like language (with a new operator cube by, introduced by Gray et al.'96)**

  SELECT item, city, year, SUM (amount)

  FROM SALES

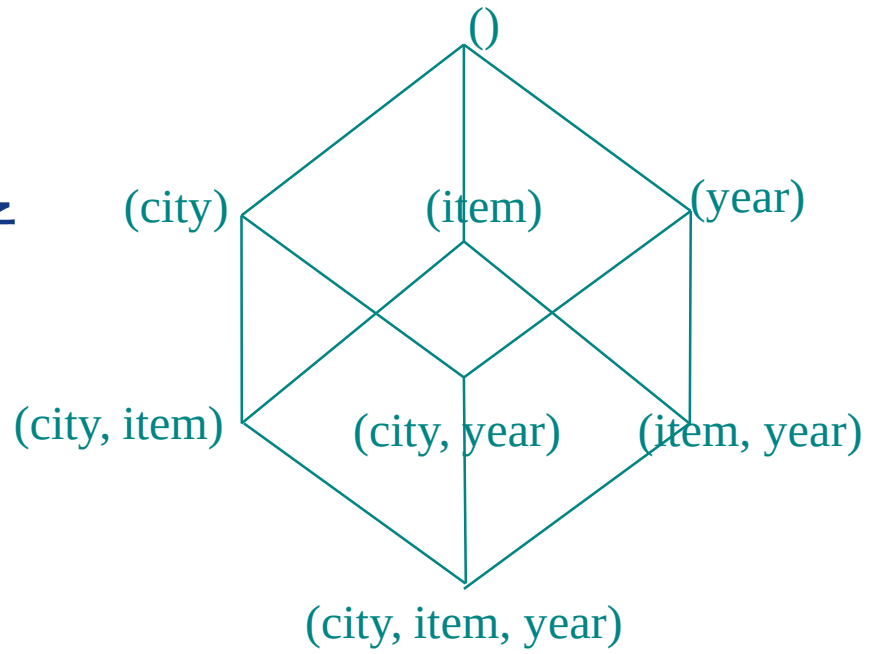
  CUBE BY item, city, year

- **Need compute the following Group-Bys**
  - **(item, city, year) –3D**
  - **(item, city), (item year), (city, year) —2D**
  - **(item), (city), (year) —1D**
  - **() --0D**



数据仓库的实现—— 方体的操作

- DMQL中的方体定义和计算
  - — *define cube* sales[item, city, year]: sum(sales_in_dollars)
  - — *compute cube* sales
- 上述的compute cube子句可以转化为一个类似于SQL的语句
  - — SELECT item, city, year, SUM (amount)
  - — FROM SALES
  - — *CUBE BY* item, city, year
- 这个相当于SQL中以下的group by子句
  - — (item, city, year) –3D
  - — (item, city), (item year), (city, year) —2D
  - — (item), (city), (year) —1D
  - — () --0D



一种数据挖掘查询语言: DMQL

- DMQL首先包括定义数据仓库和数据集市的语言原语，这包括两种原语定义：一种是立方体定义，一种是维定义
  - —立方体定义 (事实表)
    - define cube <cube_name> [<dimension_list>]: <measure_list>
  - —维定义 (维表)
    - define dimension <dimension_name> as (<attribute_or_subdimension_list>)
  - —特殊案例 (共享维表的定义)
    - 第一次作为维表定义 “cube definition”
    - 然后：define dimension <dimension_name> as <dimension_name_first_time> in cube <cube_name_first_time>

实例：使用DMQL定义星型模式

```
define cube sales_star [time, item, branch,
    location]:
        dollars_sold = sum(sales_in_dollars),
          avg_sales = avg(sales_in_dollars), units_sold
          = count(*)
define dimension time as (time_key, day,
    day_of_week, month, quarter, year)
define dimension item as (item_key, item_name,
    brand, type, supplier_type)
define dimension branch as (branch_key,
    branch_name, branch_type)
define dimension location as (location_key, street,
    city, province_or_state, country)
```

实例：使用DMQL定义雪花模式

define cube sales_snowflake [time, item, branch, location]:

dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)

define dimension time as (time_key, day, day_of_week, month, quarter, year)

define dimension item as (item_key, item_name, brand, type, supplier(supplier_key, supplier_type))

define dimension branch as (branch_key, branch_name, branch_type)

define dimension location as (location_key, street, city(city_key, province_or_state, country))

实例：使用DMQL定义事实星座模式

```
define cube sales [time, item, branch, location]:
        dollars_sold = sum(sales_in_dollars), avg_sales =
          avg(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter,
    year)
define dimension item as (item_key, item_name, brand, type,
    supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city,
    province_or_state, country)
define cube shipping [time, item, shipper, from_location, to_location]:
        dollar_cost = sum(cost_in_dollars), unit_shipped = count(*)
define dimension time as time in cube sales
define dimension item as item in cube sales
define dimension shipper as (shipper_key, shipper_name, location as
    location in cube sales, shipper_type)
define dimension from_location as location in cube sales
define dimension to_location as location in cube sales
```

数据立方体的物化

- 数据立方体的物化就是预计算并存储数据立方体的方体，有三种选择

—不物化

  - 不预先计算任何“非基本”方体，这可能导致回答查询时，因进行昂贵的多维聚集计算，速度非常慢

—全物化

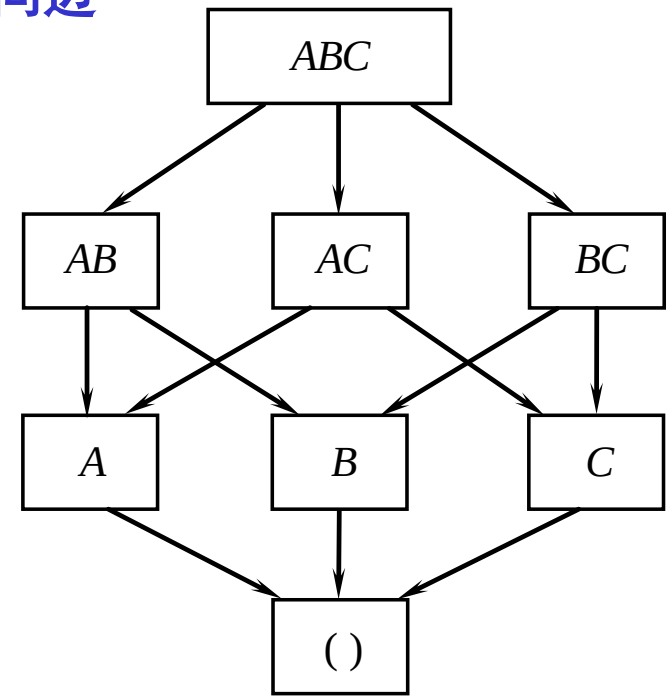
  - 预先计算所有方体，可能产生维灾难 $T=\prod_{i=1}^{n}(L_i+1)$

—部分物化

  - 有选择的计算一个所有方体的适当子集
  - 考虑因素：(1) 确定要物化的方体，考虑工作负荷下的查询、它们的频率和它们的开销等等；(2) 在查询时利用物化的方体；(3) 在装载和刷新时，有效的更新物化的方体

全物化：基于依赖关系的聚集计算

- 可以从k+1维方体通过单个维聚集产生k维方体，称这两个方体之间有依赖关系，由依赖关系构成**依赖图**
- 依赖图的思路是
  - —为了计算所有方体，将所有方体作为图的结点，如果两个方体的维数相差1，则较小的方体可以通过维聚集用较大的方体计算出来
  - —有这种关系的结点间，就从较大的方体到较小的方体之间画一条有向边



对于3维立方体（A，B，C），对应的依赖图中每个结点代表了一种方体，从根结点（ABC对应的结点）到该结点的路径指出计算该方体的过程

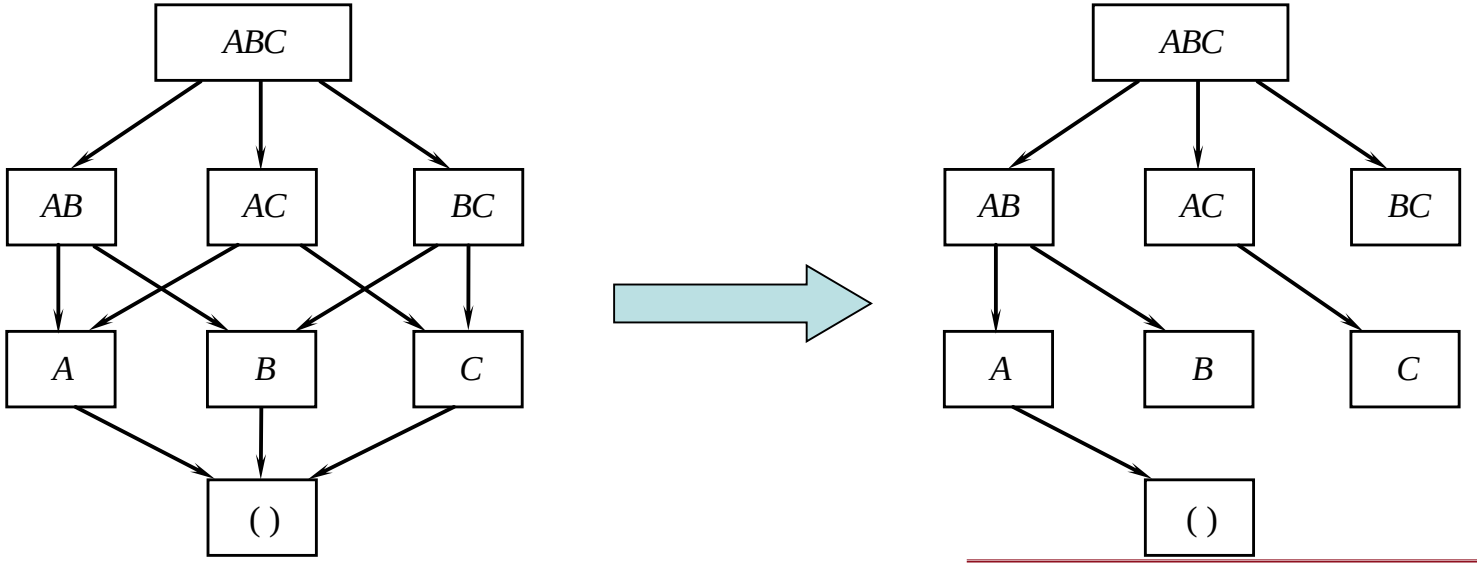
全物化：基于依赖关系的聚集计算

- 在全物化时，需要计算出依赖图中所有结点对应的方体，这一过程是十分耗时的。改进的方法是构造出依赖树，构造依赖树的过程如下：

① 对于n维立方体cube[d1，d2，…，dn]，|di|表示维di的大小，即不同维成员的个数，对cube接各维排序，使得|d1|≤|d2|≤…≤|dn|

② 对同一层的所有结点排序，使得各维按词典序排列

③ 从高维到低维，从小到大处理每个结点p：对p的每一个减1维子方体q，若q没有父结点，从p引一条有向边到q



部分物化

- 部分物化是指在部分维及其相关层次上进行聚集，即从数据立方体的所有方体中选择一个子集进行物化
- 在一般情况下，通常20%的聚集就能够满足80%的查询需要，如何确定该20%的聚集是提高聚集效率的关键

  ——部分物化是存储空间和响应时间二者之间的很好折衷

  - 确定要物化的方体子集或子立方体
  - 在查询处理时利用物化的方体或子立方体
  - 在装入和刷新时，有效地更新物化的方体或子立方体

部分物化

- 有些OLAP产品采用启发式方法选择方体和子立方体。一种较流行的方法是物化这样的方体集，其他经常引用的方体是基于它们的
- 还有一种方法是计算冰山立方体（icebery cube）
  - —冰山立方体是一个数据立方体，只存放其聚集值（如count）大于某个最小支持度阈值minsup的立方体单元
  - —如仅考虑记录个数大于minsup的维方体，采用SQL模拟，对应的语句如下

```
SELECT 日期, 地点, 商品, COUNT(*)
FROM 销售表
GROUP BY 日期, 地点, 商品
HAVING COUNT(*)>=minsup
```

数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

数据立方体的压缩存储

- 通常数据立方体中包含海量数据，为了节省存储空间，人们提出了各种数据压缩方法
- 对于给定的数据立方体，由于度量的不同值个数可能很大，难以压缩存储，主要是对维进行压缩存储

维 1	维 2	维 3	维 4
DMP1	40	CMOS	Yes
DMP2	20	CMOS	Yes
DMP3	20	TTL	Yes
DMP4	20	TTL	Yes
DMP5	28	CMOS	No
DMP6	40	CMOS	No
DMP7	28	CMOS	No

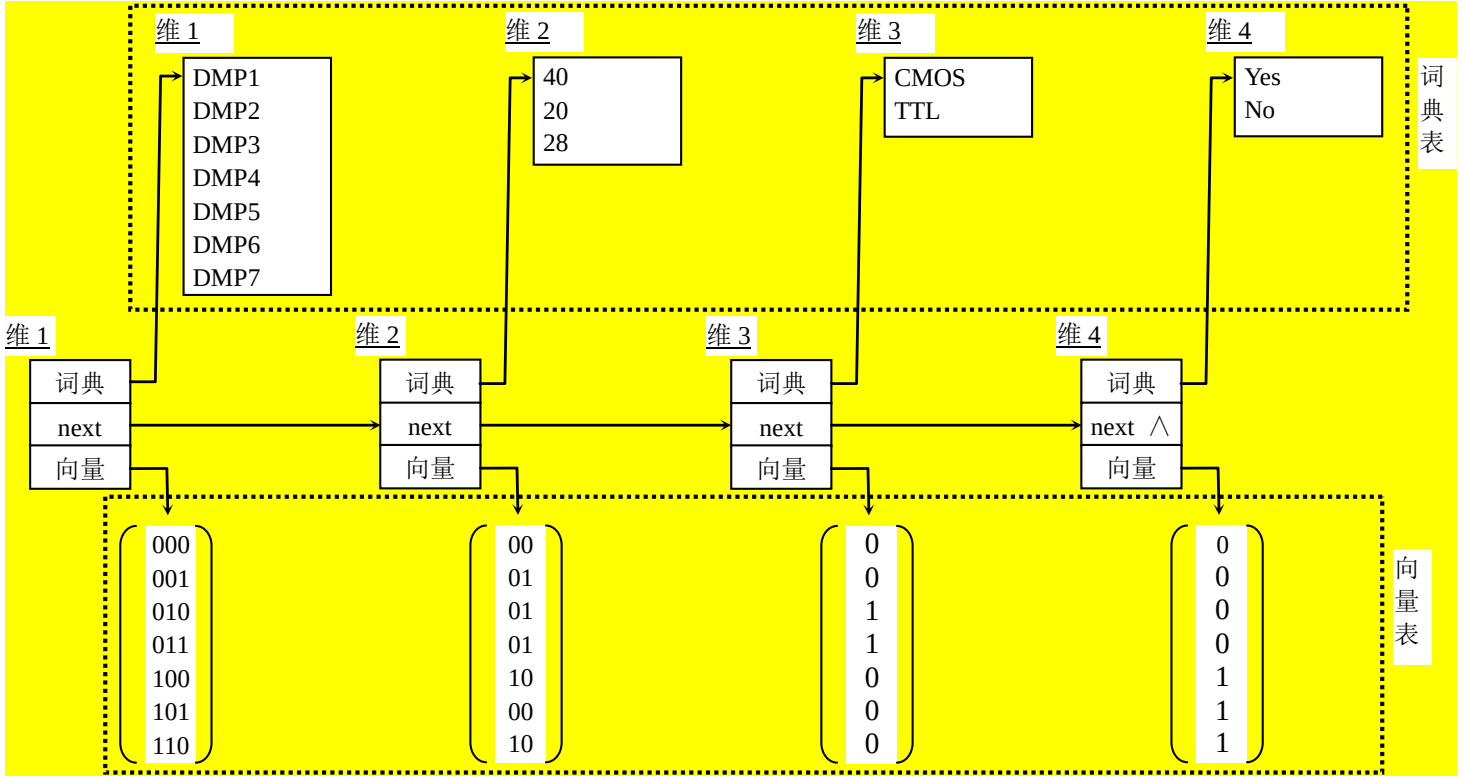
压缩存储

维 1	维 2	维 3	维 4
000	00	0	0
001	01	0	0
010	01	1	0
011	01	1	0
100	10	0	1
101	00	0	1
110	10	0	1

对于数据立方体的一个维度d_i，求出其中不同的维成员个数为$|d_i|$，采用二进制编码，对应的二进制位数为m_i，它为满足条件$\geq |d_i|$的最小m_i，对于左图所示的数据立方体，m_1=3，m_2=2，m_3=1，m_4=1，编码后的数据立方体如右图所示

数据立方体的压缩存储

- 为方便存取，设计对应的压缩存储结构如图，词典表中包含各维成员的编码表
  - —维2的编码表中，40对应编码为00，其物理地址也为00，这设计便于高效查找
  - —向量表中包含各维对应的事实数据，以编码方式存放，通过相同地址关联，如维1向量的第2行、维2向量的第2行、维3向量的第2行、维4向量的第2行构成第2个事实元组



数据仓库实现

- 数据仓库包含海量数据，为了提高OLAP服务的速度，必须进行高效的数据立方体计算
  - —计算方法
  - —存取方法
  - —查询处理方法

Indexing OLAP Data: Bitmap Index

- Index on a particular column在特定栏上的索引
- Each value in the column has a bit vector: bit-op is fast这一栏上的每一个值都对应于一个位向量
- The length of the bit vector: # of records in the base table位向量的长度：基本表中特定栏属性值的个数
- The i-th bit is set if the i-th row of the base table has the value for the indexed column如果基本表中的给定行的属性值为v，则在为图索引的对应行，表示改值的位为1，该行的其它位均为0
- not suitable for high cardinality domains对于基数较大的域不大适合
  - A recent bit compression technique, Word-Aligned Hybrid (WAH), makes it work for high cardinality domain as well [Wu, et al. TODS'06]

Base table

Cust	Region	Type
C1	Asia	Retail
C2	Europe	Dealer
C3	Asia	Dealer
C4	America	Retail
C5	Europe	Dealer

Index on Region

RecID	Asia	Europe	America
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1
5	0	1	0

Index on Type

RecID	Retail	Dealer
1	1	0
2	0	1
3	0	1
4	1	0
5	0	1

索引OLAP数据：位图索引

- 通过实例看位图索引
  - — 该表的最后两列，分别是针对性别为“男”和性别为“女”的Bitmap索引的具体取值
  - — 在针对性别取值为“男”的Bitmap索引中，当该记录的性别字段的取值为“男”的时候，其取值为1，否则为0，每个记录只需要一个比特(Bit)保存索引信息
    - 由于Bitmap索引出现大量的连续的0和1，再经过压缩，其空间占用一般很小
  - — 上述Bitmap索引，支持查询条件为“User. sex = ‘男’ ”这样的查询
    - 通过扫描针对性别字段取值为“男”的Bitmap索引，可以快速定位符合条件的记录为1号、4号和6号

Bitmap 索引实例

序号	用户 ID	用户名	性别	出生年月	其他字段	“男”Bitmap 序列	“女”Bitmap 序列
1	5	Tom	男	1980.01	…	1	0
2	6	Mary	女	1980.02	…	0	1
3	7	June	女	1979.12	…	0	1
4	14	Mark	男	1979.11	…	1	0
5	15	Kate	女	1985.03	…	0	1
6	16	John	男	1982.02	…	1	0
7	…						

索引OLAP数据：位图索引

- **高基数字段的位图索引**
  - —分容器Bitmap索引，也就是需要对数值范围进行分段
  - —建立Bitmap索引时，不是为字段的每个不同值建立索引，而是针对每个容器建立索引，也就是对整个值域的一个子集范围进行索引
  - —比如，对于资产总额字段，资产范围[0,100)，我们建立了[0,10), [10, 20),…,[90,100)等片段(假设最大资产数量小于100万)

表格数据

用户 ID	姓名	性别	出生年月	总资产	其它字段	总资产区间
5	Tom	男	1980.01	22 万	…	[20,30)
6	Mary	女	1980.02	57 万	…	[50,60)
7	June	女	1979.12	33 万	…	[30,40)
14	Mark	男	1979.11	25 万	…	[20,30)
15	Kate	女	1985.03	42 万	…	[40,50)
16	John	男	1982.02	53 万	…	[50,60)

总资产字段的 Bitmap 索引

[20, 30)	[30, 40)	[40, 50)	[50, 60)
1	0	0	0
0	0	0	1
0	1	0	0
1	0	0	0
0	0	1	0
0	0	0	1

查询条件“25 <= 总资产 <=56”对应的范围

索引OLAP数据：位图索引

- 高基数字段的位图索引
  - 当查询“25 <= 总资产 <=56”的用户的时候，[30, 40)、[40, 50)两个片段对应的记录肯定符合条件，把两者的Bitmap索引通过或操作(Bit-wise OR)进行合并，就可以寻访到符合条件的记录
  - 而[20, 30)、[50, 60)两个片段包含的记录则有可能符合查询条件，也可能不符合查询条件，把两者的Bitmap索引通过或操作进行合并，寻访到相应记录后，进一步检查，确认该记录是否真正符合“25 <= 总资产 <=56”的条件

表格数据

用户 ID	姓名	性别	出生年月	总资产	其它字段	总资产区间
5	Tom	男	1980.01	22 万	…	[20,30)
6	Mary	女	1980.02	57 万	…	[50,60)
7	June	女	1979.12	33 万	…	[30,40)
14	Mark	男	1979.11	25 万	…	[20,30)
15	Kate	女	1985.03	42 万	…	[40,50)
16	John	男	1982.02	53 万	…	[50,60)

总资产字段的 Bitmap 索引

[20, 30)	[30, 40)	[40, 50)	[50, 60)
1	0	0	0
0	0	0	1
0	1	0	0
1	0	0	0
0	0	1	0
0	0	1	0

查询条件“25 <= 总资产 <=56”对应的范围

索引OLAP数据：位图索引

基本表

编号	*A*	*B*	*C*
1	H	X	Y
2	S	X	Y
3	H	Y	N
4	C	X	N

维 A 位图索引表

编号	*H*	*S*	*C*
1	1	0	0
2	0	1	0
3	1	0	0
4	0	0	1

共有 4 行

每行对应 3 个二进制位

维 B 位图索引表

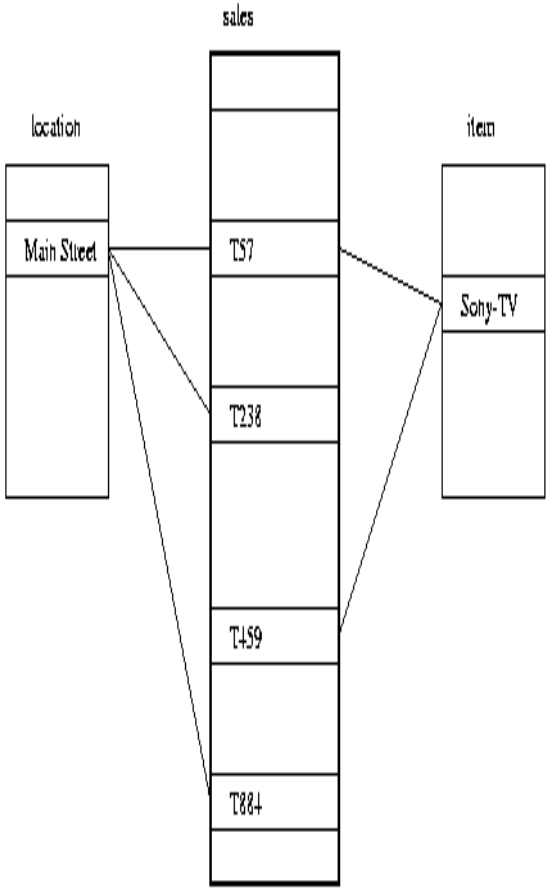
编号	*X*	*Y*
1	1	0
2	1	0
3	0	1
4	1	0

维 C 位图索引表

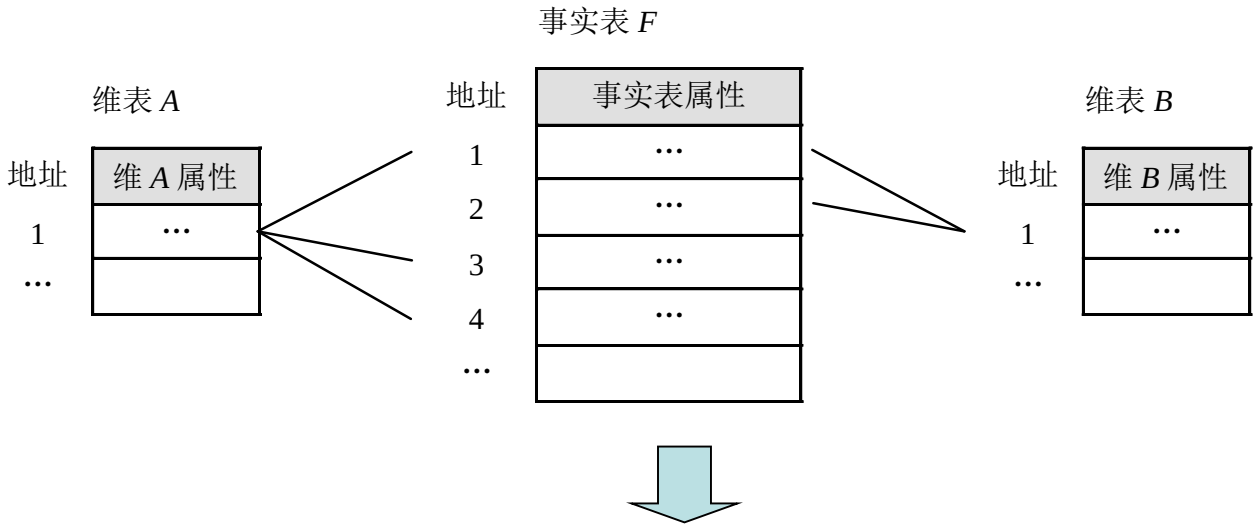
编号	*Y*	*N*
1	1	0
2	1	0
3	0	1
4	0	1

Indexing OLAP Data: Join Indices连接索引

- Join index: JI(R-id, S-id) where R (R-id, …) ⋈ S (S-id, …)
- 连接索引: 如果两个关系R(RID ,A)和S(B,SID)在属性A和B上连接，则连接索引记录包含JI（RID,SID)对，其中RID和SID分别来自R和S的记录标识符
- Traditional indices map the values to a list of record ids
  - It materializes relational join in JI file and speeds up relational join
- 传统的索引将给定列上的值映射到具有该值的列表上
  - 它物化了JI文件中的关系连接并加速了关系连接——这是一个相当昂贵的操作
- In data warehouses, join index relates the values of the dimensions of a start schema to rows in the fact table.
  - E.g. fact table: *Sales* and two dimensions *city* and *product*
    - A join index on *city* maintains for each distinct city a list of R-IDs of the tuples recording the Sales in the city
  - Join indices can span multiple dimensions
- 在数据仓库中，连接索引把星型模式的维值连接到事实表的行
  - 例如：对sales 事实表和city,product 两个维,城市的连接索引使每一个不同的城市有一个元组的记录标识符列表记录个城市的销售
  - 连接索引可以跨越多维



索引OLAP数据：连接索引



维 *A*/事实表
连接索引表

维表 *A*	事实表 *F*
1	1
1	3
1	4
…	

维 *B*/事实表
连接索引表

维表 *B*	事实表 *F*
1	1
1	2
…	

维 *A*/维 *B* 事实表
连接两个维的连接事实表

维表 *A*	维表 *B*	事实表 *F*
1	1	1
…		

Efficient Processing OLAP Queries

- Determine which operations should be performed on the available cuboids
  - Transform drill, roll, etc. into corresponding SQL and/or OLAP operations, e.g., dice = selection + projection
- Determine which materialized cuboid(s) should be selected for OLAP op.
  - Let the query to be processed be on {*brand, province_or_state*} with the condition "*year = 2004*", and there are 4 materialized cuboids available:
    1) {*year, item_name, city*}
    2) {*year, brand, country*}
    3) {*year, brand, province_or_state*}
    4) {*item_name, province_or_state*} where *year = 2004*

    Which should be selected to process the query?
- Explore indexing structures and compressed vs. dense array structs in MOLAP

OLAP查询的有效处理

- 物化方体和索引OLAP数据的目的是加快数据立方体查询处理的速度。通常，OLAP查询处理的步骤如下：
  ① 确定哪些操作应当在可利用的方体上执行：这涉及将查询中的选择、投影、上卷（分组）和下钻操作转换成对应的SQL或OLAP操作。例如，数据立方体的切片和切块可能对应于物化方体上的选择或投影操作
  ② 确定相关操作应当使用哪些物化的方体：这涉及找出可能用于回答查询的所有物化方体，使用方体之间的依赖关系，剪去上集合，评估使用剩余物化方体的代价，并选择代价最低的方体

OLAP查询的有效处理

- 例如，对于定义一个形式为“Sale_cube[Dates，Products，Locates]：SUM(销售量)”的数据立方体，所用的维层次
  - —对于Dates维是：日期≤月份≤季度≤年份≤All”
  - —对于Products维是“型号≤品牌≤子类≤分类≤All”
  - —对于Locates维是“地址≤县≤市≤省份≤地区≤All”
- 假定对{品牌，省份}处理查询，选择常量为“year=2013”。还假定有4个物化的方体可用，4个方体应当选择哪一个处理查询？
  - —方体1：{年份，型号，市}
  - —方体2：{年份，品牌，地区}
  - —方体3：{年份，品牌，省份}
  - —方体4：{2013，型号，省份}

OLAP查询的有效处理

- 该查询对应的方体为{2013，品牌，省份}。由于较细粒度的数据不能由较粗粒度的数据产生，因此不能使用方体2，因为地区是比省份更一般的概念。可以用方体1、3和4处理查询，因为：
  - —它们与查询具有相同的维集合
  - — 查询中的选择子句蕴涵对方体的选择
  - — 与品牌和省份相比，这些方体中的维Products和Locates的抽象层在更细的层次

- 由各方体产生查询方体的操作如下：
  - —对于方体1：将Products维从“型号”上卷到“品牌”，将Locates维从“市”上卷到“省份”，然后对Dates维按条件“年份=2013”切片便可得到查询方体
  - —对于方体3：对Dates维按条件“年份=2013”切片便可得到查询方体
  - —对于方体4：将Products维从“型号”上卷到“品牌”便可得到查询方体
  - —方体4可能是较好的选择

OLAP Server Architectures

- Relational OLAP (ROLAP)
  - Use relational or extended-relational DBMS to store and manage warehouse data and OLAP middle ware
  - Include optimization of DBMS backend, implementation of aggregation navigation logic, and additional tools and services
  - Greater scalability
- Multidimensional OLAP (MOLAP)
  - Sparse array-based multidimensional storage engine
  - Fast indexing to pre-computed summarized data
- Hybrid OLAP (HOLAP) (e.g., Microsoft SQLServer)
  - Flexibility, e.g., low level: relational, high-level: array
- Specialized SQL servers (e.g., Redbricks)
  - Specialized support for SQL queries over star/snowflake schemas

OLAP服务器类型

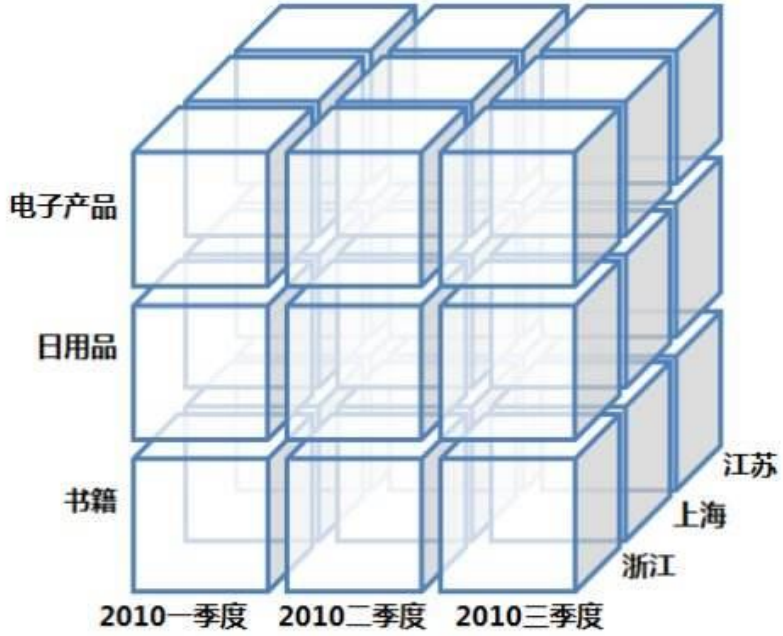
- 逻辑上，OLAP服务器从数据仓库或数据集市中给商业用户提供多维数据
- 物理上，OLAP的底层数据存储实现可以有多种不同的方式
  - —关系OLAP服务器(ROLAP)
    - 使用关系数据库或扩展的关系数据库存放并管理数据仓库的数据，而用OLAP中间件支持其余部分
    - 包括每个DBMS后端优化，聚集导航逻辑的实现，附加的工具和服务
    - 较大的可扩展性
  - —多维OLAP服务器(MOLAP)
    - 基于数组的多维存储引擎（稀疏矩阵技术）
    - 能对预计算的汇总数据快速索引
  - —混合OLAP服务器(HOLAP)
    - 结合上述两种技术，更大的使用灵活性
  - —特殊的SQL服务器
    - 在星型和雪花模型上支持SQL查询

关系OLAP服务器(ROLAP)

- ROLAP将分析用的多维数据存储在关系数据库中，每个ROLAP分析模型基于关系型数据库中一些相关的表，这些相关的表中有反映观察角度的维度表和含有度量的事实表。这些表在关系型数据库中通过外健相互关联，典型的组织模型有星形模式、雪花模式和事实星座模式
- 通过把OLAP操作表达成SQL查询的形式，在关系数据库上执行，获得汇总结果
- ROLAP主要优点是，采用关系数据库保存原始明细数据，占用空间有限
- 但由于每个OLAP操作都被转换成一个SQL查询重新执行，性能受到影响

多维OLAP服务器(MOLAP)

- MOLAP（Multidimensional OLAP）将OLAP分析所用到的多维数据，物理上存储为多维数组的形式，形成“立方体”(Cube)的结构。例如，一个包含有时间维、地区维、品牌维和销售量的数据集通过多维数组可表示成：（时间维、地区维、产品维、销售量）
- 星型模型的各个维的属性值，被映射成多维数组的下标值或下标的范围，而聚集数据作为多维数组的值存储在数组的单元中
- MOLAP的主要优点是性能高，主要缺点是占用大量的空间保存汇总数据

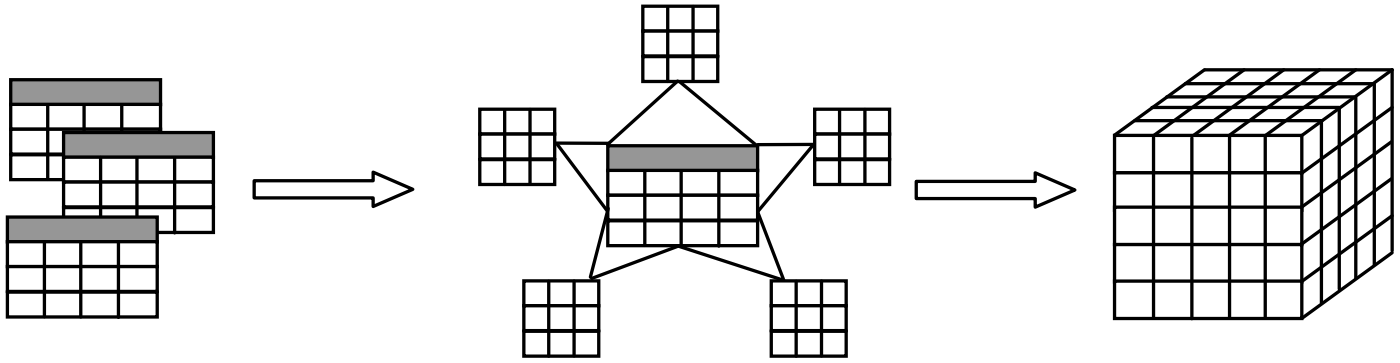


ROLAP与MOLAP比较

比较项	ROLAP	MOLAP
优点	没有存储大小限制	性能好、响应速度快
	现有的关系数据库的技术可以沿用	专为OLAP所设计
	对维度的动态变更有很好的适应性	支持高性能的决策支持计算
	灵活性较好，数据变化的适应性高	支持复杂的跨维计算
	对软硬件平台的适应性好	支持行级的计算
缺点	一般比MOLAP响应速度慢	增加系统培训与维护费用
	系统不提供预综合处理功能	受操作系统平台中文件大小的限制
	关系SQL无法完成部分计算	系统所进行的预计算，可能导致数据爆炸
	无法完成多行的计算	无法支持数据及维度的动态变化
	无法完成维之间的计算	缺乏数据模型和数据访问的标准

星形模式就是关系数据库和数据立方体的桥梁

- 多维数据库在数据存储及数据聚集上都有着关系数据库不可比拟的一些优点，实际上，星形模式就是关系数据库和数据立方体的桥梁，建立数据仓库的星形模式后，就可以在关系数据库中模拟数据的多维分析



混合OLAP服务器(HOLAP)

- 人们提出一个新的OLAP结构——混合型OLAP(Hybrid OLAP，HOLAP)，它能把MOLAP和ROLAP两种结构的优点结合起来
- 通常将聚合后的、粒度较大的高层数据存储在多维数据库中，粒度较小的细节层数据存储在关系型数据库中
- 这种HOLAP具有更好的灵活性

Chapter 5: Data Warehousing and On-line Analytical Processing

- Data Warehouse: Basic Concepts
  数据仓库：基本概念
- Data Cube and OLAP
  数据仓库建模：数据立方体与 OLAP
- Data Warehouse Design and Usage
  数据仓库的设计与使用
- Data Warehouse Implementation
  数据仓库的实现
- **Summary**

Summary

- Data warehousing: A multi-dimensional model of a data warehouse
  - A data cube consists of *dimensions* & *measures*
  - Star schema, snowflake schema, fact constellations
  - OLAP operations: drilling, rolling, slicing, dicing and pivoting
- Data Warehouse Architecture, Design, and Usage
  - Multi-tiered architecture
  - Business analysis design framework
  - Information processing, analytical processing, data mining, OLAM (Online Analytical Mining)
- Implementation: Efficient computation of data cubes
  - Partial vs. full vs. no materialization
  - Indexing OALP data: Bitmap index and join index
  - OLAP query processing
  - OLAP servers: ROLAP, MOLAP, HOLAP

References (I)

- S. Agarwal, R. Agrawal, P. M. Deshpande, A. Gupta, J. F. Naughton, R. Ramakrishnan, and S. Sarawagi. On the computation of multidimensional aggregates. VLDB'96
- D. Agrawal, A. E. Abbadi, A. Singh, and T. Yurek. Efficient view maintenance in data warehouses. SIGMOD'97
- R. Agrawal, A. Gupta, and S. Sarawagi. Modeling multidimensional databases. ICDE'97
- S. Chaudhuri and U. Dayal. An overview of data warehousing and OLAP technology. ACM SIGMOD Record, 26:65-74, 1997
- E. F. Codd, S. B. Codd, and C. T. Salley. Beyond decision support. Computer World, 27, July 1993.
- J. Gray, et al. Data cube: A relational aggregation operator generalizing group-by, cross-tab and sub-totals. Data Mining and Knowledge Discovery, 1:29-54, 1997.
- A. Gupta and I. S. Mumick. Materialized Views: Techniques, Implementations, and Applications. MIT Press, 1999.
- J. Han. Towards on-line analytical mining in large databases. *ACM SIGMOD Record*, 27:97-107, 1998.
- V. Harinarayan, A. Rajaraman, and J. D. Ullman. Implementing data cubes efficiently. SIGMOD'96

References (II)

- C. Imhoff, N. Galemmo, and J. G. Geiger. Mastering Data Warehouse Design: Relational and Dimensional Techniques. John Wiley, 2003
- W. H. Inmon. Building the Data Warehouse. John Wiley, 1996
- R. Kimball and M. Ross. The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling. 2ed. John Wiley, 2002
- P. O'Neil and D. Quass. Improved query performance with variant indexes. SIGMOD'97
- Microsoft. OLEDB for OLAP programmer's reference version 1.0. In http://www.microsoft.com/data/oledb/olap, 1998
- A. Shoshani. OLAP and statistical databases: Similarities and differences. PODS'00.
- S. Sarawagi and M. Stonebraker. Efficient organization of large multidimensional arrays. ICDE'94
- P. Valduriez. Join indices. ACM Trans. Database Systems, 12:218-246, 1987.
- J. Widom. Research problems in data warehousing. CIKM'95.
- K. Wu, E. Otoo, and A. Shoshani, Optimal Bitmap Indices with Efficient Compression, ACM Trans. on Database Systems (TODS), 31(1), 2006, pp. 1-38.

数据仓库以及 OLAP 实验

- 数据仓库构建实验
  - 运用ETL工具对数据库sakila中的数据进行清洗操作，构建一个DVD租赁商店数据仓库系统，实现定期从源数据库sakila中抽取增量数据，转换成符合DVD租赁业务的数据，最后加载到目标数据仓库中
- OLAP 实验
  - 进行的 OLAP 操作（上卷、下钻、旋转、切块、切片等）